

Python for RTL Verification

Contents

Python for RTL Verification

- Contents

Introduction

- The book assumes you know how to program

 - Code examples

- Why Python and why UVM?

 - The methodology wars

 - What about VHDL?

 - Why Python?

 - Why cocotb?

 - Why UVM and pyuvvm?

- Summary

Python concepts

- Python objects

 - Functions vs methods

- Dynamic typing

- Summary

Python basics

- Built-in constants

 - None

 - True/False

- Built-in number types

 - bool

 - int

 - float

 - complex

- Operators

 - Bitwise operators

 - Conditional operators

 - The id() function

 - Combining conditions

 - Augmented assignments

- Constructors

 - Setting variables to constants

 - Calling constructors

- Comments

- Summary

Conditions and loops

- if Statements

 - elif replaces case and switch

- Block comments and indentation

- Conditional assignment

- while loops

 - The continue statement

 - The break statement

- Summary

Python sequences

- Immutable sequence operations
- Mutable sequence operations
- The for loop
- Summary

Tuples

- Creating tuples
- Assigning tuple values
- Accessing individual tuple items
- Looping through a tuple
- Tuples are immutable
- Returning multiple values
- Summary

Ranges

- Creating ranges
- Using a range in a for loop
- Summary

Strings

- Triple quotes and triple double-quotes
- String operations
- String methods
- Formatted Strings
- Summary

Lists

- Three ways to create lists
 - List comprehension
- Sorting a list using a method
- Sorting a list using a function
- Summary

Sets

- Creating sets
- Set Operations
 - Checking set memberships
 - Checking for overlap
 - Comparison Operators
 - Combination Operations
- Removing Duplicates
- Summary

Commonly used sequence operations

- Checking for membership
- Finding the number of items in a sequence
- Concatenating sequences
- Repeating sequences
- Indexing into a sequence
- Emulating a two-dimensional array
- Create a new sequence from an existing sequence using a slice
- Access a range of sequence objects using a step greater than one
- Find the maximum and minimum objects in a sequence
- Find the index of a value in a sequence
 - Finding the first instance of a value
 - Finding the first instance of a value after a specified index
 - Finding the index of an item in a range
- Count the number of values in a sequence
- Summary

Modules

- A quick word about scopes
- What happens when you import a module?
- Importing a module with the import statement
 - Aliasing a module
- Importing resources to our dir() list
- Importing all resources from a module
 - Python coding style shuns import *
- Finding modules
- Summary

Functions

- Defining a function
- Functions are callable objects
 - Naming convention in this book
- Arguments
 - Defining Arguments
 - Default and non-default arguments
- Calling functions using arguments
- Defining a variable number of arguments
- Documentation strings
- Summary

Exceptions

- Catching exceptions
 - Catching multiple exceptions
 - Raising an exception after processing it
 - Printing an exception object
- Finally! A finally block
- The assert statement
 - The assert statement is not a function
- Summa2ry

Dictionaries

- Creating a Dictionary
 - Create and fill an empty dictionary
 - Create a dictionary using the data that will fill it
 - Use dictionary comprehension
- Handling missing keys
 - The get() method
 - The setdefault() method
- Dictionaries and for loops
 - The .keys(), .values(), and .items() iterators
- Removing items from a dictionary
- Summary

Generators

- Summary

Classes

- Defining classes and instantiating objects
- Functions vs. methods
- Defining method attributes
- The self variable
- Initializing an object
- Class variables
- Class methods
- Static methods

Summary

Protecting attributes

Protected attributes

Private attributes

Properties

Summary

Inheritance

Inheritance in Python

The super() function

Calling methods using class objects

Multiple Inheritance

Multiple Inheritance and **init()**

Summary

Design patterns

The singleton pattern

The factory pattern

Summary

Coroutines

An event loop for simulation

Defining coroutines

Awaiting simulation time

Starting tasks

Ignoring a running task

Awaiting a running task

Running tasks in parallel

Returning values from tasks

Killing a task

Summary

cocotb Queue

Task communication

Blocking communication

An infinitely long queue

A Queue of size 1

Queues and simulation delay

Nonblocking communication

Summary

Simulating with cocotb

Verifying a counter

cocotb triggers

Testing reset_n

Importing modules and objects

The tinyalu_utils module

The get_int() function

The logger object

Starting the clock and running the test

Passing (or failing) a test

Checking that the counter counts

A common coroutine mistake

Summary

Basic testbench: 1.0

The TinyALU

A cocotb testbench

Importing modules

- The Ops enumeration

- The alu_prediction() function

- Setting up the cocotb TinyALU test

- Sending commands

- Sending a command and waiting for it to complete

- Checking the result

- Finishing the test

- Summary

TinyAluBfm

- The TinyALU BFM coroutines

- The tinyalu_utils module

- Living on the clock edge

- The TinyAluBfm singleton

- Initializing the TinyAluBfm object

- The reset() coroutine

- The communication coroutines

- result_mon()

- cmd_mon()

- cmd_driver()

- Launching the coroutines using start_soon()

- Interacting with the bfm loops

- get_cmd()

- get_result()

- send_op()

- The cocotb test

- Sending commands

- Monitoring the commands

- Summary

Class-based testbench: 2.0

- The class structure

- The BaseTester class

- The RandomTester

- The MaxTester

- The Scoreboard class

- Initialize the scoreboard

- Define the data-gathering tasks

- The Scoreboard's start_tasks() function

- The Scoreboard's check_results() function

- The execute_test() coroutine

- The cocotb tests

- Summary

Why UVM?

- How do we define tests?

- How do we build testbenches?

- How do we reuse testbench components?

- How do we create verification IP?

- How do multiple components monitor the DUT?

- How do we create stimulus?

- How do we share common data?

- How do we modify our testbench's structure in each test?

- How do we log messages?

- How do we pass data around the testbench?

- Summary

uvm_test testbench: 3.0

The HelloWorldTest class

Refactoring testbench 2.0 into the UVM

The BaseTest class

The RandomTest and MaxTest classes

Summary

uvm_component

1. build_phase(self)

2. connect_phase(self)

3. end_of_elaboration_phase(self)

4. start_of_simulation_phase(self)

5. run_phase(self)

6. extract_phase(self)

7. check_phase(self)

8. report_phase(self)

9. final_phase(self)

Running the phases

Building the testbench hierarchy

The uvm_component instantiation arguments.

TestTop (uvm_test_top)

MiddleComp (uvm_test_top.mc)

BottomComp (uvm_test_top.mc.bc)

Running the simulation

Summary

uvm_env testbench: 4.0

Converting the testers to UVM components

BaseTester

RandomTester and MaxTester

__init__() and UVM components

Scoreboard

Using an environment

Creating RandomTest and MaxTest

Summary

Logging

Creating log messages

Logging Levels

Setting Logging Levels

Logging Handlers

logging.StreamHandler

logging.FileHandler

Adding a handler

Removing a handler

Removing the default StreamHandler

Disabling logging

Changing the log message format

Summary

ConfigDB()

A hierarchy-aware dictionary

The ConfigDB().get() method

The ConfigDB().set() method

Wildcards

Global data

Longest path wins

- Parent/child conflicts

- Summary

Debugging the ConfigDB()

- Missing data

 - Catching exceptions

- Printing the ConfigDB

 - Debugging parent/child conflict

- Tracing ConfigDB() operations

- Summary

The UVM factory

- The create() method

- uvm_factory()

 - Creating objects using uvm_factory()

 - Overriding types

- Factory overrides by instance

- Using the create() method carefully

- Debugging uvm_factory()

 - Logging uvm_factory() data

- Summary

UVM factory testbench: 5.0

- AluEnv

- RandomTest

- MaxTest

- Summary

Component communications

- Why use TLM 1.0?

 - Operations

 - Styles of operations

- Ports

- Exports

 - uvm_tlm_fifo

- Nonblocking communication in pyuvm

- Debugging uvm_tlm_fifo

- Summary

Analysis ports

- The uvm_analysis_port

- Extend the uvm_analysis_export class

 - Forgetting to override write()

- Extend the uvm_subscriber class

- Instantiate a uvm_tlm_analysis_fifo

 - Providing the analysis_export data attribute

- Summary

Components in testbench 6.0

- The testers

 - BaseTester

 - RandomTester and MaxTester

- Driver

- Monitor

- Coverage

- The Scoreboard

 - Scoreboard.build_phase()

 - Scoreboard.connect_phase()

 - Scoreboard.check_phase()

Summary

Connections in testbench 6.0

The AluEnv TLM diagram

AluEnv.build_phase()

AluEnv.connect_phase()

RandomTest and MaxTest

Summary

uvm_object in Python

Creating a string from an object

Comparing objects

Copying and cloning

Copying with Python

Using super() in methods

Copying using the UVM

do_copy()

Summary

Sequence testbench: 7.0

UVM Sequence Overview

Driver

AluEnv

AluSeqItem

Creating sequences

BaseSeq

RandomSeq and MaxSeq

Starting a sequence in a test

BaseTest

RandomTest and MaxTest

Summary

Fibonacci testbench: 7.1

Fibonacci numbers

FibonacciSeq

Driver

Sequence timing

AluEnv

Summary

get_response() testbench: 7.2

AluResultItem

Driver

FibonacciSeq

get_response() pitfalls

Summary

Virtual sequence testbench: 8.0

Launching sequences from a virtual sequence

Running sequences in parallel

Creating a programming interface

OpSeq

TinyALU programming interface

FibonacciSeq

Summary

The future of Python in verification

Leverage the existing SystemVerilog ecosystem.

Create a Python verification ecosystem.

Bringing EDA into the 21st century

Acknowledgements

Appendix

Other Books by Ray Salemi

The Tucker Mysteries (Writing as Ray Daniel)

Copyright

License

Indemnification

Introduction

Python for RTL Verification is a book for verification engineers who want to use Python and Python engineers who want to learn the Universal Verification Methodology (UVM).

New applications such as machine learning are forcing verification engineers to develop new verification techniques and leverage broader ecosystems of libraries. As the most popular programming language on the planet, Python has the capacity and ecosystem to solve modern verification challenges. Testbench developers will either access Python libraries from SystemVerilog or VHDL or write their testbenches in Python.

This book teaches you how to write testbenches in Python. It teaches you enough Python to use the `pyuvvm` and `cocotb` modules to create the next generation of testbenches.

The book assumes you know how to program

Python for RTL Verification assumes that the reader wants to add Python or the UVM to their existing programming skillset. A reader who knows Python can skip to the **cocotb** chapters, and a reader who knows **cocotb** can skip to the **pyuvvm** chapters.

Code examples

You can get a copy of the code examples in this book at pythonverification.com. There are two types of examples, **Jupyter Notebook** examples and directory examples. The Jupyter Notebook examples demonstrate the Python language. The directory examples are in directories demonstrate **cocotb** interacting with a simulator. We store these in directories. The instructions to run **cocotb** examples are also in `README.md`.

The example files and directories have the same names as the chapters, and the numbers at the beginning the names cause them to print in the same order as the book chapters when you list them alphabetically.

Each example has a figure number that matches the example in the book to the example in the code. The numbers reset each chapter. For example, figure 1 demonstrates the classic first program we write in all languages. It is in the file `00_Introduction.ipynb`.

All the book's code examples use the same format, a figure number and description, the code, then a line with a separator `--`, followed by the output.

```
# Figure 1: The classic first program
```

```
print("Hello, world.")
```

```
--
```

```
Hello, world.
```

Why Python and why UVM?

There is an urban legend that the width of the Space Shuttle's solid rocket booster was determined by the width of a train tunnel it had to pass through on the way to the launch pad. That width was determined by the width of the train tracks, which was determined by the length of the axles, which was ... etc. The story goes back and back until we get to the width of the ruts left by a Roman chariot. A similar story could be told about the languages most of us use to verify a design written in a register transfer language (RTL). SystemVerilog and VHDL testbenches are the product of decades of incrementalism.

The 1980s saw a revolution in digital design methodologies with the introduction of register transfer languages such as Verilog (created by Gateway Design Automation) and VHDL (created by a US Government military specification for documentation but leveraged to create synthesizable code ¹).

We have argued whether VHDL or Verilog is the better hardware design language for thirty years. But one cannot claim that either of them was initially designed to create testbenches. Testbench development is a software problem not readily solved by hardware design languages. Several approaches have recognized and addressed the need for a testbench language.

In 1996, a startup named InSpec took the most obvious approach and announced an entirely new language named e . Explicitly designed for verification, e introduced the industry to object-oriented and aspect-oriented programming. It also championed what was then an advanced verification technique, constrained-random verification, implemented with a methodology called the e Reuse Methodology (eRM).

e gained popularity, but it was limited in its growth by its very newness. Teams that adopted e often needed considerable consulting help to create their first testbenches. The other problem was that e was proprietary to Verisity (the renamed InSpec). You needed to buy Verisity's software, and only their software, to run an e testbench.

Three years later, another startup named Co-Design Automation announced SUPERLOG, which extended the standard Verilog language to provide verification-friendly concepts such as interfaces. Co-Design immediately put SUPERLOG into the public domain, intending to create an open ecosystem.

The competition between e and SUPERLOG tilted in SUPERLOG's favor when Synopsys bought Co-Design. Synopsys donated its verification languages Vera and SUPERLOG to the Accellera standards body. Accellera created SystemVerilog by merging Vera, SUPERLOG, and the ForSpec assertion language.

The methodology wars

Having an object-oriented language cannot, by itself, lead to dramatic reuse improvements since any two engineers are likely to solve the same problem differently, thus defeating reuse efforts. One needs a documented methodology that is standard across the EDA industry.

In 2005 and 2006, the EDA industry's three dominant software companies responded to the need for a single, compatible methodology by creating three incompatible methodologies:

- Cadence, who had bought Verisity, delivered the Universal Reuse Methodology (URM)
- Synopsys, who had bought Co-Design, delivered the Verification Methodology Manual (VMM)
- Mentor Graphics, who had committed to SystemVerilog, delivered the Advanced Verification Methodology (AVM)

These three companies vied for dominance, trying to convince customers to rewrite testbenches from one methodology to the other. Unfortunately (from the EDA customer's point of view), incompatible methodologies made it hard to switch simulators. This angered simulator customers who craved healthy competition between their simulation vendors.

Having three standard methodologies wasn't serving anyone. So in 2008, Mentor Graphics and Cadence merged their methodologies to create the Open Verification Methodology (OVM). methodology (OVM). Once two EDA vendors make a move, the third is sure to follow. So, when Cadence and Mentor Graphics donated the OVM to Accellera, Synopsys joined the team to create the Universal Verification Methodology (UVM).

In 2017, ten years after creating the first methodologies, the IEEE released the 1800.2 Spec, codifying the UVM.

What about VHDL?

Meanwhile, VHDL was following its path. While the US Government had dropped the requirement to write RTL in VHDL, there was still a substantial library of existing IP and a community of VHDL engineers. VHDL RTL was never going away, so the VHDL community turned its eyes to the problem of writing testbenches in VHDL. Still, the effort never got the sort of object-oriented boost seen with Verilog, now called *SystemVerilog*.

The VHDL engineering community sidestepped the methodology wars for many years, allowing engineers to devise unique solutions to creating VHDL testbenches.

This state of affairs changed in 2012 when Jim Lewis created the Open Source VHDL Verification Methodology (OSVVM), a VHDL-based methodology that provided constrained-random constructs and a standardized way of encapsulating functionality. In 2015 Espen Tallaksen created the Universal VHDL Verification Methodology (UVM) ². As of this writing, engineers are using both methodologies.

Why Python?

It is natural to think of testbenches as hardware test fixtures, and so one would design “hardware” that drives signals into the device under test (DUT) and checks results.

However, this is a rudimentary way of thinking about testbenches. It can work for the most uncomplicated cases, but it is quickly overwhelmed by larger designs.

It is better to think of testbenches as software that communicates with the DUT through drivers. The drivers, in this case, are APIs that communicate with a bus functional model.

In Figure 2, we see a DUT with two interfaces named A and B.

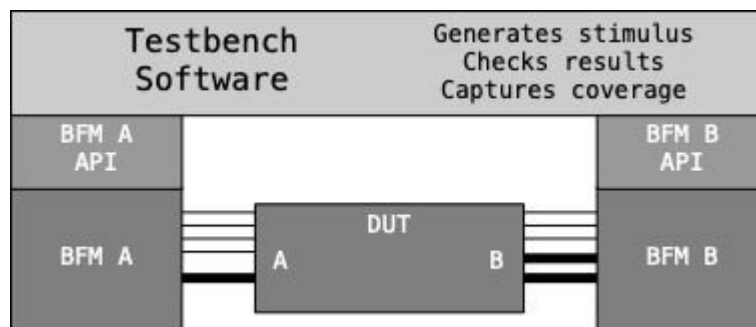


Figure 2: Testbench software with bus functional models

This diagram illustrates the basic testbench building blocks:

- **The device under test (DUT)** —This is the hardware block or chip we’re verifying. We instantiate the DUT in the testbench and connect testbench signals to its inputs and outputs.
- **Bus functional models (BFMs)** —Bus functional models connect to the DUT and implement the communication protocols the DUT expects. For example, you’d have a USB bus functional model if the DUT talks using the USB protocol. The diagram calls the interfaces A and B.
- **BFM application programming interface (API)** —Programming interfaces are blocking function calls that connect the testbench software to an RTL simulation. One can implement them using threads, SystemC, or, in our case, cocotb.

- **Testbench software** —This software creates operations and sends them to the DUT through the BFM. It reads results using the BFM and compares the actual results to the expected results. It also checks that we've verified all the functionality in the DUT.

Once we understand that our testbench is software, we have to ask, “Why are we writing complicated software using a register transfer language, even an enhanced RTL such as SystemVerilog?” The answer is the same as why the size of the space shuttle's solid rocket boosters was related to the axle length of a Roman chariot—incrementalism.

The EDA industry created the UVM so that engineers could use a supercharged register transfer language, running in an event-driven simulator, to write software. But, a register transfer language, even a supercharged one, is not the best tool for software development.

It makes more sense to use a powerful software programming language to write powerful software. Python is a powerful programming language, and packages such as cocotb and pyuvvm allow us to create powerful testbenches.

Why cocotb?

cocotb stands for Coroutine CoSimulation TestBench. Python developers added coroutines to Python to provide a mechanism for event-driven code. For example, you might want code to respond when a user presses a key on a keyboard.

cocotb's developers recognized that event-driven Python code would work well with an event-driven simulator. So they created a package that interacts with a running simulator. **cocotb** beautifully links Python code to the simulator, and so it was the perfect foundation for **pyuvvm**.

Why UVM and pyuvvm?

As we saw previously, the UVM is an amalgam of methodologies. Mentor Graphics, Cadence Design Systems, and Synopsys brought their contributions to Accellera, and Accellera created a class library uniquely suited to the problems of IC verification.

The UVM is a flexible class library that supports creating reusable hierarchical testbenches in which block-level testbenches quickly become monitoring resources when instantiated in larger testbenches. It has a powerful stimulus-generating system that hides testbench complexity from test writers. And it provides a standardized Verification IP (VIP) structure that allows VIP vendors to ensure that their offerings will fit into any UVM testbench.

What makes the UVM challenging to use often has more to do with the limitations of SystemVerilog than the inherent complexity of the class library. For example, the obscure incantations needed to operate the UVM factory or configuration database are responses to the strong typing that permeates SystemVerilog.

pyuvvm implements the IEEE 1800.2 UVM specification in Python. We'll see that problems such as typing complexity don't exist in Python, making reuse more straightforward.

Combining a world-class software development language with a world-class verification class library creates a world-class solution. One you will learn to use in this book.

Summary

Python combined with the UVM is a powerful way to create testbenches. This book teaches you enough Python to create testbenches using cocotb and pyuvvm . You can find the example at www.pythonverification.com .

Python concepts

To appreciate the difference between Python and SystemVerilog or VHDL, one has to indulge in a little bit of computer language history and remember that programming started with engineers writing machine code to manipulate bits. Eventually, the machine code was abstracted to assembly code, and the assembly code was abstracted to programming languages. But after compilation, all of these languages boiled down to manipulating bits.

The challenge in manipulating bits is that the data they represent come in many sizes. For example, an `int` in C contains 16 bits, and a `char` contains 8 bits. Data would get lost if a programmer accidentally wrote 16 bits of an `int` into the 8 bits reserved for a `char`.

Most programming languages address this problem with types. Variable types define the memory size associated with variables and check that programs transfer data between compatible variables, also known as compatible types.

Languages such as Pascal and VHDL are strict when it comes to typing. For example, they issue a syntax error if you try to assign variables of different types, even if the sizes are the same.

On the other hand, C and SystemVerilog are more permissive, allowing you to transfer data between different types of compatible sizes. In SystemVerilog, for example, you can put an 8-bit `byte unsigned` into a 16-bit `int unsigned`, and SystemVerilog will automatically set the extra eight bits to zeros. This can be dangerous since SystemVerilog will happily chop off the top eight bits if you put a 16-bit `int unsigned` into an 8-bit `byte unsigned`.

Typing becomes a language challenge when creating higher-level data structures. For example, if you create a SystemVerilog FIFO that transfers `int` data, you don't want to copy and modify the FIFO to create one for `byte` data. Instead, you want to reuse the code and change only the type.

Languages such as SystemVerilog and C++ address this problem using parameterization. You write the FIFO once, but leave the data's type flexible until you declare a variable as in the SystemVerilog in figure 1.

```
// Figure 1: SystemVerilog parameterized declarations
```

```
my_FIFO #(int) int_fifo;  
my_FIFO #(byte) byte_fifo;
```

SystemVerilog calls the spot between the `#()` a *parameter*, while VHDL calls the equivalent thing a *generic*. Managing parameters can make it challenging to learn the SystemVerilog UVM.

Python objects

We've seen that languages such as SystemVerilog use types to define the number of bits needed to store a datum. Python sidesteps the problem of counting bits because assignments in Python do not transfer bits. They transfer objects. More precisely, Python variables store object handles.

Everything we manipulate in Python is an object. An object is an instance of a class. A class defines how an object behaves. All objects of the same class behave the same way. We'll see later that we can define classes, but we will focus on Python's built-in classes, for now, the ones that come defined with the language.

Languages such as C++ differentiate between *types* and *classes*. However, in Python, types are the same things as classes. So we use the `type()` function to find an object's class.

For example, let's consider the number 5 . Using the type function in figure 2, we see that the number 5 is an instance of class `int` . We call instances of classes objects .

```
# Figure 2: Constants are objects of type int

print(type(5) )
--
<class 'int'>
```

The number 5 is an object of class `int` .

Since everything in a Python program (classes, functions, constants, types, etc.) is an object, we don't have to worry about data size when transferring data between variables. And that means no type checking and no parameterization.

Consider a situation where we want to store a floating-point number in a variable containing an integer. Unfortunately, this transfer can't work in a typed language because the memory size is different for floating-point and integer.

Figure 3 illustrates a Python example with a variable named `AA` that stores an integer and a variable named `BB` that stores a floating-point number. We see that when we assign `BB` to `AA` , Python copies the handle to the object, not the data.

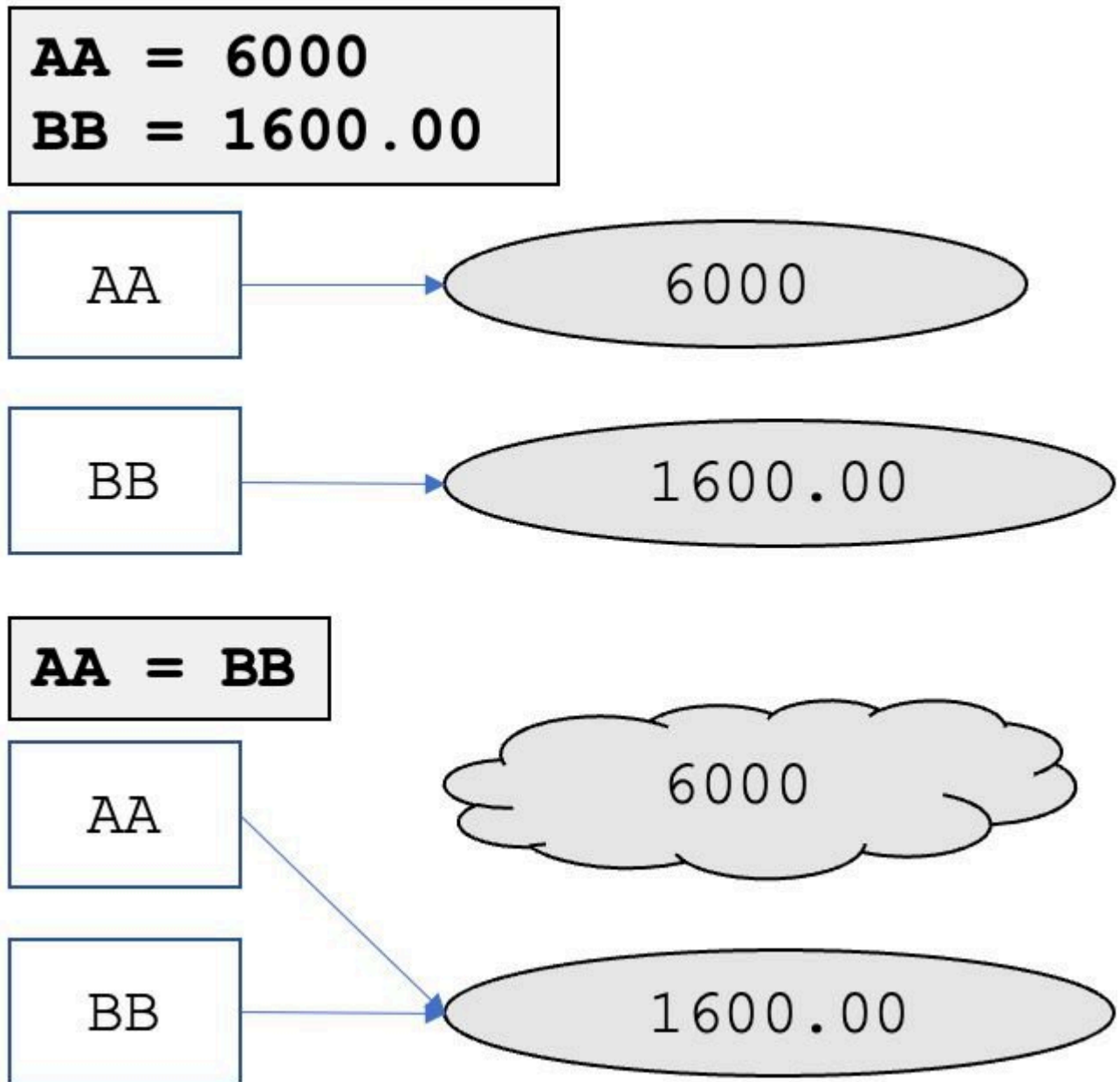


Figure 3: Storing handles not data

Python disposes of the 6000 during periodic garbage collection. We can implement the scenario in figure 3 using the code in figure 4.

```
# Figure 4: Storing handles not data
```

```
AA = 6000
BB = 1600.0
print("Type of A:", type(AA) )
print("Type of B:", type(BB) )
AA = BB
print("AA is now", AA)
print("New type of A", type(AA) )
--
Type of A: <class 'int'>
Type of B: <class 'float'>
AA is now 1600.0
New type of A <class 'float'>
```

Python does the right thing because we're not transferring numbers. Instead, we are transferring handles to numbers.

Functions vs methods

Functions and methods are both subroutines that return a value when you call them. The difference between them is that we call functions directly, whereas we call methods in the context of a given object. Methods are functions defined inside a class.

For example, figure 5 creates a string (type `str`) object by setting a variable (`msg`) to a string. Then we'll do the following:

- Pass `msg` to the `print()` function to print it.
- Call the `type()` function to get the type of `msg` and pass that to the `print()` function. We see that `msg` is an instance of the `str` class.
- The `str` class defines a function named `endswith()`. (A function defined within a class is called a *method*.) `endswith()` returns `True` if its argument matches the end of the string. Notice that we call `endswith()` using the `msg` variable. We can do this because `endswith()` is a method in the `str` class.

```
# Figure 5: Calling a str method
msg = "Dropped the pots with a bang"
print(msg)
print(type(msg) )
print("msg ends with a bang:", msg.endswith("a bang") )
--
Dropped the pots with a bang
<class 'str'>
msg ends with a bang: True
```

Using the `endswith()` method

The `msg.endswith("a bang")` line demonstrates another feature of methods. They can operate on the object that calls them. The `msg.endswith("a bang")` automatically compares "a bang" to `msg`. There is no need to put both in the argument list.

Dynamic typing

SystemVerilog and VHDL force their programmers to ask permission before accessing data. The programmers must declare a variable to hold the data, and the compiler checks the types on the assignment statement. This declaration-based typing is called static typing.

Python uses dynamic typing. Dynamic typing allows programmers to ask forgiveness rather than permission. It allows us to attempt any operation on any object. Python raises an error if the object cannot implement the operation.

The code in figure 6 tries to use a string method on an integer. The `endswith()` returns `True` if the string ends with the argument. This works when we call `mystring.endswith()` because `str` defines `endswith()`. If we try to call `myint.endswith()`, we get an error message.

```

# Figure 6: Calling an undefined method
mystring = "Hello, world"
print(mystring.endswith("orld"))
myint = 42
print(myint.endswith("a whimper"))
--
True
-----
AttributeError Traceback
2 print(mystring.endswith("orld"))
3 myint = 42
----> 4 print(myint.endswith("a whimper"))
AttributeError: 'int' object has no attribute 'endswith'

```

The `AttributeError` is called an exception and we'll look at them in more detail later. The exception stops the program at the spot where we transgressed. We'll see later how to ask forgiveness when an operation raises an exception.

Summary

Python for RTL Verification teaches enough Python for a verification engineer to create standard testbenches using the UVM.

Python is different than SystemVerilog and VHDL in that it is dynamically typed. Every variable stores a handle to an object, and everything is an instance of an object. Dynamic typing makes it easier to create testbenches and results in a more straightforward implementation of the UVM in Python than in SystemVerilog.

Of course, there will be runtime bugs that would have been caught by static typing. It's been my experience that these errors are rare enough that the work of matching parameters is not worth the time lost on a testbench that takes more than a minute to compile.

Python basics

Since everything in Python is an instance of a class, Python defines a set of built-in classes, such as `str` and `int` alongside built-in constants. We'll examine these in this chapter.

The word *type* is interchangeable with class, though it usually refers to a built-in class.

Built-in constants

When you say `byte xx = 5` in a language such as SystemVerilog, the program sets aside an 8-bit memory location and stores `0x05` in it. When you say `xx = 5` in Python, the program instantiates an instance of an `int` class of value `5`. The `xx` variable gets the handle to that `int` object.

Once Python instantiates a constant value, it uses the same handle whenever any other part of your program uses that constant. In addition to user-defined constants such as `5`, Python has built-in constants. Here is a list of some of them.

None

Use this object to initialize an empty variable. Functions with no useful return value such as `print()` also return `None`.

True/False

These are Boolean constants that you can test in a conditional. If you pass a value to a conditional statement, it gets converted to one of these constants. For example, `None` always converts to `False`.

Built-in number types

Here are the most common built-in number types in Python.

bool

The `bool` type has two self-explanatory constants `True` and `False`.

int

A numeric value without a decimal point. There is no maximum `int` in Python.

float

A numeric value with a decimal point. An `int` value converts to a `float` when in an operation with a `float`. The division operator (`/`) always returns a `float`.

```
# Figure 1: A float in operations means float output
ii = 1
print("type of ii", type(ii) )
ff = 2.0
print("type of ff", type(ff) )
ss = ii + ff
print("type of ss", type(ss) )
```

```
dd = ii/ii
print("type of dd", type(dd) )
--
type of ii <class 'int' >
type of ff <class 'float'>
type of ss <class 'float'>
type of dd <class 'float'>
```

complex

Not used in verification often, but cool nonetheless. Any number with a `j` (engineering style imaginary notation) is `complex`.

```
# Figure 2: Complex numbers!
print(1j * 1j)
--
(-1+0j)
```

Operators

Python has all the same operators as SystemVerilog and VHDL. You can read the details of all the operators (and anything Python) at docs.python.org. This section provides a reference overview of operators often used in verification.

Bitwise operators

Python has the same bitwise operators as SystemVerilog or C.

Operator	Description	Example
A B	Bitwise OR	101 010 = 111
A ^ B	Bitwise XOR	101 ^ 111 = 010
A & B	Bitwise AND	101 & 011 = 001
A << N	Left shift A by N	001 << 2 = 100
A >> N	Right shift A by N	100 >> 1 = 0101
~ A	Invert all bits	~101 = 010

Table 1: Bitwise operators

Conditional operators

Python has the same conditional operators as SystemVerilog and C with the addition of `is`.

Operator	Description	Example
A is B	True if variables A and B contain handles of the same object. Usually used with None	A = None A is None returns True
A == B	Is equal to	5 == 5 returns True
A != B	Is not equal to	5 != 'five' returns True
A < B	Strictly less than	5 < 5 returns False
A <= B	Less than or equal to	5 <= 5 returns True
A > B	Strictly greater than	5 > 5 returns False
A >= B	Greater than or equal to	5 >= 5 returns True

Table 2: Conditional operators

The `is` operator checks that the two objects being compared are the same object. You should use `is` to check whether a variable contains `None`.

```
# Figure 3: The 'is' operator checks for object identity
A = None
if A is None:
    print("A is, in fact, None")
--
A is, in fact, None
```

The id() function

This is an appropriate moment to talk about the `id()` function. Every object in a Python program has a unique identification number. If two variables contain objects with the same ID number, they contain handles to the same object.

For example, we put the constant `5` into `xx` and `yy` and then see that the constant is the same object in both variables. We can check this using the `is` operator.

```
# Figure 4: The id() function returns a unique id for each object
xx = 5
yy = 5
print ("ID of xx", id(xx) )
print ("ID of yy", id(yy) )
print ("xx is yy", xx is yy)
--
ID of xx 4378986976
ID of yy 4378986976
xx is yy True
```

Combining conditions

The condition combination operators in Python are different than the SystemVerilog combination operators. Python uses the words `and`, `or`, and `not` as follows.

```
# Figure 5: Compound conditional operations
print(" True and True:", True and True)
print(" True or False:", True or False)
print("True and not (1 == 2) :", True and not (1 == 2) )
--
True and True: True
True or False: True
True and not (1 == 2) : True
```

Python is efficient in that it only evaluates enough conditions to tell whether the overall condition will be `True` or `False`.

- The `or` operator only evaluates the second half of an expression if the first half is `False` since, if the first half of the expression is `True`, the second half is irrelevant.
- The `and` operator only evaluates the second half of an expression if the first half is `True` since, if the first half is `False`, the second half is irrelevant.
- The `not` operator is evaluated after conditional expressions. `not 1 == 2` acts like `not (1 == 2)`.

This behavior allows us to test that an operation is legal before trying it. For example, figure 6 assumes that we can increment `xx`, but since `xx` is `None`, we get a `TypeError` exception. Here is the error.

```
# Figure 6: Trying to increment None
xx = None
if (xx + 1) == 5:
    print("True")
else:
    print("False")
-----
TypeError Traceback
1 xx = None
----> 2 if (xx + 1) == 5:
3 print("True")
4 else:
5 print("False")
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

Figure 7 avoids the error by checking that `xx` is not `None` in the first half of the conditional statement and taking advantage of the fact that Python stops evaluating as soon as `xx is not None` returns `False`.

```
# Figure 7: False first "and" expression
xx = None
if xx is not None and (xx + 1) == 5:
    print("True")
else:
    print("False")
--
False
```

Augmented assignments

Languages such as SystemVerilog and C have a `++` operator that increments a variable. If we have `xx` set to `1`, the `xx++` operation sets `xx` to `2`.

Python has a generalized version of this operation. It allows you to use any operator before the `=` to change a value. Figure 8 shows us that if we have `xx` set to `1` we can increment `xx`, multiply it, and divide it using augmented assignments.

```
# Figure 8: Augmented assignments
xx = 1
print("type(xx) :", type(xx) )
print("xx:",xx)
xx += 1
print("xx += 1:",xx)
xx *= 3
print("xx *= 3:",xx)
xx /= 4
print("xx /= 4:", xx)
print("xx ends as ", xx)
print("type(xx) :", type(xx) )
--
type(xx) : <class 'int'>
xx: 1
xx += 1: 2
xx *= 3: 6
xx /= 4: 1.5
xx ends as 1.5
type(xx) : <class 'float'>
```

Notice that `xx` changed from `1` to `2` to `6` to `1.5` over the course of these operations. `xx` also changed type from `int` to `float` due to the `/=` operator.

Constructors

Everything in Python is an object, which raises a question about how to create these objects. There are two ways:

- Setting variables to constants
- Calling constructors

Setting variables to constants

The simplest way to create an object is to set a constant to a variable. Figure 9 shows that we can set a variable to `5` and get an `int` object, or we can set a variable to `None` and get a `NoneType` object.

```
# Figure 9: Creating objects as constants
int_obj = 5
none_obj = None
print("type(int_obj) :", type(int_obj) , "value:", int_obj)
print("type(none_obj) :", type(none_obj) , "value:", none_obj)
--
type(int_obj) : <class 'int'> value: 5
type(none_obj) : <class 'NoneType'> value: None
```

Calling constructors

A constructor is a function that creates an object of a specific type. We can call a constructor using the type name as a function call with trailing parentheses. The argument is the value we want in the object.

One typical use for calling constructors is to “convert” a variable from one type to another. I use quotation marks around “convert” because we’re not really converting the object; we’re creating a new object using the old object in the constructor’s argument list.

Figure 10 shows that we can create a `float` from a `str`. Python makes sure that the value in the argument makes sense for the type you’re creating. We see that the string “3.14159” can convert to a `float` but not an `int`.

Notice that Python does not cast a variable to be a different type, instead it creates a new object using the first variable as an argument to the constructor.

```
# Figure 10: Creating a number from a string
pi = float("3.14159")
print("type(pi) :", type(pi) , "pi:", pi)
no_pi_for_you = int("3.14159")
--
type(pi) : <class 'float'> pi: 3.14159
-----
ValueError
1 pi = float("3.14159")
2 print("type(pi) :", type(pi) , "pi:", pi)
----> 3 no_pi_for_you = int("3.14159")
ValueError: invalid literal for int() with base 10: '3.14159';
```

Comments

Comments are our gift to future programmers (unless we let them get out of date, but that is a different story.)

Python provides two kinds of comments, line comments and block comments.

To create a line comment, use the `#` character. Python ignores everything on a line after the `#`. So in this example, we have one line that is all comment and another that is an assignment followed by a comment.

```
# Figure 11: Various comments
# This is a comment
PI = 3.14159 # Comment to end of line
```

To create a block comment, use a set of triple double-quotes at the beginning of the block comment and the end of the block comment. For example, in this block comment, we define `PI`.

```
# Figure 12: Multiline string as a comment
"""
PI is the circumference of a circle
divided by its diameter
"""
```

There is a difference between the `#` comment and the triple double-quote comment. The first is an actual comment, ignored by Python. The second is a Python convention that uses an unassigned multi-line string as a comment, though this distinction does not affect your program.

Summary

The chapter discussed the nuts and bolts of Python:

- constants—Objects containing a fixed value
- built-in types—As opposed to the user-defined types we'll see later
- operators—These create new objects by combining existing objects
- constructors—Create new objects given zero or more arguments

Conditions and loops

This chapter will look at Python's conditional operators and conditional expressions. We'll also examine an advanced topic—creating definitions of how our objects respond to comparisons.

if Statements

`if` statements consist of a conditional test and a block of code to run if the condition evaluates as `True`. Python is famous for designating its blocks of code using indentation rather than curly braces as in C or `begin` and `end` as in SystemVerilog.

Python does not require parentheses in the `if` statement, unlike C and SystemVerilog. Instead, you write the conditional expression, using parentheses for clarity if necessary. As with all blocks, we use a colon to designate a block as we see in Figure 1.

```
# Figure 1: A Python if statement
name = "Roy"
if name != "Danny":
    print("Hey, you're not Danny.")
--
Hey, you're not Danny.
```

It is best to use spaces for indentation instead of tabs and you should use four spaces or the space police will come for you. It is illegal to have different indentations on different lines in the same file. Most editors handle spaces for you.

The `else` statement must align with the `if` statement.

```
# Figure 2: The else statement
name="Danny"
if name != "Danny":
    print("Hey, you're not Danny.")
else:
    print("Hello, Danny!")
--
Hello, Danny!
```

elif replaces case and switch

`case` statements in SystemVerilog and `switch` statements in C compare a variable to a set of values and do different things based on which value the variable matches.

Python generalizes this idea with the `elif` statement. As the name implies, `elif` combines `else` and `if` so that you can make multiple cascading comparisons.

```
# Figure 3: Using elif as a switch
A = B = 5
operation = "divide"
if operation == "add":
    print("A + B =", A + B)
elif operation == "subtract":
    print("A - B =", A - B)
elif operation == "multiply":
    print("A * B = ", A * B)
else:
    print("Illegal Operation:", operation)
--
Illegal Operation: divide
```

Thus the `elif` provides similar functionality to case and switch statements, but with more flexibility at the cost of more code.

Block comments and indentation

In the *Python concepts* chapter, we talked about block comments that are captured between triple double-quotes (`""" ... """`). We noted that this statement defines an unassigned multi-line string. Since Python sees the string and executes it, we must indent the string within a block as with any other statement.

```
# Figure 4: Indenting block comments
name = "Danny"
if name != "Danny":
    """
    This is the code
    we run if this isn't Danny
    """
    print("Hey, you're not Danny.")
else:
    print("Hello, Danny!")
--
Hello, Danny!
```

Conditional assignment

Ternary operations assign one of two expressions to a variable based on a conditional value. In SystemVerilog and C, they look like the code in figure 5.

```
# Figure 5: Ternary operation in SystemVerilog
my_reg = (aa == 5) ? five_val : other_val;
```

Figure 5 checks that `aa` is a 5 and sets `my_reg` to the variable `five_val` if the condition is `True` . Otherwise, it sets `my_reg` to `other_val` .

Python also has a ternary assignment but without the weird `?` operator as in figure 6.

```
# Figure 6: Ternary operation in Python
aa = 5
message = "five_val" if aa == 5 else "other_val"
print(message)
--
five_val
```

while loops

Python has two looping statements:

- the `for` statement (discussed in the following chapter) and
- the `while` statement.

The `while` statement starts with a conditional expression and loops for as long as the expression returns `True`. The looping ends when the condition returns `False` or the code reaches a `break` statement.

Figure 7 uses a loop to print the integers from 0 to 13. (The `print()` function has an `end` argument that replaces the `\n` (carriage return) with whatever you specify. In figure 7, we use a space `" "`).

```
# Figure 7: A while loop in action
nn = 0
while nn <= 13:
    print(nn, end=" ")
    nn += 1
--
0 1 2 3 4 5 6 7 8 9 10 11 12 13
```

The continue statement

The `continue` statement jumps back to the top of the loop and rechecks the condition. Figure 8 prints the odd integers between 1 and 13 by using `continue` to skip over the even integers.

```
# Figure 8: continue jumps back to the top
nn = 0
while nn <= 13:
    nn += 1
    if (nn % 2) == 0:
        continue
    print(nn, end=" ")
--
1 3 5 7 9 11 13
```

The break statement

Languages such as SystemVerilog and C have a “do . . . while” loop that is guaranteed to run once, then check to see whether it should loop again.

Python has no such loop, but you can emulate one using the `break` statement. Figure 9 prints out the numbers 0 to 13 by breaking when we see the 14.

```
# Figure 9: Breaking out of a loop
```

```
nn = 0
```

```
while True:
```

```
    print(nn, end=" ")
```

```
    nn += 1
```

```
    if nn == 14:
```

```
        break
```

```
--
```

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13
```

Summary

Conditions test values. You can use them in `if` statements, `elif` constructs, and loops. Python contains only `for` loops and `while` loops. You can use these to recreate the kinds of loops available in other languages.

Python has no `case` or `switch` statement. Instead, it uses the `elif` and `else` constructs.

Python uses indentation to designate code blocks rather than curly brace or words such as `begin` and `end` .

Python sequences

Many algorithms boil down to looping through a series of objects and operating on them. This programming approach is so common that software developers created the concept of an *iterable*. An iterable object can provide one datum at a time when asked.

A deck of playing cards is a good example of an iterable. It contains 52 cards, and when you ask for the next card, you get the one on the top of the deck.

An *iterator* is an object that takes an iterable and provides one item at a time until all the items are used up. For example, you've created an iterator when you shuffle a deck of cards. Now the dealer (who is acting as an iterator) can get cards from the top of the deck until there are no more cards.

Python *sequences* are iterables that store ordered data and provide the data one at a time. Sequences come in two flavors:

- **immutable** —These cannot be changed once created. Immutable classes feature a standard set of operations.
- **mutable** —You can add and remove elements from these objects or sort them. Mutable objects feature all the operations in immutable objects plus the `add`, `remove`, and `sort` operations.

Immutable sequence operations

Python has operations that work for any immutable sequence. Figure 1 is a table of operations with examples. The examples use a string object (type `str`) as the immutable sequence example. Notice that we can apply the sequence methods directly to a constant because the constant is an object. In the *Strings* chapter, we will discuss the `str` class.

Operator	Description	Example
x in s	Contains	"s" in "Miss" -> True
x not in s	Does not contain	"y" not in "Miss" -> True
s + t	Concatenate	"Miss" + "ed me." -> "Missed me."
s * n n * s	add to self n times	"x" * 5 = "xxxxx" 5 * "x" = "xxxxx"
s[ii]	iiith item of s, origin 0	"Miss"[1] -> "i"
s[ii:jj]	slice of s from ii to jj-1	"Miss"[1:3] -> "is"
s[ii:jj:kk]	slice from ii to jj-1 step kk	"0123456"[1:6:2] -> "135"
len(s)	length of s	len("0123456") -> 7
min(s)	smallest item of s	min("abc123") -> "1"
max(s)	largest item of s	max("abc123") -> "c"
s.index(x[,i[,j]])	index of first s after i and before j	"abcdef".index("d") -> 3 "xyzxzy".index("z",3) -> 4
s.count(x)	total number of occurrences of x in s	"Mississippi".count("s") -> 4

Figure 1: Immutable operations

Mutable sequence operations

As we'll see in later chapters, the `set` and `list` types are mutable sequences. Mutable sequences support all the immutable operations in Figure 1 and the mutable operations in figures 2 and 3. The following table examples use a list, which is like an array. The *Lists* chapter will detail lists. These examples use an example list named `l1` that contains `["a", "b", "c", "d", "e"]`. Notice the use of square brackets to access slices of the list.

Operator	Description	Example
s[ii] = x	Item ii replaced by x	ll[1] = 'B' ['a', 'B', 'c', 'e', 'f']
s[ii:jj] = t	items ii:jj-1 replaced by iterable t	ll[2:4] = "XY" ['a', 'b', 'X', 'Y', 'e']
del s[ii:jj]	items ii to jj-1 removed	del ll[2:3] ['a', 'b', 'd', 'e']
s[ii:jj:kk] = t	items ii to jj-1 replaced by items in t, step kk	ll[0:5:2]="123" ['1', 'b', '2', 'd', '3']
del s[ii:jj:kk]	Delete items ii up to jj-1, step by kk	del ll[0:5:2] ['b', 'd']

Figure 2: Mutable sequence operations

The methods in Figure 3 modify mutable sequences.

Operator	Description	Example
ss.append(xx)	Append xx to end of sequence ss	ll.append("f") ['a', 'b', 'c', 'd', 'e', 'f']
ss.clear()	Clears out all items	ll.clear() []
ss.copy()	Returns a shallow copy	kk = ll.copy() ['a', 'b', 'c', 'd', 'e']
ss.extend(tt) ss += tt	Adds sequence tt to ss	ll += [1,2] ['a', 'b', 'c', 'd', 'e', 1, 2]
ss *= nn	Replicate list nn times	ll = ['X'] ll *= 3 ['X', 'X', 'X']
ss.insert(ii,xx)	Insert item xx at ii	ll.insert(0, 'A') ['A', 'a', 'b', 'c', 'd', 'e']
ss.pop(ii)	Remove item at ii and return it	print(ll.pop(1)) 'b' print(ll) ['a', 'c', 'd', 'e']
ss.remove(xx)	Remove item xx from ss	ll.remove('c') ['a', 'b', 'd', 'e']
ss.reverse()	Reverse the order of the items in ss	ll.reverse() ['e', 'd', 'c', 'b', 'a']

Figure 3: Mutable methods

The for loop

We loop through sequences and other iterables using the `for` loop. It has the structure in figure 4.

```
# Figure 4: for loop structure
for <variable> in <iterable>:
    <loop body>
```

The `for` loop uses the iterable to create an iterator that deposits one value at a time into the variable, which we can use in the body.

For example, since a string is an iterable, Figure 4 prints the first four numbers using a `str` as an iterable object. In figure 5, `cc` is the variable, and `"01234"` is the `str`.

```
# Figure 5: A string as an iterable
for cc in "01234":
    print(cc, end=" ")
--
0 1 2 3 4
```

As in the `while` loop, the `continue` statement jumps to the top of the `for` loop, and the `break` statement breaks out of the loop.

The code in Figure 5 executes `continue` when we see the "1" and `break` at the "3" leaving only "0", and "2" to be printed.

```
# Figure 6: continue and break in a for loop
for cc in "01234":
    if cc == "1":
        continue
    if cc == "3":
        break
    print(cc, end=" ")
--
0 2
```

Summary

Iterators are at the heart of Python, and sequences are ordered iterators that can be looped, indexed, and searched. You can add and remove values from mutable objects such as `list` but cannot change immutable objects such as `tuple`.

You loop through an iterator using the `for` loop.

We'll examine sequences such as tuples, ranges, sets, strings, and lists in the following chapters.

Tuples

There is nothing like a tuple in SystemVerilog, VHDL, or any other languages based on Algol or CPL. A tuple is an immutable object that stores more than one value.

Creating tuples

There are several ways to create a tuple. One is to list values with commas between them. Figure 1 stores the Dynamic Duo.

```
# Figure 1: Creating a tuple
dynamic_duo = "Batman", "Robin"
print("Nananana! Batman!", dynamic_duo)
--
Nananana! Batman! ("Batman","Robin")
```

We can also explicitly put parentheses around tuple's values to match the way Python prints tuples.

```
# Figure 2: Creating a tuple using parentheses
dynamic_duo = ("Batman", "Robin")
print("Created using parentheses:", dynamic_duo)
--
Created using parentheses: ("Batman","Robin")
```

Finally, we can create a tuple from an iterator by passing it to the `tuple` constructor. Figure 3 uses a string as an iterator that returns one character at a time so that we can convert a string to a tuple full of single characters.

```
# Figure 3: Creating a tuple from an iterator
villain = tuple("Joker")
print(villain)
--
("J","o","k","e","r")
```

Assigning tuple values

We can assign a tuple to a single variable that stores the whole tuple or assign tuple values to multiple variables where each variable gets one of the tuple items. In figure 4, we assign ("Batman", "Robin") to a single variable. When we print it, we see both values in parentheses.

We can also assign the tuple to a pair of variables, `caped_crusader`, and `boy_wonder`. The first variable gets "Batman" and the second gets "Robin"

```
# Figure 4: Assigning tuple values
dynamic_duo = ("Batman", "Robin")
print("Dynamic Duo:", dynamic_duo)
# Assign each to their own variable
caped_crusader, boy_wonder = dynamic_duo
print("Caped Crusader:", caped_crusader)
print("Boy Wonder:", boy_wonder)
--
Dynamic Duo: ("Batman", "Robin")
Caped Crusader: Batman
Boy Wonder: Robin
```

Accessing individual tuple items

We can access individual elements in a tuple using the `[]` operator to create a slice as in figure 5. (Note that `print()` automatically inserts a space after the string.).

```
# Figure 5: Slicing a tuple with square brackets
dynamic_duo = "Batman", "Robin"
print("I'm", dynamic_duo[0] )
--
I'm Batman
```

Looping through a tuple

Tuples are Python sequences, so you can loop through them using a `for` loop.

```
# Figure 6: Looping through a tuple
dynamic_duo = "Batman", "Robin"
for caped_crusader in dynamic_duo:
    print("It's", caped_crusader)
--
It's Batman
It's Robin
```

Tuples are immutable

Once you create a tuple object, you cannot modify it. Figure 7 shows that Batman cannot replace Robin with Catwoman, as much as he may secretly yearn to do this.

```

# Figure 7: You cannot change a tuple
dynamic_duo = "Batman","Robin"
print("Sorry, Robin.")
dynamic_duo[1] = "Catwoman"
--
TypeError: 'tuple' object does not support item assignment
-----
TypeError Traceback
1 dynamic_duo = "Batman","Robin"
2 print("Sorry, Robin.")
----> 3 dynamic_duo[1] = "Catwoman"
TypeError: 'tuple' object does not support item assignment

```

Returning multiple values

Figure 8 demonstrates a function that returns the names of DC Comics heroes. Notice that the return statement contains three values, and the function call sets the function to three values.

```

# Figure 8: Returning a tuple
def dc_heroes() :
    return "Batman", "Superman", "Wonder Woman"
# Now call the function
human, Kryptonian, Amazon = dc_heroes()
print("Human ", human)
print("Kryptonian", Kryptonian)
print("Amazon ", Amazon)
--
Human Batman
Kryptonian Superman
Amazon Wonder Woman

```

Python functions can return any number of values using tuples. It's up to the calling code to decipher the tuple that the function returns, as happens in figure 8.

Summary

We learned about the immutable sequence type, the *tuple*. We saw that SystemVerilog, C, and VHDL, have no data type analogous to the tuple.

The tuple allows us to store multiple values in a single object. We can return a tuple from a function that must return multiple values. When you read a tuple, you can store it in a single object handle or store its values in separate variables.

Ranges

A `range` is an iterable object that returns numbers in order. By default, we create a range object by calling the `range()` constructor and passing how many numbers we want. `range(n)` starts counting at zero and stops at `n-1`.

For example, to print the numbers from `0` to `2`, we create a range using `range(3)` and loop through it with a `for` loop.

```
# Figure 1: A range iterator from 0 to 2
for ii in range(3) :
    print(ii, end = " ")
--
0 1 2
```

You'll note that I'm being careful with my language here. The `range(3)` in figure 1 does not return a tuple with three numbers. That would waste memory. Instead, `range(3)` returns an iterator, and the iterator returns the numbers one at a time.

Because a range is an iterator object and not a tuple, printing a range object does not print the numbers in the range. Instead, it prints the range's definition.

```
# Figure 2: Printing a range does not print the numbers
range5 = range(5)
print(range5)
--
range(0, 5)
```

You can print the numbers using a `for` loop, or you can use the range as an argument to create a `tuple()` and print the tuple.

```
# Figure 3: Printing a range using a tuple
range5 = range(5)
print(tuple(range5) )
--
(0, 1, 2, 3, 4)
```

We'll use tuples to print the output as we examine different ways to create ranges.

Creating ranges

By default, range iterators start returning numbers with zero and increment the numbers by one. But you can create other ranges and increments using the `range()` constructor.

`range(stop)` —This is the typical constructor call. The numbers in the range will start at 0 and end at one less than `stop`.

```
# Figure 4: The range stop argument
stop = 5
print(tuple(range(stop)))
--
(0, 1, 2, 3, 4)
```

`range(start, stop)` —The numbers will be greater than or equal to the `start` and less than the `stop`.

```
# Figure 5: The range start and stop arguments
start = 5
stop = 10
print(tuple(range(start, stop) ))
--
(5, 6, 7, 8, 9)
```

`range(start, stop, step)` —The numbers will be greater than or equal to the `start`, less than the `stop`, and incremented by the `step`.

```
# Figure 6: The range step argument
start = 1
stop = 10
step = 2
print(tuple(range(start, stop, step) ))
--
(1, 3, 5, 7, 9)
```

Using a range in a for loop

In SystemVerilog, you loop over a range of numbers as in figure 7.

```
# Figure 7: SystemVerilog for loop
for(int ii = 0; ii < 4; ii++) {
    $write("%0d ",ii) ;
}
```

You can use a range to do the same thing in Python.

```
# Figure 8: Looping through numbers using range()
for ii in range(4) :
    print(ii, end = " ")
--
0 1 2 3
```

Summary

Ranges are iterators that return numbers in order. When creating the range, you can define the starting number, stopping number, and step. They are often used to create `for` loops that use sequential numbers.

Strings

The most significant difference between strings in Python and C is that Python strings are not arrays of `char`. Instead, they are data objects that provide formatting and parsing functions.

You can define strings using either the single quote `'` or double-quote `"` characters. Having two kinds of definition characters allows you to create a string using one character that contains the other. For example, let's create a string with a quote from Morpheus and then print out the last six characters, including the ending double-quote (`"`). Notice that you can use square brackets to create slices of a string.

```
# Figure 1: Slicing the end of a string
line = "Morpheus said, "welcome to the desert of the real.""
print("End of the string with double-quote:", line[45:51] )
--
End of the string with double-quote: real."
```

Strings are immutable. You cannot change a string. In figure 2, we change `desert of the real` to `desert of the mall`.

```
Figure 2: You cannot modify a string
line = 'Morpheus said, "welcome to the desert of the real."'
line[45:49] = "mall"
--
TypeError: 'str' object does not support item assignment
```

You can't change a string, but you can create a new string that replaces characters using the `replace()` method. Here we create `mall_line` from `real_line` using `replace()`.

```
# Figure 3: Creating a new string with replaced characters
real_line = 'Morpheus said, "welcome to the desert of the real."'
print(real_line)
print("id(real_line) :", id(real_line) )
mall_line = real_line.replace("real", "mall")
print(mall_line)
print("id(mall_line) :", id(mall_line) )
--
Morpheus said, "welcome to the desert of the real."
id(real_line) : 140397805837200
Morpheus said, "welcome to the desert of the mall."
id(mall_line) : 140397805837872
```

Triple quotes and triple double-quotes

You can create strings containing carriage returns using triple single-quotes or triple double-quotes. Both options allow you to create a string containing the other quote character as in figure 4.

```
# Figure 4: A multi-line string as haiku
haiku = '''You can write haiku
Even speech is possible
Using "triple-quotes"
'''

print(haiku)
--
You can write haiku
Even speech is possible
Using "triple-quotes"
```

String operations

Strings implement all the immutable operations shown in the *Python Sequences* chapter. Here is a table of commonly used string operations.

Operator	Description	Example
A + B	Concatenates strings	"Bat"+"man" returns "Batman"
A * n	Repeats the string n times	"-" * 5 returns "-----"
A[x]	Provides the character stored at location x. The first index is 0.	"abcde"[3] returns "d"
A[x:y]	Returns a ranged slice not including y	"abcde"[1:3] returns "bc"
C in A	Returns True if string C is in string A	"ce" in "abcde" returns False "cd" in "abcde" returns True

Figure 5: Commonly used string operations

String methods

All `str` objects have methods to help manipulate string data. There are 45 methods, so buckle up, we're going to—just kidding. We will look only at the methods most commonly used when reading a file. docs.python.org documents the rest.

`str.join(<iterable>)` —Given an iterable containing strings, this method concatenates them together using the calling string. We can use `join` to combine lines from a file into a single string using `"\n".join()`.

```
# Figure 6: Joining a list of strings to make a new string
haiku_lines = ("Lines in a tuple",
"Seemingly unrelated",
"Can make a haiku")
haiku = "\n".join(haiku_lines)
print(haiku)
--
Lines in a tuple
Seemingly unrelated
Can make a haiku
```

`str.split(<separator>, maxsplit=-1)` —This function splits a string into a list using the separator string . `maxsplit` specifies the number of splits you want. For example, you may want to only split off the first word in a sentence. The `-1` default splits all occurrences. The figure 7 example splits a CSV line and stores the result into four variables.

```
# Figure 7: Splitting a string on a character
csv_line="Luke,Skywalker,Jedi,Tatooine"
first_name, last_name, job, homeworld = csv_line.split(",")
print(first_name, last_name, job, homeworld)
--
Luke Skywalker Jedi Tatooine
```

`str.splitlines(keepends=False)` —When we read a file, we receive a long string with lines separated by carriage returns and linefeeds. The `splitlines()` method splits the lines at the line ending characters and returns them as a list.

```
# Figure 8: Creating a list of strings using split()
grocery_str="bread\nmilk\neggs"
grocery_list=grocery_str.splitlines()
print("Grocery list:",grocery_list)
--
Grocery list: ['bread','milk','eggs']
```

`str.strip(<chars>)` —This method strips the leading and trailing whitespace characters from a string. It's useful for cleaning spaces from user input. In figure 9 we clean up `user_input` by removing leading and trailing spaces.

```
# Figure 9: Stripping whitespace
user_input=" run "
print('Extra Spaces:', [' '+user_input+''])
cmd = user_input.strip()
print('Cleaned up:', [' '+cmd+''])
--
Extra Spaces: [ run ]
Cleaned up: [run]
```

`str.casefold()` —Use this method to compare case-insensitive strings. `.casefold()` handles strings in non-English languages, so it is better than `lower()`. The code in figure 10 fixes the weird capitalization from someone on the internet.

```
# Figure 10: Using casefold to remove case
weird_cap = "why dO pEOpLE dO tHis?"
print("Cases folded:", weird_cap.casefold() )
--
Cases folded: why do people do this?
```

Formatted Strings

So far in this book, we've been printing output messages by calling `print()` with a string, a comma, and a value. Python has a better method for placing data into a string. While Python has developed several ways of formatting strings over the years, we're going to examine the most recent formatting tool, *Formatted string* literals or *f-strings* for short.

F-strings start with the letter `f` or `F` and then a quote or double quote or their tripled versions. The `f` tells Python to watch for expressions between curly braces `{}` in the string. Figure 11 uses an f-string to create `answer_str`, which contains the product of two complex numbers. Notice that we put `answer` in curly braces to get it into the string.

```
# Figure 11: Putting a value into an f-string
answer = 1j * (1+1j)
answer_str = f"1j * (1+1j) = {answer}"
print(answer_str)
--
1j * (1+1j) = (-1+1j)
```

This is a vast improvement over C-format strings containing placeholders. It's easy to read this string and see what the developer intended.

An f-string can contain formatting information between the curly braces ³. For example, figure 12 specifies the width of the field to make the first name, and the last name fields 12 characters wide so they line up:

```
# Figure 12: Creating fields of a fixed width
csv_line="Luke,Skywalker,Jedi,Tatooine"
first_name, last_name, job, homeworld = csv_line.split(",")
print("| First Name | Last Name | ")
print(f"| {first_name:12}| {last_name:12}| ")
--
| First Name | Last Name |
| Luke | Skywalker |
```

It is also helpful to output binary data in different bases. Figure 13 prints decimal 17 in different bases. Notice that we have four ways of printing the number widths:

- `{val}` prints the default decimal base with the minimum number of digits.
- `{val:08b}` prints the number in 8 binary digits and pads the upper bits with 0's.

- `{val:3o}` prints the number in 3 octal digits but uses spaces instead of `0`'s in the unused upper digits.
- `{val:x}` prints the number in hex using the minimum number of digits.

I have added square brackets to the output in figure 13 so you can see the widths.

```
# Figure 13: Printing different bases
val = 17
print(f"dec: [{val}] bin: [{val:08b}] oct:[{val:3o}] hex: [{val:x}] ")
--
dec: [17] bin: [00010001] oct:[ 21] hex: [11]
```

Summary

Strings are immutable sequences of `str` objects. We can create strings using single or double quotes. Using single quotes allows you to put double quotes in a string and vice-versa. The triple version of both quotes allows us to create strings containing carriage returns.

Strings have methods that allow you to manipulate them. Formatted strings, or f-strings, allow us to insert values into a string.

Lists

Lists are ordered iterators of objects. Lists are mutable, so you can add objects to them, remove objects from them, reverse the order of the objects, and sort the objects. They are Python's workhorse.

Three ways to create lists

The first way to create a list is to use square brackets as in `LL = [1,2,3]`. Using empty square brackets `[]` gives you an empty list. Lists can contain any objects, including other lists, as in figure 1, where we put one list into another.

```
# Figure 1: Creating a list containing a list
LL = ['x','y'] # create a list
mylist = ['a',3,LL] # put it into another list
print("List within a list:", mylist)
--
List within a list: [ 'a' , 3 , ['x','y'] ]
```

The second way to create a list is by using the `list` constructor. The constructor takes any iterable object, loops over it, and puts the returned objects in a new list. If you don't pass an argument to `list()`, you get an empty list.

```
# Figure 2: Using the list() constructor
empty = list()
one_to_four = list(range(1,5) )
print("empty:", empty)
print("one_to_four:", one_to_four)
--
empty: []
one_to_four: [1, 2, 3, 4]
```

The third way to create a list is by using a *list* comprehension . List comprehensions are a compact way to create a list that you could create using a `for` loop. Let's look at this construct in more detail.

List comprehension

Let's create a list by squaring only the even numbers between zero and ten in figure 3. First, we create an empty list. Then we use a `for` loop to process the numbers from `0` to `10`. Finally, if the number is even, we use the `append()` method from the list of methods available in mutable types to add it to the list.

```
# Figure 3: Creating a list using a for loop
even_squares = []
for nn in range(11) :
    if nn % 2 == 0:
        even_squares.append(nn**2)
print("even squares", even_squares)
--
even squares [0, 4, 16, 36, 64, 100]
```

We've used the `for` loop to build a list from an iterator, but that's a lot of typing to initialize a list.

The list comprehension statement strips the `for` loop down to its basics in one statement. List comprehension statements have four parts contained in square brackets as we see in figure 4.

```
# Figure 4: List comprehension structure
[ <expression on var> for <var> in <iterator> if <filter expression> ]
```

Let's use list comprehension to rewrite the `for` loop in figure 4, printing even squares from 0 to 10 using a lot less code:

- `<operation on var>`—An operation of some sort that takes *as an input*, (`nn**2`).
- `<var>`—A variable that holds the objects from the iterator (`nn`)
- `<iterator>`—The iterator that provides the objects (`range(11)`)
- `<filter expression>`—A conditional that filters the `var` objects allowing you to only operate on some of them (`nn % 2 == 0`)

This gives us the code in figure 4.

```
# Figure 4: Replace the for loop using list comprehension
even_squares = [nn**2 for nn in range(11) if nn % 2 == 0]
print("even squares", even_squares)
--
even squares [0, 4, 16, 36, 64, 100]
```

While it's cool to do this all on one line, it can be hard to read that line, so Python allows you to write a list comprehension statement using multiple lines.

```
# Figure 5: Multiple lines for readability
even_squares = [nn**2
                for nn
                in range(11)
                if nn % 2 == 0]
print("even squares", even_squares)
--
even squares [0, 4, 16, 36, 64, 100]
```

Sorting a list using a method

You can sort a list using the `sort()` method, which sorts the objects in the list. In figure 6 we make a list of the digits in Pi and sort them.

```
# Figure 6: Sorting a list in place
pi_list = list("3.1415926")
print(" pi chars:", pi_list)
pi_list.sort()
print("sorted pi chars:", pi_list)
--
pi chars: ['3', '.', '1', '4', '1', '5', '9', '2', '6']
sorted pi chars: ['.', '1', '1', '2', '3', '4', '5', '6', '9']
```

The `sort()` method takes two arguments.

```
<list>.sort(<key>=None, <reverse>=False)
```

- `key`—This is a function that takes a list item as an input and returns a value to sort. It defaults to `None`.
- `reverse`—This is a boolean that says whether the sort should be reversed. It defaults to `False`.

As an example of using the key, let's say we want to sort some of the Avengers in a list alphabetically:

`['antman', 'Hulk', 'Ironman']`. Figure 7 shows that the capitalization causes a sorting error.

```
# Figure 7: Badly-sorted Avengers
avengers = ["antman", "Hulk", "Ironman"]
avengers.sort()
print("Badly sorted :", avengers)
--
Badly sorted : ['Hulk', 'Ironman', 'antman']
```

The lowercase `a` comes after all the uppercase letters, so `antman` gets put at the end of the list when he should be at the beginning. We can sort the names properly by using the `str` method `casefold`, which eliminates the case.

We change the sort order by passing the `str.casefold` method as the `key` argument. Notice that we are passing the method itself—we are not calling the method. We would use parentheses if we were calling the `casefold` method (`str.casefold()`), but we're not calling it, we're passing it as an argument. The `sort()` method will call the `str.casefold` method and use the result as the key for the sort.

```
# Figure 8: Sorting correctly using a key function
avengers = ["antman", "Hulk", "Ironman"]
avengers.sort(key=str.casefold)
print("Correctly sorted:", avengers)
--
Correctly sorted: ['antman', 'Hulk', 'Ironman']
```

Sorting a list using a function

In figure 8, we sorted a list of the Avengers using the `sort()` method. The `sort()` method changed the list so that the key sorted the elements.

However, there are cases where we will want to create a new sorted list rather than change an existing list. In figure 9, we can create a `sorted_avengers` list.


```
# Figure 9: Creating a new sorted list using the sorted() function
avengers = ["antman", "Hulk", "Black widow"]
sorted_avengers = sorted(avengers, key=str.casefold)
print("Correctly sorted:", sorted_avengers)
--
Correctly sorted: ['antman', 'Black widow', 'Hulk']
```

The `sorted()` function works with any iterable. For example, we could pass it a string and get a list of sorted characters.

```
# Figure 10: Passing a key to sorted()
ss = "Brownfox"
sorted_list = sorted(ss, key=str.casefold)
print("sorted_list:", sorted_list)
--
sorted_list: ['B', 'f', 'n', 'o', 'o', 'r', 'w', 'x']
```

Summary

A `list` is an ordered iterator containing objects. Since a list is mutable, it implements all the immutable and mutable functions defined in the *Python sequences* chapter. Lists have a `sort()` method that sorts the elements in the list. The `sorted()` function takes any iterable and returns a sorted list.

Sets

Parents have been complaining about new math for as long as students have been taught math. In the 1960s, my mother complained bitterly that my math teachers were focused on sets and their operations rather than making us memorize the multiplication tables. Little did she know that my early school education would prepare me for Python programming.

Sets are unordered groups of unique items. Sets can be tested and combined with other sets using mathematical operations such as union, intersection, difference, and symmetric difference. They help us check that inputs are in a legal *set* of values, remove duplicates from a list, and find data intersections.

There are two kinds of sets:

- A `set` is a mutable object.
- A `frozenset` is an immutable object.

Creating sets

There are three ways to create a set object.

The first way is to use the `set` or `frozenset` constructors, depending upon whether you want your set to be mutable.

The constructors take an iterable as an argument and create a set by looping through it and adding unique values. So, for example, we can create a `frozenset` from a list of the first eight digits of pi.

```
# Figure 1: Creating a frozenset from a tuple
digits = (3,1,4,1,5,9,2,6)
digit_set = frozenset(digits)
print(f"digit_set: {digit_set}")
--
digit_set: frozenset({1, 2, 3, 4, 5, 6, 9})
```

Notice that there are multiple `1s` in `3.14159`, but only a single `1` got stored in the set because, by definition, sets can only have one entry for each value.

Python printed the numbers in order, but sets have no inherent order, so you cannot count on this behavior.

The second way is to create sets (but not frozensets) using curly braces.

```
# Figure 2: Creating a set using curly braces
inner_planets = {"Mercury", "Venus", "Earth", "Mars"}
print("inner_planets:", inner_planets)
--
inner_planets: {'Venus', 'Mercury', 'Mars', 'Earth'}
```

The third way is to use a set comprehension. Set comprehension works the same way as list comprehension but has curly braces instead of brackets. In figure 3, the variable `nn` loops through the iterable, and the result of the operation on `nn` gets stored in the set.

```
# Figure 3: Using set comprehension
first_squares = {nn**2 for nn in range(1,6) }
print(f"first_squares: {first_squares}")
--
first_squares: {1, 4, 9, 16, 25}
```

Set Operations

We check to see whether an object is in a set, compare sets, combine them, and use them to remove duplicates. This section shows how we do these things.

Checking set memberships

We check that a datum is in a set using the keywords `in` and `not`. So in figure 4, we check that we got a legal operation in a hypothetical ALU.

```
# Figure 4: Checking set membership
legal_ops = {"add", "and", "mul", "xor"}
print("div is legal:", "div" in legal_ops)
print("and is legal:", "and" in legal_ops)
--
div is legal: False
and is legal: True
```

Conversely, we can check that things are *not* in a set.

```
# Figure 5: Checking an element is not in a set
legal_ops = {"add", "and", "mul", "xor"}
print("You cannot div:", "div" not in legal_ops)
--
You cannot div: True
```

Checking for overlap

Two sets with no overlapping elements are said to be *disjoint*. We can check this using the `isdisjoint()` method.

```
# Figure 6: Checking that sets are disjoint
boston={"Red Sox", "Bruins", "Celtics", "Patriots"}
newyork={"Yankees", "Rangers", "Knicks", "Jets"}
print("There are no teams both cities support: ", boston.isdisjoint(newyork) )
--
There are no teams both cities support: True
```

Comparison Operators

Comparison operators allow you to compare two sets in much the same way as we compare integers. You can use these expressions in a conditional statement such as `if` or `while`.

Figure 8 uses the following sets in its example column.

```
# Figure 7: Example sets for Figure 8 and Figure 9
five = set(range(5)) # 0,1,2,3,4
ten = set(range(10)) # 0,1,2,3,4,5,6,7,8,9
even = set(range(0,10,2)) # 0,2,4,6,8
```

Notice that the `>=` and `<=` operators map to set methods that perform the same function.

Operator	Description	Example
A <= B A.issubset(B)	A is a subset of B or equal to B	<code>even <= even</code> is True <code>five <= even</code> is False
A < B	A is a proper subset of B (not equal)	<code>even < even</code> is False <code>even < ten</code> is True
A >= B A.issuperset(B)	A is a superset of B or equal	<code>five >= five</code> is True <code>five >= even</code> is False
A > B	A is a proper superset of B (not equal)	<code>even > even</code> is False <code>ten > even</code> is True

Figure 8: Set Conditional Operators

Combination Operations

You combine sets in much the same way as you combine strings of bits or integers. Set combination operations act like *or*, *and*, *subtract* and *xor* operations.

Operator	Description	Example
$A \mid B$ A.union(B)	The union of A and B	<code>five even =</code> <code>{0,1,2,3,4,6,8}</code>
$A \& B$ A.intersection(B)	The intersection of two sets	<code>five & even = {0,2,4}</code>
$A - B$ A.difference(B)	The difference between two sets	<code>ten - even =</code> <code>{1,3,5,7,9}</code>
$A \wedge B$ a.symmetrical_difference(B)	The symmetrical difference between two sets. (Returns elements in one set or the other but not both.)	<code>five ^ even =</code> <code>{1,3,6,8}</code>

Figure 9: Set operations

Removing Duplicates

Sets help you remove duplicates from a list. You convert the list to a set and back to a list. However, you lose the list order as we see in figure 10.

```
# Figure 10: Removing list duplicates using a set
birds_seen_1 = ["Robin", "Eagle", "Blue Jay"]
birds_seen_2 = ["Sparrow", "Eagle", "Crow"]
all_birds_seen = birds_seen_1 + birds_seen_2
print("Birds reported : ", all_birds_seen)
# Convert to set and back to remove duplicates
all_birds_seen = list(set(all_birds_seen) )
print("Unique birds seen: ", all_birds_seen)
--
Birds reported : ['Robin', 'Eagle', 'Blue Jay', 'Sparrow', 'Eagle', 'Crow']
Unique birds seen: ['Sparrow', 'Robin', 'Eagle', 'Crow', 'Blue Jay']
```

Summary

Sets are groups of unique objects that use the set operations some of us learned in grade school. We can check that an object is in a set, compare sets, and add and subtract them.

Commonly used sequence operations

The *Python Sequences* chapter contained immutable and mutable sequence operations tables. This chapter provides examples and details on common operations. ⁴

Checking for membership

The `in` operator returns `True` if an object is in a sequence. So, in figure 1, we see a list of numerals and strings and find the `5` is in the list but `"five"` is not.

```
# Figure 1: Using the "in" operator
numbs = [1, 2, "three", 'four', 5]
print("5 in list:", 5 in numbs)
print("five not in list:", "five" not in numbs)
--
5 in list: True
five not in list: True
```

Finding the number of items in a sequence

The `len()` function tells you how many items are in any sequence. In figure 2, we have a list of the primary colors, and we use the `len()` function to print the number of primary colors.

```
# Figure 2: The len() function
primary_colors=('red', 'yellow', 'blue')
print("How many primary colors?", len(primary_colors) )
--
How many primary colors? 3
```

Concatenating sequences

You can use the `+` operator on sequences. It acts as you'd expect, returning a new object that is the concatenation of the sequences in the expression.

```
# Figure 3: Concatenating sequences using +
pets = "cats " + "and" + " dogs"
digits = (0,1) + (2,3)
print("pets:", pets)
print("digits:", digits)
--
pets: cats and dogs
digits: (0, 1, 2, 3)
```

Repeating sequences

Use the `*` operator to concatenate multiple handles of the objects in a single sequence. You can put the number before or after the `*`.

Note: The `*` operator copies handles. It does not create new objects. So changing one of the objects will change all the objects with the same handle.

The `*` operator is helpful when creating horizontal bars in output reports as we see in figure 4.

```
# Figure 4: Creating a horizontal bar using *
horiz_bar = 15 * "-"
print("horiz_bar", horiz_bar)
--
horiz_bar -----
```

Use `len()` function with the `*` operator to do fancy things such as embed a word in a header as in figure 5.

```
# Figure 5: Embedding a word in a header
width = 20
word = "pyuvn"
word_len = len(word)
plus_around_word = width - word_len - 4
left_plus = plus_around_word / 2
if left_plus.is_integer() :
    right_plus = left_plus
else:
    left_plus = int(left_plus)
    right_plus = left_plus - 1
print("+" * width)
print("+" * left_plus, f" {word} ", "+" * right_plus)
print("+" * width)
--
+++++
+++++  pyuvn  +++++
+++++
```

Indexing into a sequence

Square brackets create a new object from a sequence based on the numbers in the brackets. This is called a *slice*. When you have a single number in the bracket, the *slice* looks like we indexed into the sequence.

```
# Figure 6: Slicing a single element
print('"abcdef"[2] :&apos;, "abcdef"[2] )
RayList = list("abcdef")
print("RayList[3] : ", RayList[3] )
print(" id(RayList: ", id(RayList) )
print("id(RayList[3] :", id(RayList[3] ) )
--
"abcdef"[2] : c
RayList[3] : d
id(RayList: 140377525787904
id(RayList[3] : 140378053836464
```

Emulating a two-dimensional array

You can create a data structure that acts like a two-dimensional array using a list that contains lists. Figure 7 shows the apparent indexing in the comment and demonstrates how to use slicing to get an element from the array.

```
# Figure 7: Using a list of lists to emulate a 2d array.
"""
0 1 2
0 "x", "y", "z"
1 "a", "b", "c"
"""

TwoDim=[["x","y","z"] ,["a","b","c"] ]
print("XYZ\nabc at [0] [2] =", TwoDim[0] [2] )
--
XYZ
abc at [0] [2] = z
```

Create a new sequence from an existing sequence using a slice

Using a `:` inside square brackets returns a subset of the original as a new sequence called a *slice*. The slice starts at the lower number and includes all the items up to, but not including the high number. Notice in figure 8 that the indexing starts at zero.

```
# Figure 8: Slicing more than one element
letters = "abcdefghi"
print("'abcdefghi'[1:5] = ", letters[1:5] )
--
'abcdefghi'[1:5] = bcde
```

Access a range of sequence objects using a step greater than one

The square bracket indexing operator (`[]`) can take two colon (`:`) operators. The first is the first item in the range, the second is one larger than the last item in the range, and the third is the step to count from the smaller number to the larger number.

In figure 9, we create a list of the first ten integers and then print the even ones up to 9 using a step of 2.

```
# Figure 9: Slicing with a step greater than 1
first10 = list(range(11) )
print("first10[0:9:2] = ",first10[0:9:2] )
--
first10[0:9:2] = [0, 2, 4, 6, 8]
```


Find the maximum and minimum objects in a sequence

The `max()` and `min()` functions return the largest and smallest items in a sequence (or any other iterable). As with the `sorted()` function, you can provide a `key` function to compare the objects supplied by the sequence.

In figure 10, we see that lowercase letters have higher Unicode values than upper case. Therefore, we need to use `str.casefold` as the key to get the correct min and max.

```
# Figure 10: Using max() and min()
letters = "aBcDeF"
print("max letter:", max(letters) )
print("max letter with casefold:", max(letters, key=str.casefold) )
print("min letter:", min(letters) )
print("min letter with casefold:", min(letters, key=str.casefold) )
print("min letter with casefold:", min(letters, key=str.casefold) )
--
max letter: e
max letter with casefold: F
min letter: B
min letter with casefold: a
```

Find the index of a value in a sequence

We can find the index of objects in a sequence using the `index()` method.

Finding the first instance of a value

In figure 11, the comment before the string `"Mississippi"` shows the index for each letter, and `msg.index('s')` tells us the first 's' is at location 2.

```
# Figure 11: Finding a value using index()
# 0123456789A <-- Indices msg[0] == "M"
msg = "Mississippi"
print("index of first s:", msg.index('s'))
--
index of first s: 2
```

Finding the first instance of a value after a specified index

We don't have to start our search at index 0. In figure 12, we use `msg.index('s',4)` to start our search at index 4 instead of 0.

```
# Figure 12: Finding the first "s" starting at 4
# 0123456789A <-- Indices msg[0] == "M"
msg = "Mississippi"
print("Start search at index 4:", msg.index('s',4) )
--
index of first s after 3: 5
```

Finding the index of an item in a range

We can search for a value within a range of indices. The upper bound is not in the search range as with the square bracket operator. Figure 13 uses `msg.index(s,5,8)` to find an `s` between indices `5` and `7`.

```
# Figure 13: Finding the first s in a range
# 0123456789A <-- Indices msg[0] == "M"
msg = "Mississippi"
print("index of first s between 5 and 7:", msg.index('s',5,8) )
--
index of first s between 5 and 7: 5
```

Count the number of values in a sequence

The `count()` method returns the number of times a given value appears in a sequence. For example, how many `s` characters are in Mississippi?

```
# Figure 14: Counting values
print("s's in Mississippi:", "Mississippi".count('s') )
print("q's in Mississippi:", "Mississippi".count('q') )
--
s's in Mississippi: 4
q's in Mississippi: 0
```

Summary

This chapter demonstrated commonly used sequence operations. We've now learned the basics of Python.

Modules

Like SystemVerilog and VHDL, Python allows us to import types and values from an external file. Python calls these files modules. When the developer stores modules in a directory structure, we call the directory a package. `cocotb`, for example, is delivered as a package.

A quick word about scopes

The word *scope* refers to the variables available to your program. In SystemVerilog, you add a variable to a scope by declaring it. In Python, you add a variable to a scope by assigning a value to it. You represent an uninitialized variable by assigning it the value `None`.

Python stores a program's variable names in a list. You can see the list using the `dir()` function. For example, we see in figure 1 that the variable `aa` and the class `foo` we created are in the `dir()` list.

```
# Figure 1: Using dir() to see defined variables in this scope
aa = 5
bb = None
print("'aa' in dir() :", 'aa' in dir() )
print("'bb' in dir() :", 'bb' in dir() )
--
'aa' in dir() : True
'bb' in dir() : True
```

Understanding scopes makes it easier to understand what happens when you import a module.

What happens when you import a module?

When you use the `import` keyword to import a module, Python runs the code in the module. Most code in a module defines variables, functions, and other resources.

Python will store the resources defined in a module in either the module's scope or your program's scope. The location depends upon how you import the module.

Importing a module with the import statement

The `import` statement executes the code in the module and adds the module name to your program's scope. Then, you access the resources in the module by using the module name and a dot (`.`) to reference the resource.

In figure 2, we import the `pyuvmm` module, which defines a variable named `FIFO_DEBUG` and sets it to `5`. Notice that the module `pyuvmm` became part of our scope, but that we need to reference the `FIFO_DEBUG` variable using a `.` between the module name and the variable name.

```
# Figure 2: Referencing from an imported module
import pyuvmm
print("FIFO_DEBUG:", pyuvmm.FIFO_DEBUG)
--
FIFO_DEBUG: 5
```

While the `import` statement makes the `FIFO_DEBUG` variable available, it does not add it to our list of variables, as we can see by checking the list returned by `dir()`.

```
# Figure 3: Important a module does not affect scope members
print("FIFO_DEBUG in dir() :", "FIFO_DEBUG" in dir() )
--
FIFO_DEBUG in dir() : False
```

Some contend that this approach to importing creates the most readable code because the reader knows that `pyuvm` defines `FIFO_DEBUG`.

Aliasing a module

The style of using the `.` operator to reference variables in an imported module can lead to long variable names, especially with modules that have long names. You can shorten the name using the `as` statement in the import to provide a shorter alias.

```
# Figure 4: Aliasing an import
import pyuvm as p
print("FIFO_DEBUG:", p.FIFO_DEBUG)
--
FIFO_DEBUG: 5
```

Importing resources to our `dir()` list

Specifying the module name when referencing a resource clarifies the resource's source. However, it creates longer programming statements and can be challenging to read.

The `from` keywords defines a module's objects in our program's scope. This means that we don't need the `<module>.` string to access the object.

This gives us shorter code, but hides the object's definition.

In figure 5, we use `from` to put `FIFO_DEBUG` into our `dir()` list and reference it without needing the `.`

```
# Figure 5: Using from to add a variable to our scope
from pyuvm import FIFO_DEBUG
print("FIFO_DEBUG in dir() ", "FIFO_DEBUG" in dir() )
print("FIFO_DEBUG:", FIFO_DEBUG)
--
FIFO_DEBUG in dir() True
FIFO_DEBUG: 5
```

In the `print()` function, you can see that we did not need the `pyuvm.` reference.

Importing all resources from a module

Those who have used the SystemVerilog UVM are familiar with an `import` statement that imports everything defined in `uvm_pkg` into our scope. Here it is.

```
import uvm_pkg::*;
```

Python has the same functionality using the `from` statement and the `*`. All the `pyuvm` examples in this book mimic the SystemVerilog approach by importing all `pyuvm`'s resources into the local `dir()` list. The statement `from pyuvm import *` puts everything defined in `pyuvm` into our `dir()` list. For example, the `uvm_object` class.

```
# Figure 6: Importing all resources to our scope
from pyuvm import *
print("uvm_object in dir() ", "uvm_object" in dir() )
--
uvm_object in dir() True
```

Python coding style shuns import *

A *Python Enhancement Proposal* or *PEP* is a numbered online document recommending a change to Python or a recommended way of programming. PEP-8 is named *Style guide for Python* code and defines Python's linting rules. PEP-8 has this to say about `import *`.

Wildcard imports (`from import *`) should be avoided, as they make it unclear which names are present in the namespace, confusing both readers and many automated tools.

I decided to ignore this rule for `from pyuvm import *` because I wanted Python UVM code to look like SystemVerilog UVM code. But one has to admit that it is clearer to write `@pyuvm.test()`, so I recommend both.

```
from pyuvm import *
import pyuvm
```

Finding modules

When you import a module, Python searches for it in a list of filesystem paths stored in `sys.path`.

```
# Figure 7: Viewing the system path
import sys
print(sys.path)
--
['/Users/raysalemi/repos/python4uvm_examples',
'/Users/raysalemi/opt/anaconda3/lib/python38.zip'
...
]
```

Python fills the list using the `PATH` environment variable and the `PYTHONPATH` environment variable. We can also append paths to this list. In later example files, we will use this to import a module named `tinyalu_utils` into our testbenches. The code in figure 8 uses the `Path` class from the `pathlib` module to get the full path to the parent directory of the directory we're running in; then, it prepends the parent's path to `sys.path`. Now Python will be able to find `tinyalu_utils.py`.

```
# Figure 8: Adding the directory above ours to the path
from pathlib import Path
import sys
# All testbenches use tinyalu_utils, so store it in a central
# place and add its path to the sys path so we can import it
sys.path.insert(0, str(Path("..") .resolve() ) )
from tinyalu_utils import get_int, logger # noqa: E402
```

We use `tinyalu_utils` as a place to store utility routines such as `get_int`, which we'll explain when we talk about **cocotb** in a later chapter.

Summary

In this chapter, we learned about importing modules and controlling which items get defined in a module or added to our `dir()` list. This ability to import functionality is key to Python and enables an enormous Python ecosystem.

Functions

We create subroutines in Python using functions. As with mathematical functions, Python functions take zero or more arguments and return a value. Which reminds me of a Pascal joke.

Niklaus Wirth, Pascal's inventor, said, "Whereas Europeans generally pronounce my name the right way ('Niklaus Wirt'), Americans invariably mangle it into 'Nick-les Worth'. This is to say that Europeans call me by name, but Americans call me by value."

While Python calls its functions by value, that is to say the function cannot change the value of the argument, that value is a handle to an object, and the function can use the handle to change the data attributes in the object.

It's best to avoid such games, and returns all values using the `return` statement. But we'll see cases later where modifying the object using its handle is the right approach.

Defining a function

The `def` keyword defines a function. The function name comes next with parentheses that contain optional arguments. A function with no arguments still needs the parentheses. As with all Python blocks, we follow the definition with the `:`, and indent the function statements.

```
# Figure 1: Defining a simple function
def say_hello(name) :
    print(f"Hello, {name}.")
    say_hello("Heba")
--
Hello, Heba.
```

Functions are callable objects

Python refers to functions as callable objects. If we have a handle to a callable object, we can call it by putting in a statement with parentheses after it. Of course, not all objects are callable. For example, in figure 2 we try to call the number `5`.

```
# Figure 2: You can try to call any object, but it doesn't always work.
5()
--
SyntaxWarning: 'int' object is not callable; perhaps you missed a comma?
```

Functions are callable objects. Here we define `say_hello()` and show its type to be `function`.

```
# Figure 3: Functions are objects of type function
def say_hello(name) :
    print(f"Hello, {name}.")

print("say_hello is a", type(say_hello))
--
say_hello is a <class 'function'>
```

Because `say_hello` is an object, we can store it in a variable and then call the variable using parentheses.

```
# Figure 4: Calling a function using a variable
def say_hello(name) :
    print(f"Hello, {name}.")

sh = say_hello

sh("Ali")
--
Hello, Ali.
```

Treating functions as objects is mind-blowing for a SystemVerilog or VHDL programmer, as those languages have no way to store a function in a variable. Functions in those languages are not objects and cannot be stored or passed directly as arguments, though one can approximate the behavior in some languages.

As we saw when we discussed the `min()` and `max()` functions in the *Commonly used sequence operations* chapter, we can pass function objects as arguments.

For example, Python allows you to create a `filter` object. Like the `range` object, a `filter` is an iterator that you can use in a `for` loop or a list comprehension. You instantiate a filter by passing it a filtering function and an iterator.

```
# Figure 5: Structure of a filter call
filter(<filtering function>, <iterator>)
```

The filter uses the filtering function to test each item in the iterator and returns only those items for which the filtering function returns `True`. Functions that provide this `True / False` behavior are also called *predicates*.

Let's create a filter that provides only the short words in a list, where we define short as fewer than five characters. First, we'll create a list by splitting a sentence into its words, and then we'll filter out all the long words by creating a predicate named `short_words_only`. Finally, we'll use the predicate and the word list to create a filter object.

```
# Figure 6: Filtering out long words using a function
words = "When Mr Bilbo Baggins of Bag End announced".split()
def short_words_only(word) :
    return len(word) < 5
short_word_iterator = filter(short_words_only, words)

for sw in short_word_iterator :
    print(sw, end = " ")
--
When Mr of Bag End
```


Naming convention in this book

Function objects can be passed as arguments, stored in variables, or called using parentheses. When this book talks about a function you'll call, it puts parentheses after the name. However, the book doesn't use parentheses when we talk about a function object that we will pass as an argument or store in a variable.

For example, this book refers to the `print()` function using parentheses because we typically call that function. However, we refer to `short_words_only` without parentheses in the preceding code because we pass it as an argument.

Arguments

There are two sides to every argument (heh). There is the defining side and the calling side. This chapter will discuss defining arguments in functions and then using them when we call the functions. Then we'll examine functions with a variable number of arguments.

Defining Arguments

Defining arguments in a Python function is more straightforward than in other languages. First, Python doesn't require argument types. Instead, you name the argument and provide a default value optionally. Second, there are no output arguments. Results get returned through the `return` statement. Figure 7 shows a simple function that increments a number and returns its remainder when divided by three. The argument is `xx`.

```
# Figure 7: Passing an argument to a function
def inc_mod_3(xx) :
    return (xx+1) %3
print("(5+1) %3:", inc_mod_3(5) )
--
(5+1) %3: 0
```

The user calling this function must provide a value for `xx` because `xx` has no default.

```
# Figure 8: Calling a function while missing an argument
def inc_mod_3(xx) :
    return (xx+1) %3
print(inc_mod_3() )
--
TypeError
      2 return (xx+1) %3
      3
----> 4 print(inc_mod_3() )
TypeError: inc_mod_3() missing 1 required positional argument: 'xx'
```

If we set `xx` to default to `0` we can call `inc_mod_3` without an argument.

```
# Figure 9: Creating an argument that has a default
def inc_mod_3(xx=0) :
    return (xx+1) %3

print("default (0+1) %3:", inc_mod_3())
--
default (0+1) %3: 1
```

Default and non-default arguments

There are two broad categories of arguments.

- *Non-default Arguments* —These have no default value. This kind of argument is also called *positional* arguments because the user must use them in the correct order.
- *Default Arguments* —These have a default value. This kind of argument is also called a *keyword* argument since its definition contains a keyword (the argument name) and a default value. You can call default arguments in any order by providing the keyword with a value.

For example, we can create a function named `my_power` that raises a base to an exponent. In this first example, we define two non-default arguments. When you call this function, you must provide the base and exponent in the correct order.

```
# Figure 10: Two non-default arguments
def my_power(base, exponent) :
    return base**exponent
print("2**3:",my_power(2,3) )
--
2**3: 8
```

We can make the `exponent` a default argument by adding a default value.

```
# Figure 11: Defining an argument default
def my_power(base, exponent=0) :
    return base**exponent
print("2**0:",my_power(2) )
--
2**0: 1
```

But if we make the `base` a default argument we must also make the `exponent` a default argument since we've stopped defining non-default arguments at that point.

```
# Figure 12: A default argument must be followed by default arguments
def my_power(base=0, exponent) :
    return base**exponent
print("0**2:",my_power(2) )
--
    def my_power(base=0, exponent) :
        ^
SyntaxError: non-default argument follows default argument
```

If we define both `base` and `exponent` as default arguments then we don't need to call the function with arguments.

```
# Figure 13: Defining all arguments as default arguments
def my_power(base=0, exponent=0) :
    return base**exponent
print("0**0:", my_power() )
--
0**0: 1
```

Calling functions using arguments

The variable in an argument list is called a *keyword*. Figures 12 and 13 show the `base` and `exponent` keywords in both non-default and default modes.

When you call a function, you can always explicitly set a keyword to a value, even if the function definition doesn't specify a default argument. Python calls this a *keyword argument*. If you don't provide keywords, you are relying on positions to match arguments, so Python calls this a *positional argument*.

In figure 15, we create a function named `vec_length` that returns the length of a three-dimensional vector. We import the `math` module to access its `sqrt()` function. The `".2f"` format string in the f-string prints two decimal places of a `float` number. We find the vector length by squaring the three coordinates and taking the square root of the sum.

First we call `vec_length()` using positional arguments. We assume that the argument order is `x`, `y`, `z`. Next, we call `vec_length()` using the keywords. Both approaches return the same answer.

```
# Figure 14: Calling the same function with and without keyword assignments
import math
def vec_length(x,y,z) :
    return math.sqrt(x**2 + y**2 + z**2)

print(f"positional: {vec_length(1,1,1) :.2f}")
print(f"keyword args: {vec_length(x=1, y=1, z=1) :.2f}")
--
positional: 1.73
keyword args: 1.73
```

We'll see that if we use a keyword for calling one of the arguments, we must do the same for all the following arguments. Unfortunately, here we make the mistake of using the keyword only for `y`.

```
# Figure 15: Once you provide a value, all the following arguments need one.
print(f"{vec_length(y=1, 2, 3) :.2f}")
--
SyntaxError: positional argument follows keyword argument
```

We know that `1` goes with `y`, but where does `2` go? Does it match up with the `x` or the `z`? Since there is no way of telling, Python returns an error.

Defining a variable number of arguments

Many functions take a variable number of arguments, `print()` for example. We implement variable numbers of arguments with two special arguments: `*args` and `**kwargs`.

Let's extend our `vec_length()` function to create `print_vec_length()` which takes any number of dimensions as arguments and prints out the length. The code in figure 16 calls `print_vec_length()` with different numbers of coordinates.

```
# Figure 16: Calling a function using different numbers of arguments
print_vec_length(1,1,1, label="three:")
print_vec_length(1,1,1,1,label="four :")
print_vec_length(1,1,1,1,1, label="five :", units="ft")
--
three: 1.73 cm
four : 2.00 cm
five : 2.24 ft
```

Our `print_vec_length()` function needs to take any number of non-default arguments and two default arguments. The code in Figure 17 implements this using `*args` and `**kwargs` in the argument list.

```
# Figure 17: Processing a variable number of arguments
import math
def print_vec_length(*args, **kwargs) :
    sqr_sum = 0
    for pp in args:
        sqr_sum += pp**2
    len = math.sqrt(sqr_sum)
    label = kwargs.get("label", "")
    units = kwargs.get("units", 'cm')
    print(f'{label} {len:2.2f} {units}')
```

The `*args` argument gives us a tuple named `args` which contains any number of non-default arguments. First, we loop through the numbers in the `args` summing the squares. Then we find the length by taking the square root of the sum.

The `**kwargs` argument gives us an object called a `dict` (for dictionary) that we will discuss in a later chapter. Here we need only to know that the `dict` class has a `get()` method that takes a key and a default value. If the key returns a value, we get the value. Otherwise, we get the default value.

We don't explicitly declare default arguments using `**kwargs`. Instead, the default values flow from our calls to `get()`. The calls to `kwargs.get()` in figure 17 have the same effect as putting `label=""` and `units="cm"` in the argument list. `label` and `units` are now optional arguments with default values.

```
# Figure 18: Using the default values for label and unit
print_vec_length(1,2,3)
--
3.74 cm
```

Documentation strings

Python has a built-in documentation system that generates documentation, supports the `help` command, and works with advanced editors such as Visual Studio Code that show programmers hints for a function.

We must place documentation in docstrings to take advantage of these features. Docstrings start with three double quotes and end with three double quotes. They go right after the `def` statement. By convention, they have a blank line after them.

The `help()` function returns the docstring as help.

```
# Figure 19: A single-line docstring
def print_hello_world() :
    """It does what you think it does"""

    print("Hello, world.")

help(print_hello_world)
--
Help on function print_hello_world in module __main__:

print_hello_world()
    It does what you think it does
```

Figure 19 uses a single-line docstring, but you can make multiple-line docstrings by putting the triple double-quotes on their own lines. By Python convention, the first line should be a complete sentence followed by a blank line as in figure 20.

```
# Figure 20: A multi-line docstring

def my_power(base, exponent) :
    """
    Return the base to the exponent

    base: The base number
    exponent: The exponent
    returns: base raised to the exponent
    """
    return base**exponent

help(my_power)
--
Help on function my_power in module __main__:

my_power(base, exponent)
    Return the base to the exponent

    base: The base number
    exponent: The exponent
    returns: base raised to the exponent
```

We can also see this information in Visual Studio Code when we hover the mouse over the function.

```
(function) my_power:(base,exponent)->Any  
Raises the base to the exponent
```

base: The base number

exponent: The exponent

returns: base raised to the exponent

Creating docstrings for all your functions is an excellent habit and will save you time and documentation work in the long run.

Summary

Functions are objects that contain other statements and execute those statements when called. For example, call function object `foo` with the statement `foo()`. Function arguments are only inputs. All return data must come through the `return` statement.

Every function has a `return` value; if you don't provide a `return` statement, the function returns `None`.

Functions have positional arguments and keyword arguments. We use `*args` and `**kwargs` to create functions that accept a variable number of arguments.

Writing docstrings for all of your functions allows you to quickly generate documentation and provide hints to programmers using an advanced editor.

Exceptions

Exceptions are an error reporting mechanism that does not exist in SystemVerilog or VHDL. As a metaphor, consider a situation where your boss asks you to do a task. In doing it, you find an error, so you raise it to your boss. Now it's off your plate. Your boss, presented with the error, raises it to their boss, and that boss raises it to their boss who, one hopes, handles it because if nobody handles the error, it gets reported to the public.

Python exceptions work the same way, except instead of raising the exception (error) to your boss, you raise it to the function that called you. The exception travels up the stack of functions until it gets to the top. If no function in the stack handles the exception, then it gets reported to the public as what looks like a crash. The user sees the entire call stack down to the line that raised the exception.

Figure 1 creates an exception by dividing by zero. Python catches the error and raises an exception. Exceptions, like everything else in Python, are objects. The object's class gives us a clue as to what caused the error. In this case, the exception object is of type `ZeroDivisionError`.

```
# Figure 1: You can't divide by zero
print("3/0 = ", 3/0)
--
ZeroDivisionError Traceback
----> 1 print("3/0 = ", 3/0)

ZeroDivisionError: division by zero
```

In this example, the division operator raised a `ZeroDivisionError` exception. This exception was passed up the call stack until Python printed the error message with the list of function calls that led to the exception (the *Traceback*.)

We've mentioned the way exceptions travel up the call stack. Figure 2 shows an example where we create a function to divide and then use it to divide by zero. We see the stack trace that shows that we called `nice_div()` on line 5, but the error originated at line 2 in the function `nice_div`, where it divided by zero.

```
# Figure 2: Follow the error in a traceback
def nice_div(dividend, divisor) :
    result = dividend/divisor
    return result

print("nice_div(3,0) = ", nice_div(3,0) )
--
ZeroDivisionError Traceback
   3 return result
   4
----> 5 print("nice_div(3,0) = ", nice_div(3,0) )
      in nice_div(dividend, divisor)
         1 def nice_div(dividend, divisor) :
----> 2 result = dividend/divisor
         3 return result
         4
         5 print("nice_div(3,0) = ", nice_div(3,0) )
```

```
ZeroDivisionError: division by zero
```

Notice that the program never reached line 3. Code execution stops as soon as a function or operation raises an exception.

Exceptions that go all the way up the stack stop the program with an ugly-looking error message. But, Python allows us to catch exceptions and respond to them. We'll discuss that next.

Catching exceptions

Python catches exceptions with the `try/except` keywords. We try an operation, and if the operation raises an exception, we catch it and process it.

The code in Figure 3 improves our `nice_div()` function by having it return the mathematically correct answer to division by zero, which is infinity. We represent infinity in Python using the `math` module and the `math.inf` constant.

We implement this by using a `try` block and an `except` block. If any of the statements in the `try` block raises an exception, control immediately passes to the `except` block. If the exception's type matches the type called for in the `except` block, we run the code in the block. Otherwise, we raise the exception up the call stack.

```
# Figure 3: Catching a ZeroDivisionError exception
import math
def nice_div(dividend, divisor) :
    try:
        result = dividend/divisor
        return result
    except ZeroDivisionError:
        return math.inf

print("nice_div(3,0) = ", nice_div(3,0) )
--
nice_div(3,0) = inf
```

Catching multiple exceptions

Dividing by zero is not the only problem that could plague `nice_div()`. The code in figure 4 tries to divide an `int` by a `str`. This makes no sense, so it creates a `TypeError` exception. However, our `except` block only catches `ZeroDivisionError` exceptions, so it passes the `TypeError` up the stack.

```
# Figure 4: We didn't catch the TypeError exception
import math
def nice_div(dividend, divisor) :
    try:
        return dividend/divisor
    except ZeroDivisionError:
        return inf

print('nice_div(3,"zero") ', nice_div(3, "zero") )
--
TypeError Traceback
```



```

        6 return math.inf
        7
----> 8 print('nice_div(3,"zero") ', nice_div(3, "zero") )
in nice_div(dividend, divisor)
    1 def nice_div(dividend, divisor) :
    2 try:
----> 3 result = dividend/divisor
    4 return result
    5 except ZeroDivisionError:

TypeError: unsupported operand type(s) for /: 'int' and 'str'

```

We need to catch more than the `ZeroDivError` exception type. We do this by using multiple `except` blocks. First, we look for `ZeroDivError` , then we look for `TypeError` . Now, if we divide by a string, we get told the result is not a number (`math.nan`).

Figure 5: Catching multiple exceptions with multiple blocks

```

import math
def nice_div(dividend, divisor) :
    try:
        return dividend/divisor
    except ZeroDivisionError:
        return math.inf
    except TypeError:
        return math.nan

print('nice_div(3,"zero") =', nice_div(3,"zero") )
print("nice_div(3,0) =", nice_div(3,0) )
--
nice_div(3,"zero") = nan
nice_div(3,0) = inf

```

Now we return the correct value for the correct error.

In cases where you want to treat multiple exceptions the same way, you can catch them with one line as in figure 6.

Figure 6: Catching multiple exceptions on one line

```

import math
def nice_div(dividend, divisor) :
    try:
        result = dividend/divisor
        return result
    except (ZeroDivisionError, TypeError) :
        print(f"You screwed up your division, human: {dividend}/{divisor}")
        return math.nan

print('nice_div(3,"zero") =', nice_div(3,"zero") )
--
You screwed up your division, human: 3/zero
nice_div(3,"zero") = nan

```

Raising an exception after processing it

There are cases where you'll catch an exception but still want to send it up the call stack for additional processing or reporting. Let's modify `nice_div` to print a snarky message, but use the `raise` statement to send the exception up the stack. In figure 7, we print our message and then call `raise`.

```
# Figure 7: Sending an exception up the stack
import math
def nice_div(dividend, divisor) :
    try:
        result = dividend/divisor
        return result
    except (ZeroDivisionError, TypeError) :
        print(f"You screwed up your division, human: {dividend}/{divisor}")
        raise

xx = nice_div(3,"zero")
```

The stack trace in figure 8 includes the `raise` call and the original error.

```
# Figure 8: Stack trace from reraising an exception
You screwed up your division, human: 3/zero
-----
TypeError Traceback
  nice_div
      8 raise
      9
----> 10 xx = nice_div(3,"zero")
      nice_div(dividend, divisor)
          2 def nice_div(dividend, divisor) :
          3 try:
----> 4 result = dividend/divisor
          5 return result
          6 except (ZeroDivisionError, TypeError) :

TypeError: unsupported operand type(s) for /: 'int' and 'str'
```

Printing an exception object

The `except` blocks filter exceptions based on the exception's type. They can also grab the exception object itself and use it to print a message.

In figure 9, we catch the `ZeroDivisionError` and `TypeError` exceptions in a single `except` block since we do the same thing for both exception types.

We also get a handle to the exception using the `as` keyword and passing it the `ex` variable. This stores the exception in a handle named `ex`. We print a snarky message and the exception, then return `math.nan`.

```
# Figure 9: Printing an exception
import math
def nice_div(dividend, divisor) :
```

```

    try:
        result = dividend/divisor
        return result
    except (ZeroDivisionError, TypeError) as ex:
        print(f"You screwed up your division, human: {ex}")
        return math.nan

print('nice_div(3,"zero") =', nice_div(3,"zero") )
--
You screwed up your division, human: unsupported operand type(s) for /: 'int' and 'str'
nice_div(3,"zero") = nan

```

Finally! A finally block

The `finally` block runs regardless of what happens in the `try` and `except` blocks. Figure 9 example demonstrates all three scenarios:

- `print_div(33, 2)` finishes normally and runs the `finally` block.
- `print_div(33, 0)` catches the `ZeroDivisionError` exception and runs the `finally` block.
- `print_div(33, "zero")` does not catch the `TypeError` exception but still runs the `finally` block.

```

# Figure 10: The finally block always executes
def print_div(dividend, divisor) :
    try:
        result = dividend/divisor
        print(f"The result of {dividend}/{divisor} is: {result}")
    except ZeroDivisionError as ex:
        print(f"You screwed up your division, human. {dividend}/{divisor} returned:
{ex}")
    finally:
        print("Division complete.")

print_div(33, 2)
print_div(33, 0)
print_div(33, "zero")
--
The result of 33/2 is: 16.5
Division complete.
You screwed up your division, human. 33/0 returned: division by zero
Division complete.
Division complete.
-----
TypeError Traceback
    10 print_div(33, 2)
    11 print_div(33, 0)
--> 12 print_div(33, "zero")
...

```

The `finally` block allows you to guarantee that an operation will happen even if the code throws an exception.

The assert statement

Python programmers ask forgiveness, not permission. Unlike SystemVerilog and VHDL, Python does not require that variables be declared with a type. Instead, it allows you to put any object into any operation and raises an exception if you got it wrong.

We can create customized error checking using the `assert` statement. The `assert` statement checks a conditional expression and raises an `AssertionError` exception if the result is `False`. We can append a message to the `assert` statement to tell the user what they did wrong.

In figure 11, we create a function that XORs any number of bytes and raises an error if you give it a number less than zero or greater than 255 (the maximum number in eight bits). Note: the `0x` makes constants hex numbers instead of decimal.

```
# Figure 11: Using assert to check inputs

def xor_bytes(*args) :
    xor = 0
    for bits in args:
        assert bits <= 255 and bits >= 0, f"Invalid byte: {bits}"
        xor ^= bits
    return xor

print(f"0x8 ^ 0x9 ^ 0x10: 0x{xor_bytes(0x8, 0x9, 0x10) :2x}")
print(f"0xFF ^ 0x100: 0x{xor_bytes(0xFF, 0x100) :2x}")
--
0x8 ^ 0x9 ^ 0x10 = 0x11
AssertionError: Invalid byte: 0x100
```

Whenever you write code that depends upon some condition being true, it is wise to insert an `assert` statement to catch inconsistencies as early as possible.

The assert statement is not a function

You might think that the `assert` statement in figure 12 will always raise an `AssertionError`.

```
# Figure 12: This code fails

assert (8 < 7, "Obviously false")
```

You would be tragically wrong. `assert` is a statement, not a function. So this code creates a `(True, "Obviously false")` tuple. Since the tuple is not `None` the `assert` statement evaluates it as `True` and this check never happens.

Don't use parentheses with `assert`.

Summa2ry

Python exceptions provide a robust system for catching errors and responding to them. Statements raise exceptions, and Python programmers catch them using `try/except/finally` blocks.

The `try` block runs the code. If a statement raises an exception, it stops execution and jumps to one or more `except` blocks. If the types on these blocks match the exception type, the block runs. Otherwise, the exception rises up the call stack.

The `finally` block always runs regardless of exceptions.

Dictionaries

To write complex programs, we need a way to store data in some sort of database using a key and retrieve it later using the same key.

In old-fashioned C, we did this with an array. The key was the integer index into the array. But in Python and later languages, this notion has been generalized so that you can use a wide variety of types as the key. These are called *associative arrays*. Python calls associative arrays dictionaries, implemented with the `dict` type.

Let's use a dictionary to store the numbers of great footballers.

Figure 1: Matching keys and values

```
players = {}
players["Messi"] = 10
players["Beckham"] = 7
players["Salah"] = 11
print("Messi's number:", players["Messi"])
--
Messi's number: 10
```

In this example, the strings `"Messi"`, `"Beckham"`, and `"Salah"` are called *keys*. The numbers are called *values*. Key/value pairs are called *items*.

Creating a Dictionary

The simplest way to create an empty dictionary is using the curly braces as in `players = {}`. Once we've done that, we can fill the dictionary using keys and values. There are several ways to do this.

Create and fill an empty dictionary

Keys can be any built-in type like `int` or any immutable sequence such as `str`. Items can be any object. Here is another example demonstrating that, unlike SystemVerilog, the same dictionary can use different types as keys.

Figure 2: Creating an empty directory and filling it

```
example = {}
example[1] = "one"
example["two"] = 2
print("example: ", example)
--
example: {1: 'one', 'two': 2}
```

Here we mixed and matched keys: the first item uses an `int` as a key (`1`) and a `str` as the value (`'one'`). The second item uses a `str` as the key (`'two'`) and an `int` as the value (`2`).

Create a dictionary using the data that will fill it

The code in figure 2 created an empty dictionary (`{}`). We filled it with data, but we could also have created the entire dictionary in one statement by using a colon (`:`) to separate the keys and values.

```
# Figure 3: Storing keys and values together
```

```
example = {1:"one", "two":2}
print("example: ", example)
--
example: {1: 'one', 'two': 2}
```

There are other ways to create dictionaries, but this is the simplest, so it's the one we'll use in this book.

Use dictionary comprehension

In the *Lists* chapter, we discussed *list comprehension* in which we could create a list from an iterator (an object one can use to feed a `for` loop) using square brackets.

Similarly, we can create dictionaries using *dictionary comprehension*. In figure 4, we create a dictionary that stores the cubes of the first three numbers using a `for` loop.

```
# Figure 4: Filling a dictionary using a for loop
```

```
cubes = {}
for ii in range(4) :
    cubes[ii] = ii**3
print ("cubes:", cubes)
--
cubes: {0: 0, 1: 1, 2: 8, 3: 27}
```

Figure 5 creates the same dictionary using dictionary comprehension. The comprehension statement loops through an iterator using the `:` character to store a key and a value as in `<key> : <value>`.

```
# Figure 5: Using a dictionary comprehension
```

```
cubes = {ii : ii**3 for ii in range(4) }
print("cubes:", cubes)
--
cubes: {0: 0, 1: 1, 2: 8, 3: 27}
```

Handling missing keys

When we try to access a key that is not in a dictionary, the statement raises a `KeyError` exception. For example, here we create a dictionary of great football (soccer) players, but then try to find a great hockey player.

Figure 6: Raising KeyError

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}
print("Number 4?", players[4] )
--
KeyError Traceback
      1 players = {7:"Beckham", 10:"Messi", 11:"Salah"}
----> 2 print("Number 4?", players[4])

KeyError: 4
```

As with all exceptions, we can use the `try` and `except` blocks to handle missing keys.

Figure 7: Recovering from a KeyError

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}
try:
    player = players[4]
except KeyError:
    player = None

print("Number 4?", player)
--
Number 4? None
```

We can use the `KeyError` to solve programming problems. The code in figure 8 counts the number of times a letter appears in a string. We implement it by storing the count in a dictionary. If there is a `KeyError`, we know we've never seen that letter, and we set the count to one.

Figure 8: Counting characters using a dictionary

```
count = {}
for cc in "Mississippi":
    try:
        count[cc] += 1
    except KeyError:
        count[cc] = 1
for cc in count:
    print(cc, count[cc])
--
M 1
i 4
s 4
p 2
```

The `get()` method

The `get()` method tries to get a value using the key and provides a value if the key doesn't exist. In this example, we attempt to get the great hockey player Bobby Orr's number from our database using `get()`. By default, `get()` returns `None` if the key is not in the dictionary.

Figure 9: Using the `get()` method to retrieve a value

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}
player = players.get(4)
print("Number 4?", player)
--
Number 4? None
```

We can also pass `get()` a value to return instead of `None`.

Figure 10: Supplying `get()` with a default

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}
player = players.get(4, "Not in database")
print("Number 4?", player)
--
Number 4? Not in database
```

The `setdefault()` method

The `setdefault()` method works like the `get()` method except that instead of only returning the default value, it stores the value in the dictionary using the key. This default-setting behavior is similar to the code that counted the letters in "Mississippi".

The `setdefault()` method allows us to write the same behavior more compactly. In figure 11, we use `setdefault()` to get the current count, and if there is no entry in the dictionary, we set the entry to 0. `setdefault()` returns the 0, we increment it, and store it in the entry for that letter.

Figure 11: `setdefault()` stores the supplied default at the key

```
count = {}
for cc in "Mississippi":
    count[cc] = count.setdefault(cc, 0) + 1

for cc in count:
    print(cc, count[cc])
--
M 1
i 4
s 4
p 2
```

Dictionaries and for loops

We printed out a dictionary using a `for` loop in figure 11. Here is the `for` loop again with the counting code snipped out.

```
# Figure 12: Printing dictionary keys and values
```

```
++ <snip> ++  
for cc in count:  
    print(cc, count[cc] )
```

We see that `count` returned the letters in the order they appeared in the word "Mississippi". That is because dictionaries in a `for` loop return their keys in the order they were created ⁵.

The `.keys()`, `.values()`, and `.items()` iterators

Dictionaries contain keys, values, and the key/value pairs named *items*. Dictionaries in a `for` loop iterate over keys by default, but you can iterate over any part of the items. Here we loop over only the keys.

```
# Figure 13: Iterating over dictionary keys
```

```
++ <snip> ++  
  
for letter in count.keys() :  
    print(letter, end=" ")  
--  
M i s p
```

Now we'll loop over only the values.

```
# Figure 14: Iterating over values
```

```
for number in count.values() :  
    print(number, end=" ")  
--  
1 4 4 2
```

Now we loop over the key/value items as tuples.

```
# Figure 15: Iterating over key/value tuples
```

```
for item in count.items() :  
    print(item, end=" ")  
--  
( 'M', 1) ( 'i', 4) ( 's', 4) ( 'p', 2)
```

As with all tuples, you can supply variables to take the values. This brings us back to the behavior from the first example written without indexing the dictionary.

Figure 16: Populating the key and value variables in a for loop

```
for number, item in count.items() :  
    print(number, item)  
--  
M 1  
i 4  
s 4  
p 2
```

Removing items from a dictionary

You can remove all items from a dictionary with the `clear()` function.

Figure 17: Clearing all data from a dictionary

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}  
print("players:", players)  
players.clear()  
print("players:", players)  
--  
players: {7: 'Beckham', 10: 'Messi', 11: 'Salah'}  
players: {}
```

You can remove an individual item by using the `del` statement and providing the dictionary entry to remove.

Figure 18: Using the `del` statement with a dictionary

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}  
print("players:", players)  
del players[7] # retired  
print("players:", players)  
--  
players: {7: 'Beckham', 10: 'Messi', 11: 'Salah'}  
players: {10: 'Messi', 11: 'Salah'}
```

You can also “pop” items from a dictionary using the `pop()` function. Popping returns the value at the key and removes the item. Here we print that Beckham (7) has retired.

Figure 19: Popping data using a key

```
players = {7:"Beckham", 10:"Messi", 11:"Salah"}  
print("players:", players)  
print(players.pop(7) , "has retired.")  
print("players:", players)  
--  
players: {7: 'Beckham', 10: 'Messi', 11: 'Salah'}  
Beckham has retired.  
players: {10: 'Messi', 11: 'Salah'}
```

Summary

Dictionaries allow us to store a value using a *key* and retrieve it later. Keys can be objects of any built-in type or immutable sequence, while the values can be objects of any class. The key/value pair is called an *item*.

We can create new dictionaries using curly braces (`{}`) or dictionary comprehension.

Accessing a dictionary using a missing key raises a `KeyError` exception. We can catch the exception to handle missing items.

The `get()` and `setdefault()` functions smoothly handle missing keys.

When you use a dictionary in a `for` loop, you get the keys in the order they were created.

Generators

A generator is a customized iterator object. We can use generators in `for` loops or any other statement that takes an iterator.

We create generators using the `yield` statement. The `yield` statement is like the `return` statement in that it yields a number to the caller. Unlike `return`, `yield` continues running in the function like any other statement.

A simple example, `my_range()` acts just like the `range()` function. It returns numbers for `0` to one less than the argument by looping through a `while` loop and calling `yield`. Here a `for` loop uses `my_range` to print three numbers.

```
# Figure 1: Creating our own range() function
def my_range(nn) :
    counter = 0
    while counter < nn:
        yield counter
        counter += 1

for ii in my_range(3) :
    print(ii, end=" ")
--
0 1 2
```

The `for` loop called `my_range(3)`. The generator set `counter` to `0` and yielded that number. Then it incremented the counter, checked the `while` loop, and yielded the new number.

My favorite generator gives us the Fibonacci sequence ⁶. In figure 2, notice the two `yield` statements. The code shows that `yield` statements don't have to be inside a loop. The generator returns 0 at the first `yield` and sets `numb` to `1`, the next Fibonacci number. Next, the generator enters the `while` loop and yields the second number. Then it starts using the Fibonacci equation of adding two numbers to get the next one. It loops until `counter` reaches `nn`.

```
# Figure 2: A Fibonacci generator that returns Fibonacci numbers
def Fibonacci(nn) :
    yield 0
    counter = 1
    lastnumb = 0
    numb = 1
    while counter < nn:
        yield numb
        newnumb = numb + lastnumb
        lastnumb = numb
        numb = newnumb
        counter +=1

for ii in Fibonacci(8) :
    print(ii, end=" ")
--
```

0 1 1 2 3 5 8 13

Summary

Create new iterators using generators. Generators are functions that use a `yield` statement to yield a number but not return from the function call.

Classes

Object-oriented programming (OOP) is the foundation of the expandability and reusability of UVM-based verification. It's an excellent match with Python, which provides a clean and readily understandable implementation of classes and OOP. In addition, Python supports modern object-oriented features such as multiple inheritance and dynamic class definitions.

Defining classes and instantiating objects

We define a class in Python using the `class` statement, which is executed like any other Python statement since there is no compilation step with an interpreted language. We follow the `class` statement with the class name. By convention, we capitalize class names and use camel-casing for multiple-word names. Here we define an empty class that does nothing.

```
# Figure 1: Defining an empty class object
class EmptyClass:
    """This class defines nothing"""
    ...
```

`EmptyClass` has a docstring after it for use in `help()` statements and in editors. There is nothing in this class except for what it inherited from `type`, the default inheritance of all classes. The `...` constant is an intentionally empty line.

A class is an object, and as we saw in the *Functions* chapter, you can call any object by appending parentheses to its name. When we call a class object, we get a handle to an object of that class.

A new object has no data attributes. We add data attributes to the object by referencing it and using the dot operator (`.`). In figure 2, we create a class named `Animal`. Then call `Animal()` to get a handle to an animal object which we store in `walrus`. Then we can add a data attribute named `kg` to `walrus` and print it.

```
# Figure 2: Instantiate a class and add data attributes

class Animal:
    """Holds a generic Animal"""

walrus = Animal()
walrus.kg = 1000
print("Walrus mass:", walrus.kg)
--
walrus mass: 1000
```

Dynamically adding a data attribute to an object is *nothing* like SystemVerilog, where the `Animal` class would have to define the `kg` variable at definition as in Figure 3.

```
# Figure 3: The Animal class in SystemVerilog

class Animal;
    int unsigned kg;
endclass
```

You don't do this in Python. Instead, you create the object and start adding data attributes to it. We'll see later there is a recommended way to do this.

Functions vs. methods

We've used the word *methods* throughout the book, mostly when discussing the methods in a built-in class. For example, we discussed the `get()` method in the `dict` class. Here is the definition of a method:

A method is a function defined within a class.

While we define a function with a `def` statement in the first column of a Python script, we define a method with a `def` statement indented within a `class`. This defines a *method attribute* in the class, also called a "method".

Defining method attributes

We define method attributes by defining functions within the class definition. In figure 4 we convert kilograms to pounds in our `Animal` class by creating a `get_pounds()` method that divides `kg` by 2.2.

Figure 4: Defining a class method

```
class Animal:
    """Holds a generic Animal"""
    def get_pounds(self) :
        return self.kg/2.2

walrus = Animal()
walrus.kg = 1000
print(f"walrus weight in pounds {walrus.get_pounds() :2.2f}")
--
walrus weight in pounds 454.55
```

The reader may have noticed a discrepancy between the definition of the `get_pounds()` method attribute and its use. The definition has one argument, while the call to the method in the `print()` function has none.

What's that about? Also, what is this `self` variable?

The self variable

When you instantiate an object by calling the class, you receive a handle to an object of that class. Then you can reference data attributes in the object using the dot (`walrus.kg` in the example.)

Python assumes that a class method will want to access the attributes in an object, so Python passes the handle to the object as the first argument. By convention, we name this argument `self`.

It is possible to call the function using the class name and explicitly pass it an object handle. For example, here we instantiate `walrus` and then call `Animal.get_pounds(walrus)`. This function call passes `walrus` to the `self` argument the same way `walrus.get_pounds()` would.

```
# Figure 5: Using the self variable
class Animal:
    """Holds a generic Animal"""
    def get_pounds(self) :
```



```

        return self.kg/2.2

walrus = Animal()
walrus.kg = 1000

pounds = Animal.get_pounds(walrus) # Explicit call

print(f"walrus weight in pounds {pounds:2.2f}")
--
walrus weight in pounds 454.55

```

While one could call class methods this way, it is better to let Python do the work by using the object handle to reference the method like this : `pounds = walrus.get_pounds()`.

When you call a method this way, Python automatically passes the object handle to the `self` variable.

Initializing an object

In figure 2, we showed that in Python, you create a handle to an object by calling its class and adding data attributes to the object. This behavior is different than C and SystemVerilog, where you declare all the data attributes in the class definition.

Figure 6 shows the `Animal` class written in SystemVerilog. This class defines a method named `new()`, called a constructor. In SystemVerilog, you declare variables in a class and pass values to them in the constructor.

```

# Figure 6: The Animal class in SystemVerilog
class Animal;
    float kg;

    function new(float mass) ;
        kg = mass;
    endfunction;

    function get_pounds() ;
        return kg/2.2;
    endfunction
endclass

```

This approach is different than initializing data attributes in Python. In Python, we get a handle to the object and add data attributes. This approach can lead to errors such as the one in figure 7 where the `get_pounds()` method assumes that the `self.kg` variable exists, but we forgot to set it after creating a `yorkie`.

```

# Figure 7: We forgot to add the kg data attribute

class Animal:
    """Holds a generic Animal"""
    def get_pounds(self) :
        return self.kg/2.2

yorkie = Animal()
pounds = yorkie.get_pounds()
print(f"Yorkie weight in pounds {pounds:2.2f}")

```

```
--
AttributeError
    2 """Holds a generic Animal"""
    3 def get_pounds(self) :
----> 4 return self.kg/2.2

AttributeError: 'Animal' object has no attribute 'kg'
```

SystemVerilog would have issued a syntax error if we had tried to reference `kg` without declaring it. But Python doesn't declare data attributes in a class. Instead, it wants you to add them to the object before using them. That didn't happen here, so Python raised the `AttributeError` exception.

The standard way of adding data to an object in Python is to create an `__init__(self)` method. This method's name has two leading underscores, the word `init`, two trailing underscores, and a `self` argument.

When you create a new instance of an object by calling its class name with parentheses, Python automatically calls the `__init__(self)` method. The `__init__()` method and other methods that use the same double-underscore naming convention are called *magic methods*. You'll also see these methods called *dunder methods* where "dunder" is a contraction of "double underscore".

The `__init__()` method defines arguments in the class creation call. Python uses the values of the argument and calls `__init__()` whenever you create a new object.

The code in figure 8 defines an `__init__()` method in the `Animal` class. The method defines two arguments `.self` is mandatory and is a handle to the new object you've just created `.kg` is an additional non-default argument that contains the mass `.kg` becomes a required argument in `Animal()`.

The code instantiates an `Animal` by calling the class name and passing the mass. We create a Yorkie `Animal` in this example and store it in `yorkie`.

```
# Figure 8: The __init__(self) method forces initialization
class Animal:
    """Holds a generic Animal"""
    def __init__(self, kg) :
        self.kg = kg

    def get_pounds(self) :
        return self.kg/2.2

yorkie = Animal(20)
print(f"The Yorkie weighs {yorkie.get_pounds() :2.1f} pounds")
--
The Yorkie weighs 9.1 pounds
```

The `__init__(self)` magic method set the `self.kg` attribute. Then `get_pounds()` could access the `kg` attribute using the `self` handle.

Class variables

Like all things in Python, classes are objects, and as objects, they can have data attributes. We add data attributes to classes by assigning values to them in the class definition block. In figure 9, we create a class named `Triangle`, and since all triangles have three sides, we give the class a data attribute named `side_count`. We reference `side_count` using the `Triangle` object.

```
# Figure 9: Adding a class variable

class Triangle:
    side_count = 3
    ...

print("Number of sides in a triangle:", Triangle.side_count)
--
Number of sides in a triangle: 3
```

When we access a data attribute in an object, Python will first search the object for the data attribute. If it doesn't find it there, it looks in the object's class for the data attribute.

```
# Figure 10: Accessing a class variable using self

class Triangle:
    side_count = 3

    def print_side_count(self) :
        print(f"I have {self.side_count} sides.")

tri = Triangle()
tri.print_side_count()
--
I have 3 sides.
```

Class methods

A method that uses only class variables is called a *class method*. Python passes the first argument to class methods automatically. The difference is that instead of passing a handle to the instantiated object in the first argument, Python passes a handle to the class object.

Python implements class methods using the `@classmethod` decorator. A decorator is a Python statement that changes the function's behavior defined right after it. Decorators start with an `@` symbol and go into the line right before the line containing the `def` statement, as in this fictional example in figure 11.

```
# Figure 11: An example decorator

@mydecorator
def my_function() :
    ...
```

The `@classmethod` decorator tells Python to pass the class object handle as the first argument in the method call instead of the instance handle. By convention, we name the first argument `cls` in a class method.

Figure 12: The `@classmethod` decorator defines a class method.

```
class Triangle:
    side_count = 3

    @classmethod
    def print_side_count(cls) :
        print(f"I have {cls.side_count} sides.")

Triangle.print_side_count()
--
I have 3 sides.
```

In figure 12, `print_side_count()` gets a handle to the `Triangle` class object where it can reference the `side_count` class variable.

Static methods

Some methods don't need access to an object or its class. These are called *static methods* and have no predefined first argument. We define static methods using the `@staticmethod` decorator.

Why would we define it within a class if the static method doesn't use `self` or `cls`? There are two reasons:

1. We want to have a logical place to store the method so people know where to find it.
2. We want to invoke the static method in the same way as all other methods related to the class, e.g., `self.my_static_method(foo)`.

All triangles have three sides, so the code in figure 13 implements `print_side_count()` in `Triangle` using a `@staticmethod`. Notice that it has no arguments and that you can call the static method using the class handle or the class name.

Figure 13: `print_side_count()` as a static method

```
class Triangle:
    @staticmethod
    def print_side_count() :
        print(f"I have 3 sides.")

tri = Triangle()
Triangle.print_side_count()
tri.print_side_count()
--
I have 3 sides.
I have 3 sides.
```

Summary

Classes are the basic building block of Python object-oriented programming. We do not declare data attributes in classes. Instead, we usually assign a value to them using the `self` variable in the `__init__()` magic method. The `self` variable is a handle to the object instance and provides access to the data attributes.

Functions defined in the context of a class definition are called *methods*. There are three kinds of methods:

- *Instance methods* —These operate on a single instance of a class. Instance methods receive the `self` variable as their first argument. Python does a little magic to make that happen when you call a method using an object handle.
- *Class methods* —These operate on the class variables. The `@classmethod` decorator causes Python to pass these methods the `cls` variable in the first argument instead of `self`.
- *Static methods* —These do not receive a handle as their first argument, but they still relate to the class, so we define them in the class as a matter of convenience. We create static methods using the `@staticmethod` decorator.

Protecting attributes

When we create a class, we make assumptions about its data attributes and how they will be used. For example, in figure 1 we create a class that stores the temperature in Celsius and returns it in either Fahrenheit or Celsius.

Here is a flawed attempt to implement this. We want to store the freezing point of water. `tt = Temperature(0)` does this correctly, but later someone changes the attribute to `32`, so we print, incorrectly, `Freezing is 32 degrees Celsius`.

```
# Figure 1: Misusing class variables.
# Freezing is not 32 Celsius

class Temperature:
    def __init__(self, temp) :
        self.temp = temp

    def to_f(self) :
        ft = self.temp * 9 / 5 + 32

    def to_c(self) :
        return self.temp

tt = Temperature(0)
# Later in the code
tt.temp = 32
print(f"Freezing is {tt.to_c()} Celsius")
--
Freezing is 32 Celsius
```

Here is a case where someone using our class would file a bug report saying that it printed the wrong temperature when, in fact, they were the ones who caused the error.

Protected variables keep users from making this sort of mistake.

Protected attributes

Protected attributes tell users that they should not be accessing certain data attributes in a class. We protect data attributes and methods by putting an underscore before the name to make them *protected* attributes.

In figure 2, we improve our `Temperature` class. We protect the `temp` attribute by putting an underscore in front of it: `_temp`. We also provide methods that allow the user to set and get the temperature using Celsius or Fahrenheit. Unfortunately, the user is still misusing our class.

```
# Figure 2: Bad programmer misuses our class

class Temperature:
    def __init__(self, temp) :
        self._temp = temp

    def set_c_temp(self, c_temp) :
```

```

        self._temp = c_temp

    def set_f_temp(self, f_temp) :
        self._temp = (f_temp - 32) * 5 / 9

    def get_f_temp(self) :
        ft = self._temp * 9 / 5 + 32

    def get_c_temp(self) :
        return self._temp

tt = Temperature(0)
tt._temp = 32 #No! Bad programmer!
print(f"Freezing is {tt.get_c_temp()} Celsius")
--
Freezing is 32 Celsius

```

Figure 2 demonstrates that we should never access attributes with an underscore in front of them. These are protected. Even if our code works today, it may not work in the future. The user is breaking that rule.

Private attributes

This user is irritating us, so it is time to get serious. Figure 3 makes the temperature a private attribute by putting two underscores in front of it. That will show them. Here the thoughtless programmer sets `tt.__temp` to 32, but this does not affect our class's proper operation.

```

# Figure 3: User tries to misuse our private variable. Fails.
class Temperature:
    def __init__(self, temp) :
        self.__temp = temp

    def set_c_temp(self, c_temp) :
        self.__temp = c_temp

    def set_f_temp(self, f_temp) :
        self.__temp = (f_temp - 32) * 5 / 9

    def get_f_temp(self) :
        ft = self.__temp * 9 / 5 + 32

    def get_c_temp(self) :
        return self.__temp

tt = Temperature(0)
tt.__temp = 32 # Bad programmer!
print ("Temp in Celsius:", tt.get_c_temp())
--
Temp in Celsius: 0

```

We have finally protected ourselves and our attributes. When you put the double-underscore in front of a variable, Python mangles the variable's name inside the class so that programmers outside the class cannot modify it. These private attributes are also called *private variables*.

Classes that extend a class with private variables cannot access them either. They, like all users, must use *accessor functions* to access a class's private variables. For example `set_f_temp()` and `get_f_temp()` are accessor functions.

Many classes, including the SystemVerilog UVM classes, provide accessor functions that start with `set_` or `get_`, but Python provides a cleaner way of accessing private variables using *properties*.

Properties

Python provides additional support for accessor methods with the `@property` decorator. This decorator allows us to create accessor methods that appear to be a data attribute to a programmer using the class.

We use the `@property` decorator in front of the "get" method. Then, we use `@<variable_name>.setter` in front of the "set" method.

In figures 4-6, the `Temperature` class creates the `c_temp` and `f_temp` properties. The `c_temp()` and `f_temp()` methods raises `ValueError` if the temperature has not been set. This check is an example of how we can do error checking in properties. We store the temperature in `self.__temp` in Celsius.

```
# Figure 4: Defining properties
class Temperature:
    def __init__(self, c_temp=None) :
        self.__temp = c_temp

    @property
    def c_temp(self) :
        if self.__temp is None:
            raise ValueError("Temperature not set")
        return self.__temp

    @c_temp.setter
    def c_temp(self, c_temp) :
        self.__temp = c_temp
```

In figure 5, we add properties to `Temperature` for `f_temp` that convert Fahrenheit temperatures to and from Celsius.

```
# Figure 5: Creating additional properties

    @property
    def f_temp(self) :
        if self.__temp is None:
            raise ValueError("Temperature not set")
            return None
        return self.__temp * 9 / 5 + 32

    @f_temp.setter
    def f_temp(self, f_temp) :
        self.__temp = (f_temp - 32) * 5 / 9

    def __str__(self) :
```



```
return f"{self.c_temp:.1f} C == {self.f_temp:.1f} F"
```

The code in figure 5 creates a `__str__()` magic method that `print()` calls to print the object. The output shows the temperature in Fahrenheit and Celsius.

The code in figure 6 uses the `Temperature` class properties. First, we instantiate the `Temperature` class with the temperature set to `0` and print it. Then we instantiate the `Temperature` class with no temperature and try to print it, causing a `ValueError`.

```
# Figure 6: # Figure 6: Printing the Temperature object
# Exception if temperature not set
```

```
tt = Temperature(0)
print("tt:", tt)
tt = Temperature()
print("unset tt:", tt)
--
tt: 0.0 C == 32.0 F
unset tt:
-----
ValueError Traceback
      37 print("tt:", tt)
      38 tt = Temperature()
----> 39 print("unset tt:", tt)
```

Summary

Underscores before variable names designate protected and private data attributes and methods. The underscores tell users that they should not be accessing these class members directly.

One leading underscore is a convention that says, "Do not use this variable," but Python does nothing to enforce this. Two leading underscores make a variable private, and Python enforces this by mangling the variable name so those outside the class cannot access the variable.

You should provide accessor methods for protected and private variables. You make the accessor methods look like variables using the `@property` and `@<property>.setter` decorators.

Inheritance

If you find yourself leveraging existing code by copying and modifying it, you are almost certainly doing it wrong. For example, here is the wrong way to create a `Dog` class and a `Cat` class.

```
# Figure 1: What NOT to do

class Dog:
    def make_sound(self) :
        print("The dog says, 'bow bow'")

class Cat:
    def make_sound(self) :
        print("The cat says, 'mião'")

Dog().make_sound()
Cat().make_sound()
--
The dog says, 'bow bow'
The cat says, 'mião'
```

The problem with the code in figure 1 is that you'll have to write the code twice if you start adding other behaviors such as sleeping or eating. Then you'll have to maintain it twice. Any bug you find will have to be fixed twice.

Inheritance allows us to avoid copying code and modifying it by creating a *base class* that contains shared behavior. To use inheritance, you recognize that one class is a specialization of a broader class, which becomes the base class. We call the extension a *child class*. In figure 2 we see that a dog is an animal, and a cat is an animal, so we can create a base class named `Animal` and extend it to create child classes `Dog` and `Cat`.

```
# Figure 2: Inheritance avoids copying

class Animal:
    def __init__(self) :
        self.species = None
        self.sound = None
    def make_sound(self) :
        print(f"The {self.species} says '{self.sound}' ")

class Dog(Animal) :
    def __init__(self) :
        self.species = "dog"
        self.sound = "bow bow"

class Cat(Animal) :
    def __init__(self) :
        self.species = "cat"
        self.sound = "mião"

Dog().make_sound()
```

```
Cat().make_sound()
--
The dog says, 'bow bow'
The cat says, 'mião'
```

The code in figure 2 demonstrates simple inheritance. The `Dog` class definition has the `Animal` class in parentheses as in `class Dog(Animal) : .` Putting `Animal` in parentheses means that the `Dog` inherits the `make_sound()` method from the `Animal` method.

Also, notice that the `Dog` and `Cat` classes *override* the `__init__()` method by defining it. So when you create `Dog` or `Cat`, they call their `__init__()` method instead of the `__init__()` in `Animal`. But they don't override `make_sound()`, so when you call `make_sound()` in `Dog` and `Cat`, they run the inherited method from `Animal`.

Inheritance in Python

Inheritance in Python is different than inheritance in C or SystemVerilog. Python classes do not declare variables, and their extensions do not inherit variables. Python classes inherit only methods.

When you call a method in an object, Python searches the object's class for the method. If it does not find it, it searches the parent class for the method, and if it doesn't find it there, it follows the list of extended classes until it reaches the end and declares an error.

For example, there are dogs, and then there are small dogs. Small dogs don't say *woof woof* or *bow bow*. They say *yap yap*. The code in figure 3 captures this by replacing a `Cat` with a `SmallDog` class⁷ that extends `Dog`.

```
# Figure 3: Forgetting to set a data attribute

class Animal:
    def __init__(self) :
        self.species = None
        self.sound = None
    def make_sound(self) :
        print(f"The {self.species} says '&apos;{self.sound}&apos; ")

class Dog(Animal) :
    def __init__(self) :
        self.species = "dog"
        self.sound = "bow bow"

class SmallDog(Dog) :
    def __init__(self) :
        self.sound = "yap yap"

SmallDog().make_sound()
```

Figure 4 shows an error that would surprise a SystemVerilog programmer who would expect `SmallDog` to inherit the variable `self.species`.

Figure 4: A surprising error for an SV programmer

```
--
AttributeError Traceback
    15 self.sound = "yap yap"
    16
--> 17 SmallDog().make_sound()

in make_sound(self)
    4 self.sound = None
    5 def make_sound(self) :
----> 6 print(f"The {self.species} says '{self.sound}'")
    7
    8 class Dog(Animal) :
```

We got this attribute error because the `__init__()` function in `SmallDog` did not set the `self.species` data attribute. So while `Dog` has a `species` data attribute, `SmallDog` does not.

To solve this problem, we need to have `SmallDog.__init__()` call `Dog.__init__()` method. One way to do that is with the `super()` function.

The super() function

When you call a class's method, Python searches the class for the method, and if it does not find the class it searches the parent classes.

Python calls this method search path the *method resolution order* (MRO). Every class has a class method named `mro()` that returns the method resolution order for that class:

Figure 5: The `SmallDog`'s method resolution order.

```
print(SmallDog.mro() )
--
[<class '__main__.SmallDog'>,
 <class '__main__.Dog'>,
 <class '__main__.Animal'>,
 <class 'object'>]
```

We see that the `SmallDog` class searches itself, then `Dog`, then `Animal`, and finally `object`. So when we called `self.__init__()` in a `SmallDog` object, Python found the method in `SmallDog` and stopped looking.

We got an `AttributeError` when we called `self.make_sound()` because the `SmallDog.__init__()` method did not call its parent class's initialization routine and so did not set `self.species`. Let's fix that using the `super()` function as shown in figure 6.

Figure 6: `SmallDog.__init__()` calling its parent's `__init__()`

```
class SmallDog(Dog) :
    def __init__(self) :
        super().__init__()
        self.sound = "yap yap"

SmallDog().make_sound()
--
The dog says 'yap yap'
```

The `super()` function looks at the `mro` list and, if the argument list is empty, returns the class after the one calling `super()`. If you pass `super()` one of the MRO entries as an argument, it will return the MRO entry after the one you passed.

We called `super()` with no argument and the class after `SmallDog` in the `SmallDog.mro` list is `Dog`. So, `super().__init__()` calls the `__init__()` method in `Dog`.

You should make it a habit to call `super().__init__()` whenever you override an `__init__()` method. You must do this for **pyuvm** to work correctly.

Calling methods using class objects

The code in figure 6 navigated the method resolution order using the `super()` function to skip over classes in the MRO list. However, we can be more explicit. We can call the function in the class directly and pass it the `self` variable explicitly. In figure 7, we call `Dog.__init__()` directly using the `Dog` class object.

```
# Figure 7: Calling methods directly using class objects
```

```
++ snipped identical code ++
```

```
class SmallDog(Dog) :
    def __init__(self) :
        Dog.__init__(self)
        self.sound = "yap yap"

SmallDog().make_sound()
--
The dog says 'yap yap'
```

The advantage of this approach over using `super()` is that you know which copy of `__init__()` you are calling.

Multiple Inheritance

We have multiple roles in our lives. For example, we can be a parent who is also a firefighter. Most object-oriented programming languages capture this fact using *multiple inheritance*. (We have to say most because SystemVerilog does not have multiple inheritance.)

Let's take the notion of our roles to create a multiple inheritance example. Consider the `Human` class that sets a human's name and age and has a `say_age()` method.

```
# Figure 8: The Human base class
```

```
class Human:
    def __init__(self, name, age) :
        self.name = name
        self.age = age

    def say_age(self) :
        print(f"{self.name} is {self.age} years old.")

human = Human("Joe", 23)
human.say_age()
--
Joe is 23 years old.
```

Now we can extend `Human` to create different new classes: `Parent` and `Firefighter`. The `Parent` class defines the `kiss()` method and the `Firefighter` class defines the `hose()` method.

```
# Figure 9: Defining different roles for Humans
```

```
class Parent(Human) :
    def kiss(self) :
        print(f"{self.name} gives the baby a kiss.")

class Firefighter(Human) :
    def hose(self) :
        print(f"{self.name} sprays water.")
```

In figure 10 we instantiate a 35-year-old person named `Pat` who is both a parent and a firefighter. One might represent this by creating two objects with the same name and age.

```
# Figure 10: Defining a parent who is also a firefighter
```

```
pp = Parent("Pat", 35)
ff = Firefighter("Pat", 35)
pp.kiss()
pp.say_age()
ff.hose()
ff.say_age()
--
Pat gives the baby a kiss.
Pat is 35 old.
Pat sprays water.
Pat is 35 old.
```

We've created two objects to represent Pat's life, which is inefficient and aesthetically wrong because Pat is one person. Instead, let's create a new class for someone who lives as Pat does. The diagram in figure 11 illustrates the solution. We create a new class named `FirefighterWithKids` that inherits from `Parent` and `Firefighter`. This is multiple inheritance.

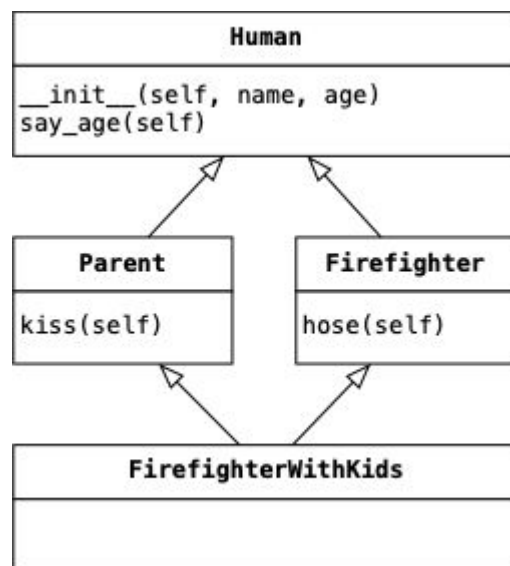


Figure 11: Multiple inheritance

The code in figure 12 defines `Human` and extends it to create the roles `Parent` and `Firefighter`. Then it defines `FirefighterWithKids` which inherits from both `Parent` and `Firefighter`. This is multiple inheritance.

```
# Figure 12: Defining FireFighterWithKids
# using multiple inheritance
```

```
class Human:
    def __init__(self, name, age) :
        self.name = name
        self.age = age
```

```

    def say_age(self) :
        print(f"{self.name} is {self.age} old.")

class Parent(Human) :
    def kiss(self) :
        print(f"{self.name} gives the baby a kiss.")

class Firefighter(Human) :
    def hose(self) :
        print(f"{self.name} sprays water.")

class FirefighterWithKids(Parent, Firefighter) :
    ...

```

The `FirefighterWithKids.mro()` list shows where the class gets its methods.

```

# Figure 13: The FirefighterWithKids MRO
print(FirefighterWithKids.mro())
--
[__main__.FirefighterWithKids,
 __main__.Parent,
 __main__.Firefighter,
 __main__.Human,
 object]

```

The `FirefighterWithKids.mro()` list shows that we look for methods in the `FirefighterWithKids` class first, and then we search through the `Parent` then `Firefighter` class, and finally the `Human` class. The order is `Parent` followed by `Firefighter` because that's the order between the parentheses in the `class FirefighterWithKids(Parent, Firefighter)` statement.

We now have a class, `FirefighterWithKids`, that can kiss the baby and spray the hose as in figure 14.

```

# Figure 14: Creating a FirefighterWithKids

pat = FirefighterWithKids("Pat", 35)
pat.say_age()
pat.kiss()
pat.hose()
--
Pat is 35 old.
Pat gives the baby a kiss.
Pat sprays water.

```

Multiple Inheritance and `__init__()`

The code in Figure 14 demonstrates an important multiple inheritance design rule for the `__init__()` method. As they say in the movie *Highlander*, "There can be only one."

The preceding design placed the `__init__()` in the base class, and its children used the data attributes the parent defined.

Sometimes you can't define all the data attributes in the ultimate parent class. The code in figure 16 adds new methods to `Parent` and `Firefighter` that need data unique to their roles. The `Parent.feed()` method uses the number of children, and the `Firefighter.drive()` method uses the truck number. The `Human` cannot have the `__init__` because not every human has a truck number.

We solve this problem by moving the `__init__()` from the top of the class hierarchy to the bottom.

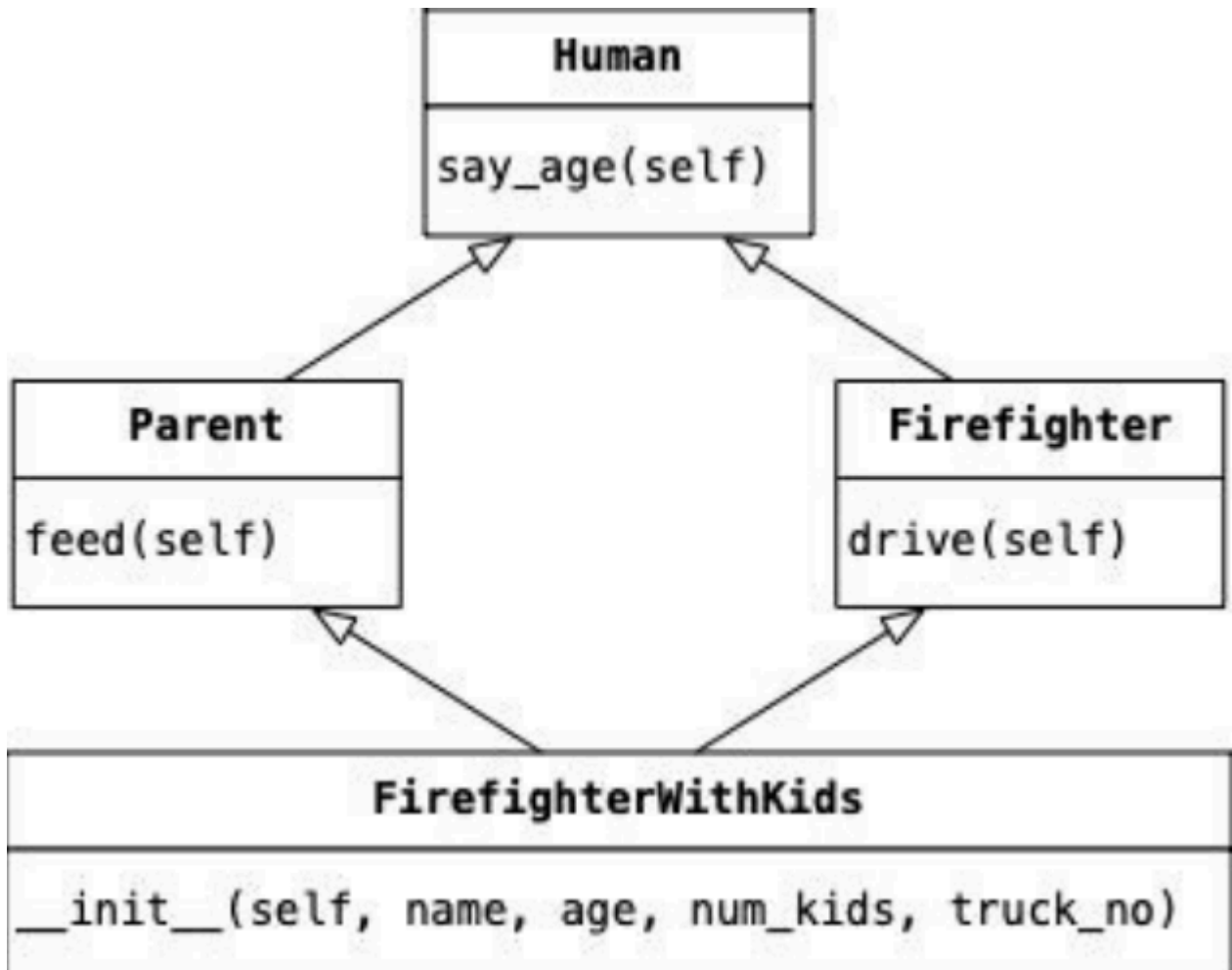


Figure 15: Putting `init()` in the bottom class

Now the `Human`, `Parent`, and `Firefighter` classes depend upon their child classes to set their data attributes as in figure 16.

Figure 16: Base classes with no `__init__()` functions

```
class Human: # No __init__()
    def say_age(self) :
        print(f"{self.name} is {self.age} old.")

class Parent(Human) : # No __init__()
    def feed(self) :
        print(f"{self.name} feeds {self.num_kids} children")

class Firefighter(Human) : # No __init__()
    def drive(self) :
        print(f"{self.name} drives truck {self.truck_no}")
```

Now we combine `Parent` and `Firefighter`.

```
class FirefighterWithKids(Parent, Firefighter) :
    def __init__(self, name, age, num_kids, truck_no) :
        self.name = name
        self.age = age
        self.truck_no = truck_no
        self.num_kids = num_kids

pat = FirefighterWithKids(name="Pat", age=35, num_kids=3, truck_no=77)
pat.say_age()
pat.feed()
pat.drive()
--
Pat is 35 old.
Pat feeds 3 children
Pat drives truck 77
```

The `Human`, `Parent`, and `Firefighter` classes can only function if you extend them because they depend upon their child classes to implement the `__init__()` method. Classes that depend upon their children to implement needed methods are called *abstract classes*. In figure 16, the `FirefighterWithKids` class implements `__init__()`, so it is called a *concrete* class.

Summary

Python classes inherit methods from their parent classes using the method resolution order (MRO) to find the method. Python classes do not inherit data members. These must be set in `__init__()`. Use `super()` to call the parent's version of an overridden method including `__init__()`.

Python supports multiple inheritance. When you design a class hierarchy using multiple inheritance, you should have only one `__init__()` method in the hierarchy, either at the top of the hierarchy or at the bottom.

Design patterns

Design Patterns are object-oriented tricks used so often that people write entire books about them. Recognizing the tricks as patterns, and naming the patterns, allows programmers to communicate design intentions more easily.

It's worth investing in a good book on design patterns. My favorite is named *Head First Design Patterns* by Freeman, Bates, Sierra, and Robson. I like it because it has a lot of pictures, though its examples are in Java.

The singleton pattern

The *singleton* design pattern provides a way to share global data cleanly. It is a class that returns the same handle every time it's called instead of creating a new object.

The code in figure 1 defines an empty class named `MyClass`. When we call `MyClass()` by appending parentheses to the class name, it returns a new `MyClass` object. We can see that these objects are unique by using the `id()` function, which returns the object ID number.

```
# Figure 1: Calling a class object using ()
```

```
class MyClass:
    ...

nc1 = MyClass()
nc2 = MyClass()
print("id(nc1) :", id(nc1) )
print("id(nc2) :", id(nc2) )
--
id(nc1) : 140221022182848
id(nc2) : 140221022380096
```

A class that implements the *singleton* pattern changes this behavior. It returns the same handle no matter how many times you call it. That way, code across the program can share this object.

At this point, I'll ask for your indulgence to accept a little **pyuvm** magic. Python can implement Singletons in many ways, but it's beyond the scope of this book to discuss them. We're going to leverage the approach defined in **pyuvm**.

We define a singleton class using the `metaclass = argument`⁸ and passing it `Singleton` imported from **pyuvm**. Now, when we call `MySingleton` twice, it returns the same object, as we see from the `id()` function in figure 2.

```
# Figure 2: The singleton returns the same object for every call
```

```
from pyuvm import Singleton

class MySingleton(metaclass=Singleton) :
    ...

ms1 = MySingleton()
ms2 = MySingleton()
```

```
print("id(ms1) :", id(ms1) )
print("id(ms2) :", id(ms2) )
--
id(ms1) : 140198470115872
id(ms2) : 140198470115872
```

pyuvm uses several singleton classes.

The factory pattern

We use the factory pattern to create different classes based on user input or random data. Since we can't predict which class we'll need, we define all the possible classes and then pick one based on the data.

I first learned about factory patterns in a book about writing a space war game. You couldn't tell which weapon the user would fire, so you used a factory to create an object based on the weapon chosen. Let's see how we do that.

Our example game defines three kinds of weapons that print out different sounds. The Federation has the `Phaser` that prints "Zzzap!" and the `PhotonTorpedo` that prints "Pew! Pew! "

The Romulans have the `GiantRedBall` that prints "WHOOSH BOOM!"

We'll use a `list` to implement a factory in this example. The `weapon` class implements `sound_effect` , then `Phaser` , `PhotonTorpedo` , and `GiantRedBall` extend `weapon` . They set their sound effect in their `__init__()` method.

We store the classes in a list named `factory_list` . Notice that we are storing the classes in the list, not instances of the classes.

Figure 3: Creating a list of classes to use as a factory

```
import random

class weapon:
    def sound_effect(self) :
        print(self.sound)

class Phaser(weapon) :
    def __init__(self) :
        self.sound = "Zzzap!"

class PhotonTorpedo(weapon) :
    def __init__(self) :
        self.sound = "Pew! Pew!"

class GiantRedBall(weapon) :
    def __init__(self) :
        self.sound = "WHOOSH BOOM!"

factory_list=[Phaser, PhotonTorpedo, GiantRedBall]
```

Now we're ready to use `factory_list`. We loop six times and choose a random number from `0` to `2`. We use the `weapon_no` to pick a weapon class from the `factory_list`, storing it in `weapon_cls`. Then we call `weapon_cls` using parentheses to get the `weapon` object. Finally, we call `sound_effect()` using the `weapon`.

Figure 4: Testing the factory list by calling the class in the list

```
for _ in range(6) :
    weapon_no = random.randint(0,2)
    weapon_cls = factory_list[weapon_no]
    weapon = weapon_cls()
    weapon.sound_effect()

--
WHOOSH BOOM!
Zzzap!
Pew! Pew!
Pew! Pew!
Pew! Pew!
Zzzap!
```

Creating different kinds of objects based on incoming data is the essence of a factory class. We don't hard code our class instantiations. Instead, we choose them at run time.

Summary

This chapter taught us about the singleton and factory design patterns. We've now completed our overview of Python. Upcoming chapters explain how to connect Python to a simulator using coroutines and **cocotb**.

Coroutines

Imagine you're writing a program such as the old adventure game *Rogue* that takes keystrokes as commands. For example, the user presses `j` to move the hero left, `;` to move the hero right, etc. You need to write software that waits for a key to be pressed and processes the keystroke.

Such software usually has a construct called an *event loop* that blocks code until an event happens and then responds. Fortunately, we don't need to write such a construct in Python because there are modules that provide event loops. The objects that go into the event loops and wait for events are called *coroutines*.

The most commonly used event-loop module is named **asyncio** since it's intended to process asynchronous I/O, as in the game example. But we're not going to use that module. Instead, we're going to use a module named **cocotb**, whose event loop works with a simulator. ⁹

An event loop for simulation

The **cocotb** module provides an event loop that interacts with an RTL simulator. Simulation events include an edge of the simulated clock, some amount of simulated time, or user-defined events that trigger when something interesting happens.

cocotb needs us to do two things:

1. Define coroutines that run when a simulation event happens.
2. Identify a top-level coroutine (called a test) that kicks off the simulation.

Defining coroutines

Coroutines are very similar to functions in that we define them using the `def` statement.

In figure 1 we first tell **cocotb** that the following coroutine is a test by using the `@cocotb.test()` decorator. Then, we define a coroutine by adding the `async` keyword before the function `def` keyword.

We see the `async` keyword in the `async def hello_world(_)` example. The `_` variable inside the argument list is a convention that says we're not using the argument.

Figure 1: Hello world as a coroutine

```
import cocotb

@cocotb.test()
async def hello_world(_):
    """Say hello!"""
    logger.info("Hello, world.")
```

The coroutine contains a docstring in triple-quotes that **cocotb** prints when **cocotb** runs the test. The `logger` object allows us to log messages to the screen using the simulation time stamp. We will discuss logger objects in a later chapter.

We run the simulation, and **cocotb** puts the test coroutine on its event loop, where it runs immediately. Not surprisingly, we pass the test.

```
# Figure 2: Hello world from the simulator
```

```
0.00ns INFO running hello_world (1/7)
          Say hello!
0.00ns INFO Hello, world.
```

Note: The cocotb outputs in this book reflect cocotb 1.7.0.dev0, the latest version in April of 2022. They may change in the future.

Awaiting simulation time

As the name implies, event-driven simulation consists of code waiting for events. Simulation languages such as VHDL and SystemVerilog have statements that wait for events. These languages also define functions that consume time. We call such a time-consuming function a *process* in VHDL, a *task* in SystemVerilog, and a *coroutine* in Python.

Figure 3 contains a VHDL example where our process waits for two nanoseconds before printing its message.

```
-- Figure 3: VHDL waits for 2 nanoseconds
```

```
process is
begin
    wait for 2 ns;
    report "I am DONE waiting!";
    wait;
end process;
```

The VHDL `process` automatically loops back to the top and would run forever if we didn't stop it with the second `wait` statement. SystemVerilog has different constructs for looping and non-looping behavior. In figure 4 we use an `initial` block to run once in SystemVerilog ¹⁰.

```
// Figure 4: SystemVerilog waits for two nanoseconds
```

```
initial begin
    #2ns;
    $display("I am DONE waiting!") ;
end
```

We can write the same behavior in Python using **cocotb**. In figure 5, we import a **cocotb** coroutine named `Timer()` to wait for two nanoseconds. Then we use the `await` keyword to block our execution until `Timer()` returns control two simulated nanoseconds later. The logger output in figure 5 shows that we started at time `0.00ns` and logged the message at `2.00ns`.

```

# Figure 5 Python waits for 2 nanoseconds
import cocotb
from cocotb.triggers import Timer

@cocotb.test()
async def wait_2ns(_) :
    """waits for two ns then prints"""
    await Timer(2, units="ns")
    logger.info("I am DONE waiting!")
--
2.00ns INFO I am DONE waiting!

```

Starting tasks

Simulating hardware often means launching behavior in the background and interacting with it. For example, you may want to test a design where you put data into one side of the device under test (DUT) and want to see it come out the other. In this case, you'd launch a coroutine that runs on its own and waits for output. We call coroutine running in the background a *running task* or a *task*.

SystemVerilog launches tasks using the `fork` and `join` keywords. **cocotb** launches tasks with the `cocotb.start_soon()` function.

You pass `cocotb.start_soon()` a handle to a coroutine, and it schedules the task to run and returns a handle to a `RunningTask` object, also called a `task`. You can `await` the task, kill it, get a value from it, or ignore it.

In figure 6, we create a coroutine to run in the background and launch it in various ways to demonstrate these behaviors. The `counter` counts from `1` up to `count` with a `delay` of delay ns between counts.

```

# Figure 6: counter counts up with a delay

import cocotb
from cocotb.triggers import Timer, Combine
import logging

# The code that gets the logger object
# has been snipped and is discussed later in the book.

async def counter(name, delay, count) :
    """Counts up to the count argument after delay"""
    for ii in range(1, count + 1) :
        await Timer(delay, units="ns")
        logger.info(f"{name} counts {ii}")

```

Now we can try out different ways of controlling running coroutines.

Ignoring a running task

The first thing we'll do is launch a coroutine and ignore it. So here we launch a coroutine that will count to 3.

Figure 7: Launching a task and ignoring it

```
@cocotb.test()
async def do_not_wait(_) :
    """Launch a counter"""
    logger.info("start counting to 3")
    cocotb.start_soon(counter("simple count", 1, 3) )
    logger.info("ignored running_task")

--
INFO start counting to 3
INFO ignored running_task
```

Note: The output messages do not include the simulation time, test name, and other irrelevant information to save space on the page.

Well, that was unsatisfying. We launched our counter, and it did not count. That is because we didn't wait for it. We ended the simulation almost as soon as it launched.

There are situations where we'll launch a task and ignore it, for example, when the coroutine contains an infinite `while` loop. In those cases, the task will run until the simulation ends.

However, we should wait for the running task to finish in this case.

Awaiting a running task

`cocotb.start_soon()` returns a handle to a running task. In figure 8 we wait for the task to finish by getting a `RunningTask` handle from `cocotb.start_soon()` and using it in an `await` statement.

Figure 8: Waiting for a running task

```
@cocotb.test()
async def wait_for_it(_) :
    """Launch a counter"""
    logger.info("start counting to 3")
    running_task = cocotb.start_soon(counter("simple count", 1, 3) )
    await running_task
    logger.info("waited for running task")

--
2.00ns INFO start counting to 3
3.00ns INFO simple count counts 1
4.00ns INFO simple count counts 2
5.00ns INFO simple count counts 3
5.00ns INFO waited for running task
```

Running tasks in parallel

In the figure 9 example, we start `counter` twice. Once for The *Count* from Sesame Street and the other one for *Mom* (who is going to "count to three, and then you are in big trouble, mister!").

We import the `Combine()` coroutine from `cocotb.triggers`. It blocks until all the tasks in its argument list have returned ¹¹. Now we can keep the simulation running until both tasks have finished.

```
# Figure 9: Mom and The Count count in parallel

@cocotb.test()
async def counters(_) :
    """Test that starts two counters and waits for them"""
    logger.info("The Count will count to five.")
    logger.info("Mom will count to three.")
    the_count = cocotb.start_soon(counter("The Count", 1, 5))
    mom_warning = cocotb.start_soon(counter("Mom", 2, 3))
    await Combine(the_count, mom_warning)
    logger.info("All the counting is finished")
```

Since The Count waits for one ns between counts and Mom waits for two ns. Figure 10 shows interleaved counting between The Count and Mom.

```
# Figure 10: Mom and The Count's interleaved output

2.00ns INFO The Count will count to five.
2.00ns INFO Mom will count to three.
3.00ns INFO The Count counts 1
4.00ns INFO Mom counts 1
4.00ns INFO The Count counts 2
5.00ns INFO The Count counts 3
6.00ns INFO Mom counts 2
6.00ns INFO The Count counts 4
7.00ns INFO The Count counts 5
8.00ns INFO Mom counts 3
8.00ns INFO All the counting is finished
```

cocotb also provides a `First()` task that returns when the first task has been completed. ¹²

Returning values from tasks

Tasks can return values. You get the returned value by placing the `await` keyword on the right side of the equal sign.

```
# Figure 11: Awaiting in an assignment statement

myval = await task_variable
```

We use `await` on the right side of the equal sign whenever we get a value from a coroutine. We can also use it to get a value from a task. For example, here is a coroutine that waits for `delay` and then returns the increment of a number.

```
# Figure 12: A coroutine that increments a number
# and returns it after a delay

async def wait_for_numb(delay, numb) :
    """waits for delay ns and then returns the increment of the number"""
    await Timer(delay, units="ns")
    return numb + 1
```

We call `cocotb.start_soon()` to start this task and await its result twice.

```
# Figure 13: Getting a return value by awaiting the
# returned RunningTask object

@cocotb.test()
async def inc_test(_) :
    """Demonstrates start_soon() return values"""
    logging.info("sent 1")
    inc1 = cocotb.start_soon(wait_for_numb(1, 1) )
    nn = await inc1
    logging.info(f"returned {nn}")
    logging.info(f"sent {nn}")
    inc2 = cocotb.start_soon(wait_for_numb(10, nn) )
    nn = await inc2
    logging.info(f"returned {nn}")
```

In the output, we see the number is incremented, and that time advances 1 ns for the first task and 10 ns for the second.

```
# Figure 14: Incrementing with a delay

1ns INFO sent 1
2ns INFO returned 2
2ns INFO sent 2
12ns INFO returned 3
```

Killing a task

There is usually no need to kill a task, as they all stop running when the simulation ends. However, there may be cases where you want to kill a task. You do that by calling the `kill()` method on a `RunningTask()` object as in figure 15.

```
# Figure 15: Killing a task

@cocotb.test()
async def kill_a_running_task(_) :
    """kill a running task"""
    kill_me = cocotb.start_soon(counter("kill me", 1, 1000) )
    await Timer(5, units="ns")
    kill_me.kill()
    logger.info("killed the long-running task.")
```

```
--  
23.01ns INFO kill me counts 1  
24.01ns INFO kill me counts 2  
25.01ns INFO kill me counts 3  
26.01ns INFO kill me counts 4  
27.01ns INFO killed the long-running task.
```

Summary

This chapter examined coroutines, event-driven functions, and **cocotb**, a module that provides an event loop that interacts with a simulator. We learned to use **cocotb** coroutines such as `Timer` to await the simulated passage of time using the `await` keyword.

We also learned how to use the `cocotb.start_soon()` function to run coroutines in the background as tasks. We use **cocotb** coroutines such as `Combine()` and `First()` to wait for background coroutines to complete.

We saw how to get a value back from a coroutine by using the `await` keyword on the right side of the assignment, and we learned how to kill a task.

cocotb Queue

Starting a task would be of limited use if tasks could not communicate. So as we begin work on testbenches, we'll be creating tasks that share data.

The Python `queue` class allows task communication. The **cocotb** extension of `queue` enables this class to work within **cocotb**. We get the **cocotb** `queue` by importing it from the module `cocotb.queue`.

```
# Figure 1: Importing cocotb Queue classes
from cocotb.queue import Queue, QueueEmpty, QueueFull
```

This `import` statement imports the **cocotb** version of the `queue` alongside the `QueueFull` and `QueueEmpty` exceptions that we'll use in our examples.

Task communication

Communicating tasks need to share data in the correct order. If a producer sends `1, 2, 3`, the consumer must receive `1, 2, 3` in that order. Since the tasks are running in parallel, we can't know whether the producer will send the data before the consumer reads it. `Queue` objects solve this problem by making consumers wait until the producer has sent the data or forcing the producer to wait until the consumer is ready to receive the data.

We can handle the waiting between tasks in two ways. Either the task blocks until it can send or receive data, or the task polls the queue and waits on another coroutine before polling the queue again. The first is called *blocking communication* and the second is called, oddly enough, *nonblocking communication*.

Blocking communication

Queues implement blocking communication with two coroutines named `put()` and `get()`. We'll demonstrate these with two coroutines named `Producer` and `Consumer`. The `Producer` sends numbers through a queue, and the `consumer` reads them from a queue.

In figure 2 we define the `Producer` coroutine. It takes three arguments: the handle to a queue, the largest number to send, and the delay between sending the numbers. Finally, the `Producer` prints a log message when it successfully puts data into the queue.

```
# Figure 2: A coroutine using a Queue to send data
async def Producer(queue, nn, delay=None) :
    """Produce numbers from 1 to nn and send them"""
    for datum in range(1, nn + 1) :
        if delay is not None:
            await Timer(delay, units="ns")
        await queue.put(datum)
        logger.info(f"Producer sent {datum}")
```

The `Consumer` in figure 3 takes only a `queue` argument. The coroutine loops forever, getting numbers from the `queue` using `datum = await queue.get()` if the `queue` is empty, the `Consumer` loop blocks. It logs the times that it got the datum.

```
# Figure 3: A coroutine using a Queue to receive data
```

```
async def Consumer(queue) :  
    """Get numbers and print them to the log"""  
    while True:  
        datum = await queue.get()  
        logger.info(f"Consumer got {datum}")
```

An infinitely long queue

The simplest way to communicate between tasks is to create an infinitely long queue. The producer writes its data into the queue, and then the consumer reads all the data in the same order.

The code in figure 5 instantiates a `Queue` and passes it to the `Producer` and `Consumer` coroutines in `cocotb.start_soon()`. The `await Timer` statement gives the Consumer the time to empty the queue before the simulation finishes.

```
# Figure 4: An infinitely long Queue consumes no time
```

```
@cocotb.test()  
async def infinite_queue(_) :  
    """Show an infinite queue"""  
    queue = Queue()  
    cocotb.start_soon(Consumer(queue) )  
    cocotb.start_soon(Producer(queue, 3) )  
    await Timer(1, units="ns")  
  
--  
0ns INFO Producer sent 1  
0ns INFO Producer sent 2  
0ns INFO Producer sent 3  
0ns INFO Consumer got 1  
0ns INFO Consumer got 2  
0ns INFO Consumer got 3
```

The infinite `Queue` allows the `Producer` to run to completion, then hand off execution to the `Consumer`.

A Queue of size 1

If we want the producer to block until the consumer has processed the data, we set the queue `maxsize` to `1`. Setting the `maxsize` to `1` means the producer can put one number into the queue but blocks when it tries to `put()` the second number into the queue. The blocked `put()` returns after the consumer empties the queue.

The code is the same but for `Queue(maxsize=1)`.

```
# Figure 5: A Queue of size 1 can block when it is full
```

```
@cocotb.test()  
async def queue_max_size_1(_) :  
    """Show producer and consumer with maxsize 1"""  
    queue = Queue(maxsize=1)  
    cocotb.start_soon(Consumer(queue) )
```

```

cocotb.start_soon(Producer(queue, 3) )
await Timer(1, units="ns")
--
1.00ns INFO Producer sent 1
1.00ns INFO Consumer got 1
1.00ns INFO Producer sent 2
1.00ns INFO Consumer got 2
1.00ns INFO Producer sent 3
1.00ns INFO Consumer got 3

```

We see that the `Producer()` puts a `1` into the queue but blocks trying to put `2` because the queue is full. The `consumer()` then gets the `1` out of the queue but blocks when the queue is empty. The `Consumer` blocking gives the `Producer()` a chance to write the `2`. The `Producer` and `Consumer` alternate running and blocking, unlike the case with the infinite queue.

Queues and simulation delay

The `Producer` and `Consumer` in figure 5 consumed no simulation time. Here we'll demonstrate that they act the same way when consuming time. This test creates a `Producer` with a 5 ns delay between writes by passing a delay when calling the `Producer` class (defined in figure 2.) The result is the same but with the simulation time advancing.

```

# Figure 6: Demonstrating simulated time delays
# in Queue communication

@cocotb.test()
async def producer_consumer_sim_delay(_):
    """Show producer and consumer with simulation delay"""
    queue = Queue(maxsize=1)
    cocotb.start_soon(Consumer(queue) )
    ptask = cocotb.start_soon(Producer(queue, 3, 5) )
    await ptask
    await Timer(1, units="ns")
--
7.00ns INFO Producer sent 1
7.00ns INFO Consumer got 1
12.00ns INFO Producer sent 2
12.00ns INFO Consumer got 2
17.00ns INFO Producer sent 3
17.00ns INFO Consumer got 3

```

The test started running at 2 ns. The `Producer` waited for 5 ns, then put the 1 into the queue at 7 ns. While the `Producer` awaited the `Timer()`, the `Consumer` ran and read the number. The `Consumer` tried to read the following number but was blocked because the `Queue` was empty. Both tasks were blocked while the `Timer` counted off the 5 ns. The process repeated for all the numbers.

This section demonstrated coroutine communication using the blocking coroutines named `put()` and `get()`. But there are situations where we cannot block the testbench while waiting for data. For example, we may be in a loop that awaits the negative edge of the clock. If that loop blocked waiting for data, it would miss clock edges.

We solve this problem using nonblocking communication.

Nonblocking communication

In blocking communication the `Queue.put()` or `get()` block until the `Queue` is ready to communicate, no matter how long this takes.

In nonblocking communication, you poll the `queue` and block using other coroutines such as a `Timer` or `FallingEdge`.

The `put_nowait()` and `get_nowait()` functions provide the ability to check a `queue` and respond to the queue being full or empty. We do not await `put_nowait()` and `get_nowait()` because they are functions, not coroutines.

The code for `ProducerNowait()` in figure 7 loops through numbers and sends them to the `queue` using `put_nowait()`. If `put_nowait()` raises the `QueueFull` exception, the code skips over the 'break' line. Instead, the code jumps to the `except` block, where it waits for one simulated nanosecond. If `put_nowait()` returns normally, the code breaks out of the `while` loop.

```
# Figure 7: Putting objects in a Queue without blocking
async def ProducerNowait(queue, nn, delay=None) :
    """Produce numbers from 1 to nn and send them"""
    for datum in range(1, nn + 1) :
        if delay is not None:
            await Timer(delay, units="ns")
        while True:
            try:
                queue.put_nowait(datum)
                break
            except QueueFull:
                logger.info("Queue Full, waiting 1ns")
                await Timer(1, units="ns")
    logger.info(f"Producer sent {datum}")
```

At the other end, `ConsumerNowait` contains two `while` loops. The outer one loops forever, reading and printing data. The inner loop loops until it successfully reads data using `queue.get_nowait()` and breaks out of the loop. Otherwise, the `QueueEmpty` exception causes it to jump to the `except` block and wait for 2 ns.

```
# Figure 8: Getting objects from a Queue without blocking
async def ConsumerNowait(queue) :
    """Get numbers and print them to the log"""
    while True:
        while True:
            try:
                datum = queue.get_nowait()
                break
            except QueueEmpty:
                logger.info("Queue Empty, waiting 2 ns")
                await Timer(2, units="ns")
        logger.info(f"Consumer got {datum}")
```


The `cocotb.test()` code in figure 9 is the same, except for instantiating `ConsumerNowait` and `ProducerNowait`.

There is one other difference. Rather than calling `cocotb.start_soon()`, this code shows that since we don't need to do anything while `ProducerNowait` is running, we can `await` it directly. So we await `ProducerNowait(queue, 3)`, which creates the coroutine and puts it on the event queue. We get control back when `ProducerNowait` has done its work, and we await the `Timer` to let the `ConsumerNowait` finish.

```
# Figure 9: Running our nonblocking test
@cocotb.test()
async def producer_consumer_nowait(_) :
    """Show producer and consumer not waiting"""
    queue = Queue(maxsize=1)
    cocotb.start_soon(ConsumerNowait(queue))
    await ProducerNowait(queue, 3)
    await Timer(3, units="ns")
```

At time 0, The Producer puts 1 into the queue but then finds the queue is full, so it waits for 1 ns. The Producer waiting allows the Consumer to read the 1, but when the Consumer tries to read the following number, the queue is empty, so it waits for 2 ns.

```
# Figure 10: The Producer and Consumer waiting when the queue is full
0 ns INFO Producer sent 1
0 ns INFO Queue Full, waiting 1ns
0 ns INFO Consumer got 1
0 ns INFO Queue Empty, waiting 2 ns
1 ns INFO Producer sent 2
1 ns INFO Queue Full, waiting 1ns
2 ns INFO Consumer got 2
2 ns INFO Queue Empty, waiting 2 ns
2 ns INFO Producer sent 3
4 ns INFO Consumer got 3
```

The Producer waited for 1 ns at 0 ns, so we see it send the next number at 1 ns. Then the queue is full again. The consumer waited for 2 ns at 0 ns, and so it gets the number at 2 ns. Because the consumer always waits 2 ns it gets the next number at 4 ns.

Summary

This chapter taught us about the Python `Queue` and how simultaneously running tasks communicate. We also learned that **cocotb** had extended the class to create the `cocotb.queue.Queue` class to work with coroutines.

We imported the `cocotb.queue.Queue` and used it in a Producer and Consumer and demonstrated both the blocking and nonblocking modes of communication. Now we're ready to see **cocotb** simulating.

Simulating with cocotb

The code in figure 1 defines a “Hello, world” **cocotb** program.

```
# Figure 1: Hello world in cocotb

@cocotb.test()
async def hello_world(_) :
    """Say hello!"""
    logger.info("Hello, world.")
```

We discussed the `@cocotb.test()` decorator and the docstring, and we ignored the argument `_`. The unused argument in `hello_world(_)` provides the top of the simulation hierarchy. Usually, this is the device under test (DUT). In this chapter, name the argument `dut` and use it.

Verifying a counter

Figure 2 defines a SystemVerilog `counter` module to show how we interact with the DUT. You reset this counter by lowering `reset_n` and raising it again (`reset_n` is synchronous). The counter then counts one number per clock.

```
// Figure 2: A SystemVerilog counter

`timescale 1ns/1ns
module counter( input bit clk,
                input bit reset_n,
                output byte unsigned count) ;

    always @(posedge clk)
        count <= reset_n ? count + 1 : 'b0;

endmodule
```

We’ll use this to demonstrate using triggers and reading and setting values.

cocotb triggers

Triggers are **cocotb** coroutines that allow us to interact with the simulator timing. We use them to await the clock or any other signal, either by counting clock cycles or waiting for signal edges. The `cocotb.triggers` module contains the triggers. The `signal` provides the signal to watch:

- `Edge(signal)` —Await the next signal edge, either rising or falling.
- `RisingEdge(signal)` —Await the next rising edge of the signal.
- `FallingEdge(signal)` —Await the next falling edge of the signal.
- `Clockcycles(signal, num_cycles, rising=True)` —Await `num_cycles` rising or falling signal edges. `Clockcycles` counts rising edges by default.

Testing reset_n

In our first test, let's ensure that `reset_n` works correctly. We'll hold it low and make sure the counter does not count. Here, we break the code into sections to discuss aspects of a `cocotb` test.

Importing modules and objects

In figure 3, we import the `Clock` class from `cocotb` to generate a clock. Then we import `ClockCycles` and `FallingEdge` to coordinate our testbench with clock edges. We import the `get_int()` function and the `logger` object from `tinyalu_utils`.

Figure 3: Importing cocotb and tinyalu_utils resources

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, FallingEdge
from pathlib import Path
import sys
parent_path = Path("..") .resolve()
sys.path.insert(0, str(parent_path) )
from tinyalu_utils import get_int, logger
```

The tinyalu_utils module

The `tinyalu_utils.py` file sits in the directory above all the testbenches and contains resources used in TinyALU tests. We get our parent class directory by importing the `Path` class from `pathlib` and then using its `resolve()` function to get the full path to `".."` (our parent directory.)

We prepend the parent path to `sys.path` and then import `tinyalu_utils`.

The get_int() function

The `get_int()` function in figure 4 takes a signal name and returns its value as an integer. We read the signal value using `signal.value` data attribute and pass the value to `int()` to create an integer. If `signal.value` contains an `x` (unknown) or `z` (high impedance), `int()` raises a `ValueError` exception. We catch the exception using a `try/except` block and set the integer to `0`.

Figure 4: get_int() converts a bus to an integer
turning a value of x or z to 0

```
def get_int(signal) :
    try:
        int_val = int(signal.value)
    except ValueError:
        int_val = 0
    return int_val
```

The decision to return a `0` for `x` or `z` is testbench-specific. For example, other testbenches may want to let the `ValueError` stop the simulation.

The logger object

The Python `logging` module prints log messages. Within `cocotb`, it also prints simulation time stamps. The logger requires initialization. The `tinyalu_utils` module handles this initialization and defines an object named `logger` to provide all logging functions.

Figure 5: Setting up logging using the logger variable

```
import logging
logging.basicConfig(level=logging.NOTSET)
logger = logging.getLogger()
logger.setLevel(logging.DEBUG)
```

1. `import logging` —Import the Python logging module
2. `logging.basicConfig(level=logging.NOTSET)` —This allows all logging messages to be logged. We discuss logging levels in the *Logging* chapter later in the book.
3. `logger=logging.getLogger()` —Get a new object that will handle logging.
4. `logger.setLevel(logging.DEBUG)` —Allow `logger` to log most messages. The *Logging* chapter has details about `logging.DEBUG`

Starting the clock and running the test

In figure 6, we start the test. `Clock(dut.clk, 2, units="ns").start()` returns a coroutine that drives `dut.clk` with a 2 ns clock. We use `start_soon()` to launch the clock coroutine as a task.

Next we see how **cocotb** sets signal values. `dut.reset_n` is a handle to the `reset_n` port on the counter. `dut.reset_n.value = 0` sets the `dut.reset_n` signal to 0. Now the counter is reset.

Figure 6: Starting the clock

```
@cocotb.test()
async def no_count(dut) :
    """Test no count if reset is 0"""
    cocotb.start_soon(Clock(dut.clk, 2, units="ns").start() )
    dut.reset_n.value = 0
```

The clock is running, and the reset is low, so the counter should remain 0. We use the `clockcycles(dut.clk, 5)` coroutine to `await` five clock cycles. After that, we get the counter's value and assert that it is 0. The assertion in figure 7 tells **cocotb** whether the test passed.

Figure 7: wait for five clocks and check the output

```
await clockcycles(dut.clk, 5)
count = get_int(dut.count)
logger.info(f"After 5 clocks count is {count}")
assert count == 0
--
8.00ns INFO After 5 clocks count is 0
```

cocotb generated messages to tell us the test it was running and whether the test had passed. In this case, the test has passed. Our reset works.

Passing (or failing) a test

After logging the value, we check that it is zero as expected, using the Python `assert` statement. **cocotb** interprets `assert` as you'd expect. If the assertion fails, the test fails. Tests pass by default. It is good practice to place an `assert` statement at the end of a **cocotb** test.

Checking that the counter counts

This example uses **cocotb** to check that the counter counts when `reset_n` is `1`. Since the DUT runs on the positive edge of the clock, we'll use the negative edge of the clock to set and get values. Notice that we pass `rising=False` to the `ClockCycles()` coroutine when we `await` it.

The code in figure 8 waits for three clock cycles and checks that the `count` value is `3`.

```
# Figure 8: Testing that the counter counts

@cocotb.test()
async def three_count(dut) :
    """Test that we count up as expected"""
    cocotb.start_soon(Clock(dut.clk, 2, units="ns").start())
    dut.reset_n.value = 0

    await FallingEdge(dut.clk)
    dut.reset_n.value = 1
    await ClockCycles(dut.clk, 3, rising=False)
    count = get_int(dut.count)
    logger.info(f"After 3 clocks, count is {count}")
    assert count == 3

--
16.00ns INFO After 3 clocks, count is 3
```

A common coroutine mistake

Forgetting to use the `await` keyword on a coroutine is a common mistake. The simulator will create a coroutine instance and eventually delete it, but it will not await it.

For example, this code in figure 9 wants to wait for six clocks but waits for zero clocks.

```
# Figure 9: Forgetting the await statement
# is a common mistake

@cocotb.test()
async def oops(dut) :
    """Demonstrate a coroutine mistake"""
    cocotb.start_soon(Clock(dut.clk, 2, units="ns").start() )
    dut.reset_n.value = 0
    clockCycles(dut.clk, 6)
    logger.info("Did not await")

--
```

```
81.00ns INFO    Starting test: "oops"
                                   Description: Demonstrate a coroutine mistake
81.00ns INFO    Did not await
82.00ns INFO    Test Passed: oops
```

Notice that the time advanced only one nanosecond. Keep this mistake in mind when you have a testbench that doesn't seem to be advancing correctly.

Summary

cocotb passes the test coroutine a `dut` argument. You can use the `dut` object to access the clock and await `RisingEdge`, `FallingEdge`, `Edge`, or `ClockCycle`.

You can set values in the `dut` with simple assignments to the signal using `dut.<signal>.value = <value>` and you can read values from the `dut` object using the `value` property as in `<var> = dut.<signal>.value`. Instantiating a coroutine such as `RisingEdge()` does not cause Python to await it. You must explicitly use the `await` keyword like this: `await RisingEdge(dut.clk)`.

Basic testbench: 1.0

The rest of the book uses a simple DUT to teach `cocotb` coding techniques and then the Universal Verification Methodology (UVM). All chapters from this point on will write increasingly modular and maintainable versions of a testbench that verifies the TinyALU.

The TinyALU

The TinyALU is, as its name implies, a tiny ALU that has four operations. It takes two 8-bit legs named `A` and `B` as input and produces a 16-bit `result`. Here is its block diagram.

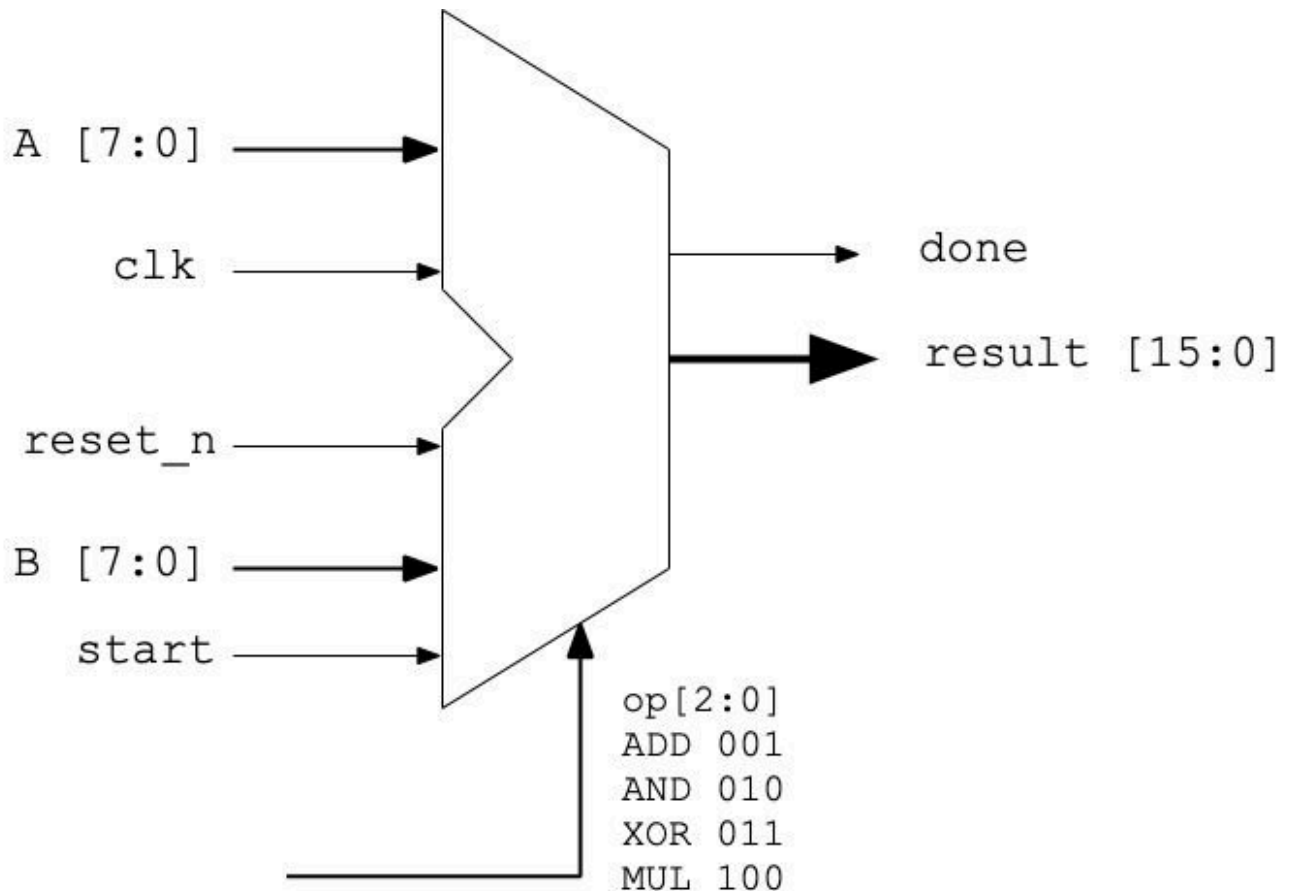


Figure 1: The TinyALU

The TinyALU has four operations: `ADD`, `AND`, `XOR`, and `MUL`, which allow it to implement any algorithm. The `ADD`, `AND` and `XOR` operations take one clock cycle. The `MUL` takes three clock cycles. Here is the timing diagram.

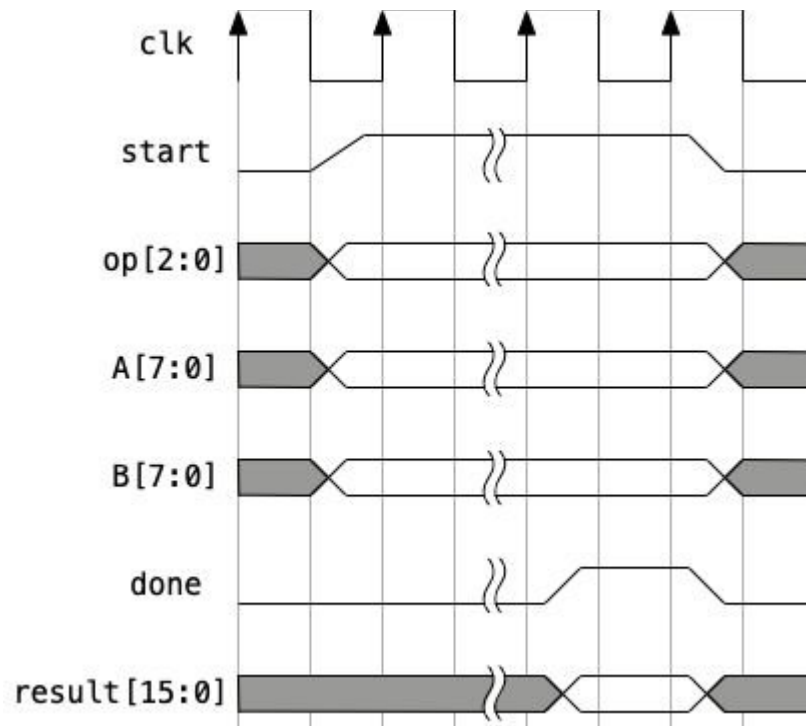


Figure 2: The TinyALU timing diagram

The user puts the operands on the `A` and `B` busses and the operation on the `op` bus, then raises the `start` signal. Thus, the waveform diagram shows that the ALU command is valid in a clock cycle where the `start` signal was `0` in the previous clock cycle and is `1` on this clock cycle. Therefore, the user must keep the `start` signal at `1` until the `done` signal goes to `1`.

The TinyALU calculates and places the result on the `result` bus while raising the `done` signal. Therefore `result` is valid in any clock cycle where the `done` signal is `0` in the previous clock cycle and is `1` on this clock cycle.

Our monitors will keep a copy of the previous `start` and `done` signals and use them to grab data.

A cocotb testbench

The rest of this book transforms a simplistic testbench into a modular, maintainable UVM testbench. Each chapter describes a testbench version that gets closer to the ultimate goal. This chapter describes the first version.

This basic testbench is a simple, easy-to-understand loop that would be difficult to maintain or enhance. This chapter intersperses the testbench code with commentary. You can see the entire testbench in the `23_Basic_testbench_1.0` directory in the `python4uvm_examples` repository.

Importing modules

In figure 3, we import the modules we need for the testbench. The first three imports are simple.

Figure 3: Importing needed resources

```
import cocotb
from cocotb.triggers import FallingEdge
import random
from pathlib import Path
parent_path = Path("..") .resolve()
sys.path.insert(0, str(parent_path) )
from tinyalu_utils import Ops, alu_prediction, logger, get_int
```

- `import cocotb` —Provides basic **cocotb** functionality including the `@cocotb.test()` decorator.
- `from cocotb.triggers import FallingEdge` —We `await` the `FallingEdge` coroutine to synchronize ourselves with the falling edge of the clock.
- `import random` —This module allows us to generate random operands.
- `from tinyalu_utils import Ops, alu_prediction, logger, get_int` —Import resources we'll need for the testbench

The Ops enumeration

An *enumeration* is a list of named values used throughout a program. For example, if you're writing a dungeon game, you may have an enumeration of legal weapons: SWORD, SPEAR, BOW.

Python implements enumerations using the `enum` module. The `enum` module defines base classes that we extend to create enumerations in our programs.

In our example, the TinyALU operations are an enumeration: ADD, AND, XOR, and MUL. The `tinyalu_utils` module uses an `enum.IntEnum` to define the `Ops` enumeration ¹³. This class not only names the operations but also maps the names to the correct opcode integer.

Figure 4: The operation enumeration

```
import enum
class Ops(enum.IntEnum) :
    """Legal ops for the TinyALU"""
    ADD = 1
    AND = 2
    XOR = 3
    MUL = 4
```

Next we'll use the `Ops` class to predict the TinyALU's behavior.

The alu_prediction() function

Constrained-random testbenches create random stimuli and then predict the result expected from the DUT. Then they compare the predicted result to the actual result and flag errors. The `alu_prediction()` function in figure 3 creates the predicted result.

The function takes `A`, `B`, and `op` as inputs. First, it asserts that the `op` is an `Ops` object. Then it uses the `op` to pick the correct operation and perform it.

Figure 5: The prediction function for the scoreboard

```
def alu_prediction(A, B, op) :  
    """Python model of the TinyALU"""  
    assert isinstance(op, Ops) , "The tinyalu op must be of type Ops"  
    if op == Ops.ADD:  
        result = A + B  
    elif op == Ops.AND:  
        result = A & B  
    elif op == Ops.XOR:  
        result = A ^ B  
    elif op == Ops.MUL:  
        result = A * B  
    return result
```

Setting up the cocotb TinyALU test

A **cocotb** test starts with the `@cocotb.test()` decorator. In figure 5, it defines a coroutine named `alu_test()`.

The `alu_test` coroutine first sets the `passed` variable to `True`. Then, we create a set named `cvg` that we'll use to check that we've tested all the operations defined in the `ops` enumeration. Finally, we reset the DUT by waiting for the falling edge of the clock and lowering the `reset_n` and `start` signals. After a clock edge, we raise the `reset_n` signal, and we're ready to run.

Figure 6: The start of the TinyALU. Reset the DUT

```
@cocotb.test(expect_error=AssertionError)  
async def alu_test(dut) :  
    passed = True  
    cvg = set() # functional coverage  
    await FallingEdge(dut.clk)  
    dut.reset_n.value = 0  
    dut.start.value = 0  
    await FallingEdge(dut.clk)  
    dut.reset_n.value = 1
```

Now that we've reset the DUT, we can start sending commands.

Sending commands

In figure 7, the testbench creates one of each of the commands defined in `ops`. It sends them into the TinyALU by looping on the negative edge of the `dut.clk` signal, checking the status of the `start` and `done` signals to figure out whether it should send a command or look for a result.

We set the `cmd_count` to 1 as we are simulating the first command. The `op_list` is a list of the operations defined in `ops`, and `num_ops` is the length of that list. We'll send one command per operation.

After that, we run in a `while` loop as long as `cmd_count` is less than the number of commands. We await the falling edge of the clock, then get the `start` and `done` values as integers.

Figure 7: Creating one transaction for each operation

```
cmd_count = 1
op_list = list(ops)
num_ops = len(op_list)
while cmd_count <= num_ops:
    await FallingEdge(dut.clk)
    st = get_int(dut.start)
    dn = get_int(dut.done)
```

We now have the values of the `start` and `done` signals at the falling edge of the clock. We're ready to check the state of the `start` and `done` signals and send a command.

Sending a command and waiting for it to complete

We can only send a command if `start` and `done` signals are `0`. So in figure 8, we first randomize the `aa` and `bb` variables using `random.randint(0,255)`, which gives us a byte of information. Then we set `op` by popping an operation off the `op_list` and adding it to the `cvg` set to record our tested operations. Finally, we put `aa`, `bb`, and `op` on the TinyALU buses and raise the `start` signal.

Figure 8: Creating a TinyALU command

```
if st == 0 and dn == 0:
    aa = random.randint(0, 255)
    bb = random.randint(0, 255)
    op = op_list.pop(0)
    cvg.add(op)
    dut.A.value = aa
    dut.B.value = bb
    dut.op.value = op
    dut.start.value = 1
```

Now, the loop goes back to the top and waits for the next negative edge of the clock. As you can see in the timing diagram in figure 2, the operation is complete once the `start` signal and the `done` signal are both `1` on the same negative clock edge, so we monitor the state of `start` and `dn` in the following `if` statements.

The first `if` statement (figure 9) checks for illegal DUT behavior. The timing diagram shows that `done` should never go high if `start` is low.

Figure 9: Asserting that a failure state never happens

```
if st == 0 and dn == 1:
    raise AssertionError("DUT Error: done set to 1 without start")
```

In figure 10, we check to see if we are in the middle of a multi-clock operation. If the `start` signal is high, but the `done` signal is not, we go back to the top of the loop and wait for the next negative edge of the clock.

```
# Figure 10: If we are in an operation, continue
```

```
if st == 1 and dn == 0:
    continue
```

Checking the result

If the `start` and `done` signals are high (figure 11), the TinyALU has completed the operation, so the `result` bus contains a valid value. First, we set `start` to 0 as the operation is complete. Then we increment the `cmd_count` to tell the testbench that we've completed a command. Finally, we read the result from `dut.result`.

```
# Figure 11: The operation is complete
```

```
if st == 1 and dn == 1:
    dut.start.value = 0
    cmd_count += 1
    result = get_int(dut.result)
```

In figure 12, we check that we got the correct results. We pass `aa`, `bb`, and `op` to the `alu_prediction()` function and compare the predicted result, `pr`, to `result`. We log an information message if we pass and an error message if we fail. Notice that we use the `op.name` operation value to print the operation's name. Also, if we fail, we set `passed` to `False`.

```
# Figure 12: Checking results against the prediction
```

```
pr = alu_prediction(aa, bb, op)
if result == pr:
    logger.info(
        f"PASSED: {aa:2x} {op.name} {bb:2x} = {result:04x}")
else:
    logger.error(
        f"FAILED: {aa:2x} {op.name} {bb:2x} ="
        f" {result:04x} - predicted {pr:04x}")
passed = False
```

Finishing the test

The loop in figures 7-12 runs until we've simulated all the operations. Then, as a belt-and-suspenders precaution, we check that we have tested all the operations by creating a set from the `ops` enumeration and subtracting the `cvg` set from it. If we've simulated all the operations, the resulting set will be empty.

If we missed any operations, we log an error message and set `passed` to false. Otherwise, we log a happy informational message.

```
# Figure 13: Checking functional coverage using a set
```

```
if len(set(ops) - cvg) > 0:
    logger.error(f"Functional coverage error. Missed: {set(ops) - cvg}")
    passed = False
else:
    logger.info("Covered all operations")
```

We end the test by asserting that `passed` is `True`. This tells **cocotb** whether we passed the test.

```
# Figure 14: This assertion relays pass/fail to cocotb
```

```
assert passed
```

cocotb outputs the test result.

```
# Figure 15: A successful test
```

```
40000.00ns INFO PASSED: e5 ADD ba = 019f
60000.00ns INFO PASSED: a3 AND 3e = 0022
80000.00ns INFO PASSED: a6 XOR 18 = 00be
130000.00ns INFO PASSED: 35 MUL ef = 317b
130000.00ns INFO Covered all operations
```

Summary

This chapter used **cocotb** to create a simple TinyALU test. The test created four commands with randomized operands and ran them through the testbench.

This testbench fit entirely in one loop and was easy to follow. But it would be hard to maintain or expand because different testbench functions are jammed together in one big `while` loop. A testbench like this works for the TinyALU because it's tiny. But it won't work for large or complex devices. Large testbenches need to be modular so that a team of engineers can work on them together and so that each operation is as simple as possible.

We're going to break the TinyALU into modular pieces. The first step is to separate the signal-level operations that control the DUT from the testbench operations that create stimulus and check results.

We do that with a bus functional model.

TinyAluBfm

In the *Basic testbench: 1.0* chapter, we created a simple constrained-random testbench for the TinyALU and verified the RTL for the TinyALU. However, that version of the testbench has a problem. It mixes a variety of functionality into one `while` loop that does the following:

- Communicates with the TinyALU at the signal level
- Generates stimulus
- Checks results
- Stores functional coverage information
- Reports test results

Mixing these behaviors makes the testbench tough to maintain or extend. For example, say we want to write a different TinyALU test. Do we copy all this code and change only the differences? Copying and changing code will lead only to frustration and tears, as every time we fix a bug or add functionality, we have to do it in multiple files.

A maintainable testbench puts behaviors into different classes and leverages those classes for all the tests. That way, we can extend or modify the testbench by changing only one class, and the behavior propagates to all the tests. Modularity is essential when we create testbenches for complicated designs.

The rest of the book will break our testbench down into smaller, simpler pieces, and in the end, we'll have a modular, scalable testbench. But, first, we'll move our communication with the DUT into a bus functional model class.

The TinyALU BFM coroutines

A bus functional model (BFM) is a class that sends data using coroutine tasks. A BFM class contains coroutines that manipulate signals to talk to the DUT. For example, the TinyALU BFM has three such coroutines. One sends commands to the TinyALU, a second monitors commands entering the TinyALU, and the third monitors the results.

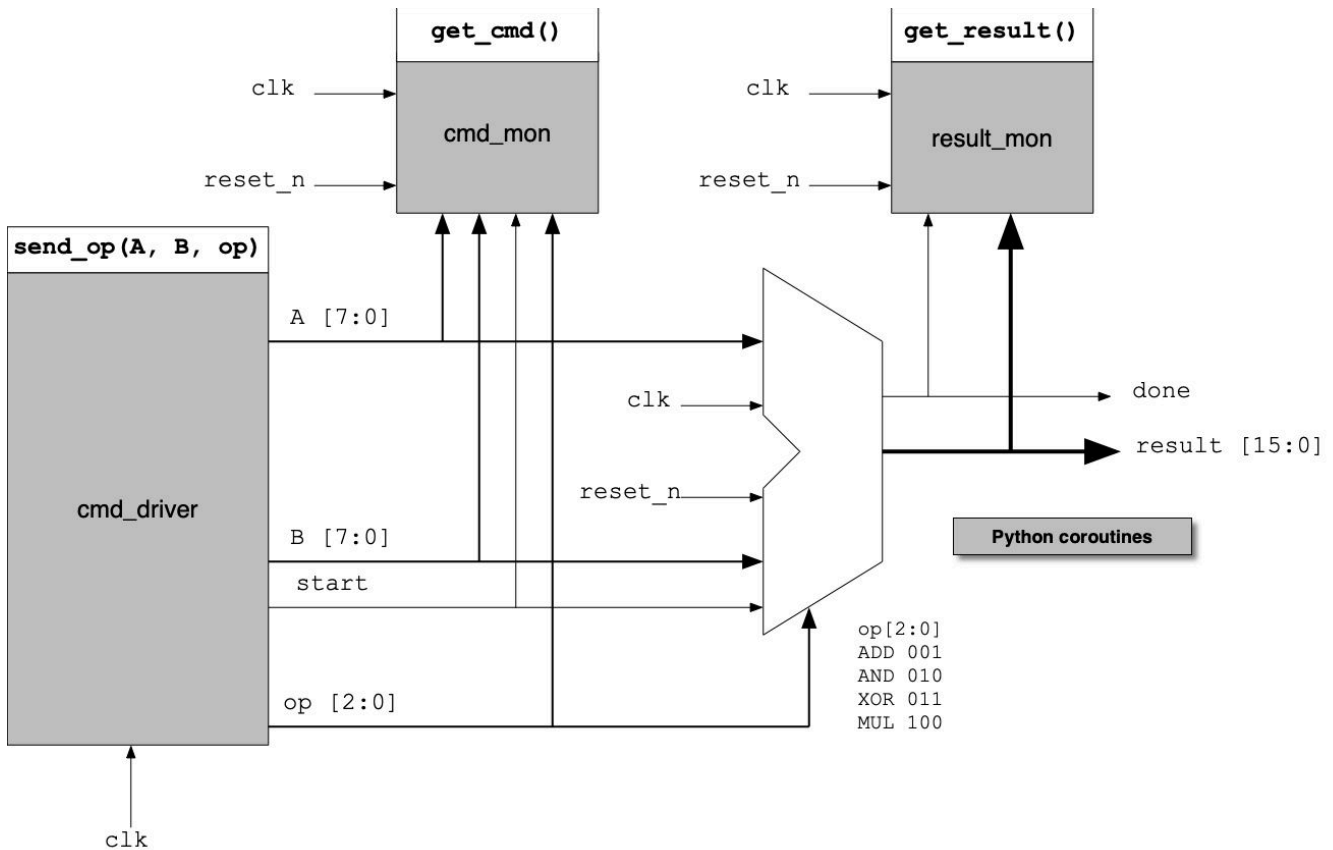


Figure 1: TinyAluBfm attached to DUT

There are three coroutines:

- **cmd_driver** — This coroutine checks the state of the **start** and **done** signals on each negative edge of the clock. If both are **0**, the coroutine check the command queue for a command, and puts data on **A**, **B**, and **op** if there is a command in the queue.
- **cmd_mon** — This coroutine watches for **start** transitioning from **0** to **1** over two clocks. When this happens, the coroutine creates a tuple with the values from **A**, **B**, and **op** buses and places it into a command monitor queue.
- **result_mon** — This courting watches for the **done** signal transitioning from **0** to **1** over two clocks. When this happens it puts the value on **result** bus into the result monitor queue.

In the following sections, we'll see how to write the coroutines.

The tinyalu_utils module

All the testbench versions in the book will now use the **TinyAluBfm** class. So, we'll store this code in **tinyalu_utils.py** and import it into the testbenches.

Living on the clock edge

Digital systems wait for the edge of a clock and then transfer data based on the state of signals. The TinyALU waits for the positive edge of the clock, so our BFM coroutines will wait for the falling edge. All of our coroutines contain a **while** loop that looks like this.

```
# Figure 2: Awaiting the falling edge of the clock
```

```
while True:
    await FallingEdge(dut.clk)
    <check signals and do the work>
```

The TinyAluBfm singleton

The testbench communicates with the TinyALU using the `TinyAluBfm` object. Since there is only one TinyALU, we have only one BFM. Having a single BFM suggests that `TinyAluBfm` should be a singleton class, and it is. The `class` statement demonstrates how to declare a singleton using `pyuvvm`.

The code `bfm = TinyAluBfm()` delivers a handle to the same object regardless of where the testbench calls it. The following sections document `TinyAluBfm`.

Initializing the TinyAluBfm object

The first time a user calls `TinyAluBfm()`, we create a new BFM object and initialize it. Here is the initialization code. First, it gets a handle to the DUT. Then, it creates three queues.

```
# Figure 3: Initializing the TinyAluBfm singleton
```

```
class TinyAluBfm(metaclass=pyuvvm.Singleton) :
    def __init__(self) :
        self.dut = cocotb.top
        self.cmd_driver_queue = Queue(maxsize=1)
        self.cmd_mon_queue = Queue(maxsize=0)
        self.result_mon_queue = Queue(maxsize=0)
```

The initialization does the following:

- `self.dut = cocotb.top` —The `cocotb.top` variable contains a handle to the DUT, the same handle the `cocotb.test()` coroutine got as its `dut` argument. We will ignore that `dut` argument and use `cocotb.top` instead.
- `self.driver_queue` —The `maxsize=1` argument means this queue only has room for one operation, so the `send_op()` coroutine will block if the queue is full.
- `cmd_mon_queue` —The `maxsize=0` argument means this queue is infinitely large¹⁴. The command monitor BFM uses `put_nowait()` to put `(A, B, op)` command tuples into this queue. The `get_cmd()` coroutine reads data from the queue using `get()` and blocks if the queue is empty.
- `result_mon_queue` —The `maxsize=0` argument means this queue is infinitely large. The result monitor BFM uses `put_nowait()` to put the value from the `result` bus into the queue. The `get_result()` coroutine uses `get()` to read results from the queue and blocks if the queue is empty.

The reset() coroutine

We always reset our DUT before we run tests. In figure 4, the `reset()` coroutine sets the TinyALU inputs to zero and sets `reset_n` to 0. Then it waits for one clock and sets `reset_n` back to 1.

Figure 4: Centralizing the reset function

```
async def reset(self) :
    await FallingEdge(self.dut.clk)
    self.dut.reset_n.value = 0
    self.dut.A.value = 0
    self.dut.B.value = 0
    self.dut.op.value = 0
    await FallingEdge(self.dut.clk)
    self.dut.reset_n.value = 1
    await FallingEdge(self.dut.clk)
```

The communication coroutines

The `TinyALUBfm` has three coroutines that loop on the negative edge of the clock and interact with the TinyALU. Here is the `TinyALU` timing diagram to use for reference as we examine the code.

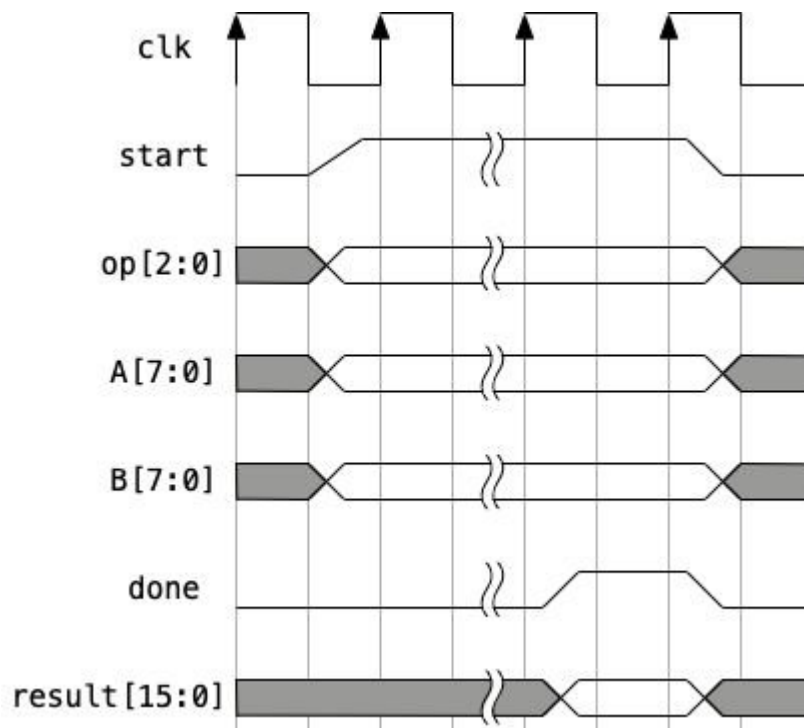


Figure 5: TinyALU Timing Diagram

result_mon()

The `result_mon` monitors the `done` signal and the `result` bus. The timing diagram in figure 5 shows that `result` is valid when `done` goes from 0 to 1 between clock edges. So we read `result` when `prev_done` is 0 and `done` is 1.

Figure 6: Monitoring the result bus

```
async def result_mon(self) :
    prev_done = 0
    while True:
        await FallingEdge(self.dut.clk)
        done = get_int(self.dut.done)
        if prev_done == 0 and done == 1:
            result = get_int(self.dut.result)
            self.result_mon_queue.put_nowait(result)
        prev_done = done
```

We create an integer from `result` and put the integer into the `result_mon_queue` using `put_nowait()`.

cmd_mon()

`cmd_mon()` reads the `A`, `B`, and `op` buses when the `start` signal goes from 0 to 1, converting the values to integers. It creates a tuple from the three integers and puts it into the `cmd_mon_queue` with `put_nowait()`.

Figure 7: Monitoring the command signals

```
async def cmd_mon(self) :
    prev_start = 0
    while True:
        await FallingEdge(self.dut.clk)
        start = get_int(self.dut.start)
        if start == 1 and prev_start == 0:
            cmd_tuple = ( get_int(self.dut.A),
                          get_int(self.dut.B),
                          get_int(self.dut.op))
            self.cmd_mon_queue.put_nowait(cmd_tuple)
        prev_start = start
```

cmd_driver()

The `cmd_driver()` coroutine loops on the falling edge of the clock and checks `start` and `done`. When `start` and `done` are zero we try to get a command out of the `driver_queue` using `get_nowait()`. If the queue is empty, `get_nowait()` raises the `QueueEmpty` exception.

In figure 8, we set all `start` and all the input buses to 0. Then, we loop and wait for the negative edge of the clock. At the clock edge, we read `dut.start` and `dut.done`.

Figure 8: Driving commands on the falling edge of clk

```
async def cmd_driver(self) :
    self.dut.start.value = 0
    self.dut.A.value = 0
    self.dut.B.value = 0
    self.dut.op.value = 0
    while True:
        await FallingEdge(self.dut.clk)
        st = get_int(self.dut.start)
        dn = get_int(self.dut.done)
```

Now, we check `st` and `dn`. If they are both `0` we try to get a command from `self.driver_queue()` using `get_nowait()`. If we get a command, we put it on the buses and set `start` to `1`. If `get_nowait()` raises a `QueueEmpty` exception we go back to the top of the `while` loop.

Figure 9: Driving commands to the TinyALU when
start and done are 0

```
if start == 0 and done == 0:
    try:
        (aa, bb, op) = self.driver_queue.get_nowait()
        self.dut.A <= aa
        self.dut.B <= bb
        self.dut.op <= op
        self.dut.start <= 1
    except QueueEmpty:
        continue
```

If `st == 1`, we check `dn` to see if the operation has finished. If it has, we set `start` to `0`.

Figure 10: If start is 1 check done

```
elif st == 1:
    if dn == 1:
        self.dut.start.value = 0
```

Notice that the `driver` coroutine doesn't worry about reading the result. Instead, it only drives commands to the TinyALU and relies upon the `result_mon()` coroutine to get the result. The ability to ignore other testbench elements is an advantage of modularity.

Launching the coroutines using start_soon()

Every test will want to launch the three BFM's, so the `TinyAluBfm` defines a `start_bfms()` function to do this.

```
# Figure 11: Start the BFM coroutines
```

```
def start_tasks(self) :  
    cocotb.start_soon(self.cmd_driver())  
    cocotb.start_soon(self.cmd_mon())  
    cocotb.start_soon(self.result_mon())
```

Notice that `start_soon()` is a function, not a coroutine, so `start_tasks()` is also a function. Neither consumes simulation time when it runs.

Interacting with the bfm loops

The `TinyAluBfm` contains `get_cmd()`, `get_result()`, and `send_op()`. These coroutines read data from the TinyALU and send commands to it.

get_cmd()

This coroutine `awaits` until there is a command in the `cmd_mon_queue` and then returns it.

```
# Figure 12: The get_cmd() coroutine returns the next command
```

```
async def get_cmd(self) :  
    cmd = await self.cmd_mon_queue.get()  
    return cmd
```

get_result()

This coroutine `awaits` until a result is in the `result_mon_queue` and then returns it.

```
# Figure 13: The get_result() coroutine returns the next result
```

```
async def get_result(self) :  
    result = await self.result_mon_queue.get()  
    return result
```

send_op()

We give the `send_op()` coroutine the values for `A`, `B`, and `op`, and it combines them into a command and puts the command in the `cmd_driver_queue`. If the `cmd_driver_queue` contains a previous command, then `put()` blocks until the queue is free.

```
# Figure 14: send_op puts the command into the command Queue
```

```
async def send_op(self, aa, bb, op) :  
    command_tuple = (aa, bb, op)  
    await self.cmd_driver_queue.put(command_tuple)
```

We're ready to write a `cocotb` test that uses `TinyAluBfm`.

The cocotb test

This version of the testbench generates stimulus and checks results, but it does not interact with the TinyALU signals. Instead, it instantiates the `TinyAluBfm()` singleton and uses the object's coroutines to communicate with the TinyALU and the simulator.

Since we instantiate the `TinyAluBfm` we ignore the argument to `test_alu`. We'll never use that argument as all future testbench versions will use `TinyAluBfm`.

Our test gets a handle to `TinyAluBfm`, resets the DUT, and launches the tasks that drive and monitor the TinyALU signals. Then it creates a functional coverage set named `cvg`.

```
# Figure 15: Starting a test by resetting the DUT
# and starting the BFM tasks
```

```
@cocotb.test()
async def test_alu(_) :
    """Test all TinyALU Operations"""
    passed = True
    bfm = TinyAluBfm()
    await bfm.reset()
    bfm.start_tasks()
    cvg = set()
```

Sending commands

We have a more straightforward `while` loop because we're not contending with a clock edge. Instead, we can create a list of operations and loop through them, using `await bfm.send_op(aa, bb, op)` to send the command to the TinyALU.

```
# Figure 16: Creating a command and sending it
```

```
ops = list(ops)
for op in ops:
    aa = random.randint(0, 255)
    bb = random.randint(0, 255)
    await bfm.send_op(aa, bb, op)
```

Monitoring the commands

Now that we've sent the command to the TinyALU, we will read the command back from the monitor. The reader may wonder why we're reading the command back from the monitor when we have just created and sent the same command. We do this for two reasons:

1. We can catch errors in the driver task by independently reading the inputs from the TinyALU. Such an error is unlikely in a simple BFM such as the TinyALU driver task but is more valuable when working with complicated interfaces such as USB.
2. We may want to use the BFM in a future testbench where the TinyALU is part of a higher-level design. In that case, we won't create the commands but will still want to monitor them.

First, we get the command tuple back from the BFM by awaiting `bfm.get_cmd()`. Then we pull the operation out of the tuple and add it to the `cvg` set using the `seen_op` variable.

```
# Figure 17: wait to get the command from the DUT
# and store it in the coverage set

seen_cmd = await bfm.get_cmd()
seen_op = Ops(seen_cmd[2] )
cvg.add(seen_op)
```

Now that we have the command we await the result from the BFM and create a predicted result (`pr`) using `alu_prediction`.

```
# Figure 18: wait for the result, then create a prediction

result = await bfm.get_result()
pr = alu_prediction(aa, bb, op)
```

Now compare the predicted result to the actual result:

```
# Figure 19: Check the result against the predicted result

if result == pr:
    logger.info(f"PASSED: {aa:02x} {op.name} {bb:02x} = {result:04x}")
else:
    logger.error( f"FAILED: {aa:02x} {op.name} {bb:02x} = "
                 f"{result:04x} - predicted {pr:04x}")

passed = False
```

We're done with our test loop, so we check that we've covered all the operations and finish the `cocotb` test by asserting that `passed` is still `True`.

```
# Figure 20: Assert that we passed to pass to cocotb

assert passed
```

Success!

```
# Figure 21: Another successful test
50000.00ns INFO PASSED: ce ADD b7 = 0185
70000.00ns INFO PASSED: fa AND f5 = 00f0
90000.00ns INFO PASSED: 9a XOR 74 = 00ee
140000.00ns INFO PASSED: db MUL 7d = 6aef
140000.00ns INFO Covered all operations
```

Summary

In this chapter, we created the `TinyAluBfm` class, which communicates with the TinyALU at the signal level. We created a simple cocotb testbench that uses the BFM's `send_op()`, `get_cmd()`, and `get_result()` coroutines to send stimulus to the TinyALU and check the result.

Encapsulating DUT communication in `TinyAluBfm` is a good start toward breaking the testbench into pieces. However, we still have operations such as stimulus generation and result checking mixed in one loop.

In the next chapter, we'll take the next step to modularize the testbench by refactoring it to use classes.

Class-based testbench: 2.0

This chapter demonstrates how to write a **cocotb** testbench using classes and objects. Classes allow us to break testbench functionality into small, easier-to-maintain pieces.

The testbench in this chapter shows how we use classes to avoid copying code and making small changes.

The class structure

This Universal Modeling Language (UML) diagram shows how we'll use classes and inheritance to avoid copying code and create a modular testbench:

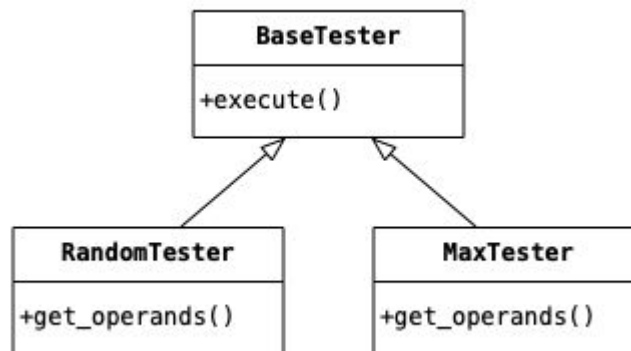


Figure 1: Tester class structure

Figure 1 says that we'll create a class named `BaseTester` that contains an `execute()` method. The `RandomTester` and `MaxTester` classes extend `BaseTester`, inheriting the `execute()` method. They add a `get_operands()` method that either returns eight random bits, or a `xFF` to use on the `A` and `B` buses.

The BaseTester class

The `BaseTester` defines the `execute()` coroutine that gets operands, creates commands, and sends the commands to the TinyALU.

The `execute()` coroutine gets a handle to the BFM and creates a list of the operations. Then it loops through the operations and uses `self.get_operands()` to get values for `aa` and `bb`. Finally, it uses `self.bfm.send_op()` to send the command to the TinyALU.

The code in figure 2 implements the `BaseTester`. Surprisingly, the `get_operands()` method seems to be missing. As the diagram shows, it gets defined in the child classes

Figure 2: Common behavior across all tests

```
class BaseTester() :

    async def execute(self) :
        self.bfm = TinyAluBfm()
        ops = list(ops)
        for op in ops:
            aa, bb = self.get_operands()
            await self.bfm.send_op(aa, bb, op)
        # send two dummy operations to allow
```



```
# last real operation to complete
await self.bfm.send_op(0, 0, 1)
await self.bfm.send_op(0, 0, 1)
```

`BaseTester` is an abstract class that depends upon its descendent classes to define the `get_operands()` method. Python enables this design style as part of its “ask forgiveness not permission” dynamic typing philosophy.

We call classes that serve as the parent to many related classes base classes, hence the name *BaseTester*.

The RandomTester

We’ve said that we want to create many small objects that do one thing and then put them together. The `RandomTester` is a good example of this. All it does is extend `BaseTester` and provide a `get_operands()` function. This makes `RandomTester` a *concrete class* that inherits from an abstract class.

```
# Figure 3: RandomTester overrides get_operands()
class RandomTester(BaseTester) :
    def get_operands(self) :
        return random.randint(0, 255) , random.randint(0, 255)
```

The `RandomTester` inherits `execute()` from the `BaseTester`, so we don’t need to copy that code here.

The MaxTester

The `MaxTester` extends `BaseTester` and provides `0xFF` as the operands for `A` and `B`.

```
# Figure 4: MaxTester overrides get_operands()
class MaxTester(BaseTester) :
    def get_operands(self) :
        return 0xFF, 0xFF
```

The Scoreboard class

This book defines the word “scoreboard” as a class that gathers data from the DUT, predicts results, and compares actual results to predicted results. Our `Scoreboard` class works like this:

1. It starts two tasks that gather the commands and results from the DUT and store them in lists. The commands and results match up because the tasks store them in the same order.
2. It defines a `check_results()` function that loops through the lists, predicting results from the commands and comparing them to the results. The function also fills the coverage set to ensure the test reaches all the commands.

One might ask why we need functional coverage if the `Tester` loops through all the operations. We check coverage because the `Scoreboard` needs to work with any tester, and some testers might not invoke all operations.

Initialize the scoreboard

First, we store a handle to the BFM and initialize the lists that will store commands and results. Then, we create a `cvg` set to use for functional coverage.

Figure 5: Initializing the Scoreboard

```
class Scoreboard() :
    def __init__(self) :
        self.bfm = TinyAluBfm()
        self.cmds = []
        self.results = []
        self.cvg = set()
```

Now we're ready to gather data.

Define the data-gathering tasks

The scoreboard will start tasks that gather commands and results from the DUT and store them in lists.

`get_cmd()` loops forever and awaits `self.bfm.get_cmd()`. Once it gets a command, it appends it to `self.cmds`.

Figure 6: The Scoreboard gets a command

```
async def get_cmd(self) :
    while True:
        cmd = await self.bfm.get_cmd()
        self.cmds.append(cmd)
```

The `get_result()` coroutine loops forever and awaits `self.bfm.get_result()`. When it gets results it appends to `self.results`.

Figure 7: The Scoreboard gets a result

```
async def get_result(self) :
    while True:
        result = await self.bfm.get_result()
        self.results.append(result)
```

Now we're ready to start these coroutines as tasks.

The Scoreboard's `start_tasks()` function

`start_tasks()` starts the `get_cmd()` and `get_result()` coroutines as tasks.

Figure 8: The scoreboard launches data-gathering tasks

```
def start_tasks(self) :
    cocotb.start_soon(self.get_cmd())
    cocotb.start_soon(self.get_result())
```

The Scoreboard's check_results() function

The `check_results()` function loops through the TinyALU commands, predicts the result, and compares the predicted to the actual results. It also stores the operations in the `cvg` set and checks we've tested all the operations.

Figure 9: The check_results() phase

```
def check_results(self) :
    passed = True
    for cmd in self.cmds:
        aa, bb, op_int = cmd
        op = Ops(op_int)
        self.cvg.add(op)
        actual = self.results.pop(0)
        prediction = alu_prediction(aa, bb, op)
        if actual == prediction:
            logger.info(f"PASSED: {aa:02x} {op.name} {bb:02x} =
{actual:04x}")
        else:
            passed = False
            logger.error(
                f"FAILED: {aa:02x} {op.name} {bb:02x} =
                f" - predicted {prediction:04x}")
```

Here we check that we've seen all the operations.

Figure 10: The Scoreboard checks functional coverage

```
if len(set(Ops) - self.cvg) > 0:
    logger.error(
        f"Functional coverage error. Missed: {set(Ops) -self.cvg}")
    passed = False
else:
    logger.info("Covered all operations")
return passed
```

We return the `passed` variable to tell the testbench whether the test has passed.

The execute_test() coroutine

Running a test looks the same regardless of the stimulus, so we'll create a coroutine that works for any stimulus and takes the tester class as an argument.

First, we start up the testbench by resetting the DUT and starting the tasks in `bfm` and `scoreboard`.

Figure 11: The `execute_test` coroutine starts the tasks

```
async def execute_test(tester_class) :
    bfm = TinyAluBfm()
    scoreboard = Scoreboard()
    await bfm.reset()
    bfm.start_tasks()
    scoreboard.start_tasks()
```

Now we're ready to create stimulus and test the results. The `tester_class` argument contains either a `RandomTester` or a `MaxTester`. It doesn't matter which. We instantiate `tester_class` and await `tester.execute()`.

When we return from `tester.execute()` the `scoreboard` will have filled its lists. We call `scoreboard.check_results()` to check for errors and return the `passed` variable to the cocotb test.

Figure 12: Execute the tester

```
tester = tester_class()
await tester.execute()
passed = scoreboard.check_results()
return passed
```

The cocotb tests

There are two **cocotb** tests. One uses randomized operands, and the other uses `0xFF` as operands. These tests are tiny because we did all the work in the class definitions.

We pass the `execute_test()` coroutine a tester class. It returns a `bool` to the `passed` variable, and we assert that `passed` is `True`. If it is false, **cocotb** logs an error.

Figure 13: cocotb will launch the `execute_test` coroutine

```
@cocotb.test()
async def random_test(_) :
    """Random operands"""
    passed = await execute_test(RandomTester)
    assert passed

@cocotb.test()
async def max_test(_) :
    """Maximum operands"""
    passed = await execute_test(MaxTester)
    assert passed
```

Summary

This chapter created a TinyALU testbench using classes. We saw that we could share common code by creating a `BaseTester` that drove commands into the TinyALU. We extended `BaseTester` to create `RandomTester` and `MaxTester`.

Our `Scoreboard` object monitored the TinyALU output and checked that the operations worked correctly.

We saw that we could pass classes to an `execute_test()` coroutine that would initialize the testbench and run the test. Once again, we avoided copying code.

The **cocotb** tests passed `execute_test()` a `RandomTester` or `MaxTester` class and awaited the `passed` variable. Then it asserted that `passed` should be `True`.

Why UVM?

While the Universal Verification Methodology is the most successful verification methodology in the history of the world, it's not the first one. The eRM, VMM, AVM, and OVM came earlier, but the UVM came last and is the sole survivor thanks to an agreement between Cadence, Synopsys, and Mentor Graphics (now Siemens). Today, the UVM's evolution is in the hands of a committee of EDA vendors and end-users.

A verification methodology provides a standard set of answers to questions common to all testbench developers. Here are some of the questions the UVM answers.

How do we define tests?

We saw that **cocotb** defines tests using the `@cocotb.test()` decorator. **pyuvvm** defines tests by extending the `uvm_test` class and using the `@pyuvvm.test()` decorator.

How do we build testbenches?

We created an `execute_test()` coroutine in the class-based testbench that instantiated the `Tester` and the `Scoreboard` classes. That is one of many ways to build a testbench.

The UVM provides a common way to instantiate objects.

How do we reuse testbench components?

If the TinyALU became part of a larger design, perhaps a TinyMicrocontroller, you might want to reuse the TinyALU testbench as a monitoring device in the larger testbench. Reusing a testbench in a larger context is called vertical reuse and the UVM makes it easy to implement vertical reuse.

How do we create verification IP?

Verification IP is a testbench component that implements a protocol such as Ethernet or USB. We don't want to rewrite these for every testbench. Instead, we want to define them in a standard way that is easy to reuse.

The UVM delivers that standard.

How do multiple components monitor the DUT?

In the class-based testbench the `Scoreboard` object calls `get_cmd()` and `get_result()`. But, what if other testbench components wanted that data? They can't get it out of the BFM because the `Scoreboard` took it.

The UVM solves this problem by allowing components to send data to each other.

How do we create stimulus?

The `Tester` classes in our class-based testbench created the stimulus and sent it to the TinyALU. We avoided rewriting the `execute()` coroutine by creating a base class, but we had to rebuild the testbench for every form of stimulus.

The UVM separates stimulus generation from the testbench structure.

How do we share common data?

The `tinyalu_utils` module shares the `TinyAluBfm` by implementing it as a singleton. But large testbenches need to share data and have different data go to different components.

The UVM has a mechanism for sharing objects across the entire testbench and giving different data to different parts of the testbench.

How do we modify our testbench's structure in each test?

The class-based testbench demonstrated a design where we changed the testbench structure by replacing the `RandomTester` with the `MaxTester` in different tests. The UVM makes it easy to implement component swapping like this.

How do we log messages?

The `tinyalu_utils` module defined a logger object to log messages. But, what if we wanted to know where the messages were logged or wanted to allow logging in only one part of the testbench?

The UVM provides logging modules to all testbench components and a way to change logging levels in different parts of the testbench.

How do we pass data around the testbench?

Testbench 2.0 used a Python tuple— `(A, B, op)` —to pass data from the command monitor. The programmer had to remember that `op` was `cmd[2]`. It would be better to pass data around in user-defined classes with names for the data fields. The UVM standardizes how we do that.

Summary

One does not have to use the UVM to create testbenches with Python and **cocotb**, but one does have to answer all the questions in this chapter for testbenches of various sizes. However, if engineers answer these questions differently for their testbenches, they'll have trouble reusing components across teams and projects.

Using the UVM (through **pyuvvm**) automatically gives us testbenches that work the same way regardless of the device under test. Learning and using the UVM allows us to move beyond basic questions of structure and reuse and focus on the more complicated question of verifying our DUT.

uvm_test testbench: 3.0

We made testbench 2.0 modular by adding a `Tester` class and a `Scoreboard` class. We're now ready to use the UVM and import **pyuvm**.

The HelloWorldTest class

As is tradition, we will start writing UVM tests by creating the `HelloWorldTest` to examine the mechanics of creating and running a UVM test using **cocotb** and **pyuvm**. I'll show the code in figure 1 and explain it below.

Figure 1: The basic pyuvm use model in HelloWorldTest

```
import pyuvm
from pyuvm import *

@pyuvm.test()
class HelloWorldTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        self.logger.info("Hello, world.")
        self.drop_objection()
```

Here's the minimum you need to do to run a UVM test in **cocotb**:

- `import pyuvm` —Import `pyuvm` so we can use the `@pyuvm.test()` decorator to identify our test to **cocotb**.
- `from pyuvm import *` —Importing all the objects from the `pyuvm` module matches how we do imports in SystemVerilog UVM where we type `import uvm_pkg::*`.
- `@pyuvm.test()` —The `@pyuvm.test()` decorator tells **cocotb** that this **pyuvm** class is a **cocotb** test.
- `class HelloWorldTest(uvm_test)` —The `HelloWorldTest` class extends the `uvm_test` class. Python's tradition is to capitalize the first character of each word in a class name, but **pyuvm** uses the underscored class names from the UVM specification.
- `async def run_phase(self)` —We will discuss phasing in the next chapter, but suffice it to say this coroutine must be named `run_phase()` so that the UVM can find it and execute it. It is the equivalent of the `execute()` function in our class-based testbench.
- `self.raise_objection()` —This function tells the UVM that a test is running. The UVM will stay in the run phase (discussed next chapter) until the test drops the objection.
- `self.logger.info("Hello, world.")` —UVM components, including `uvm_test`, all have a `self.logger` object that sends messages to the log file using the Python `Logging` module. The *Logging* chapter discusses logging details.
- `self.drop_objection()` —This function tells the UVM that the test has finished and that the `run_phase()` can end.

This simple example demonstrates the basics of defining a UVM test. Now we'll run the test.

Here is the result when we run the test.


```
# Figure 2: Hello, world!
```

```
INFO testbench.py(17) [uvm_test_top] : Hello, world.
```

The `INFO: testbench.py(17) [uvm_test_top] : Hello, world.` line comes from **pyuvm** and looks like the reports in SystemVerilog UVM. The **pyuvm** logger puts the path to the component that logged the message between the square brackets.

The UVM instantiates the class with the `@pyuvm.test()` decorator and names it `uvm_test_top`. For example, `uvm_test_top` is the name of the `HelloWorldTest` object.

We've successfully defined and run our first **pyuvm** test.

Refactoring testbench 2.0 into the UVM

The `uvm_test` class comes from a long line of class extensions defined in **pyuvm**. Here is the complete UML diagram along with our extensions.

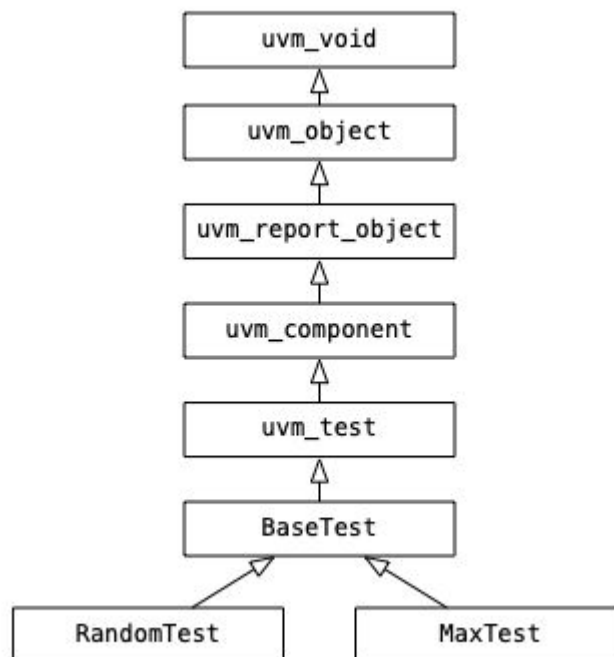


Figure 3: uvm_test UML diagram

The diagram shows that we will extend `uvm_test` to create `BaseTest` and then `RandomTest` and `MaxTest`. These mirror the behavior in testbench 2.0. Here is a short description of the UVM classes in the diagram.

- `uvm_void` —This is the ancestor of all UVM classes. The UVM factory (discussed in chapter *UVM Factory*) can create any class that extends `uvm_void`. Users don't extend `uvm_void` (hence the name.)
- `uvm_object` —Extend this class when you want to define a class in your testbench that leverages `uvm_object` features.
- `uvm_report_object` —Provides the `self.logger` object that we'll discuss in the *Logging* chapter.
- `uvm_component` —The backbone of the testbench's structure. It provides phase methods that we'll discuss in the *uvm_component* chapter. We'll use two of those methods in this testbench.

- `uvm_test` —Extends `uvm_component` and should be used when creating test classes.

The BaseTest class

Figure 3 showed the complete test class hierarchy all the way up to `uvm_void`. Figure 4 shows us only the tests. We see in figure 4 that while `BaseTest` provides the common `run_phase()`, `RandomTest` and `MaxTest` provide different `build_phase()` methods.

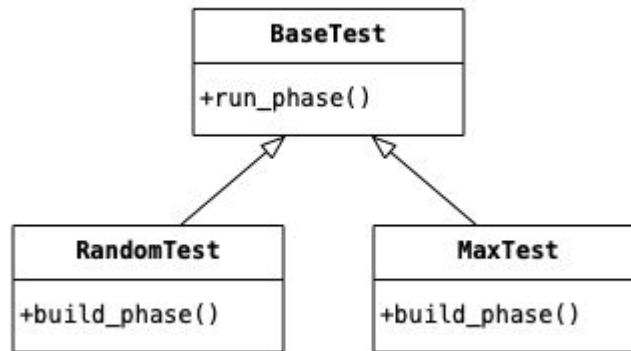


Figure 4: The test UML diagram

Testbench 2.0 contained an `execute_test()` coroutine that we launched in the **cocotb** test. We'll take that code and put it into the `BaseTest`, but we'll call the coroutine `run_phase()` instead of `execute_test()`.

Figure 5: BaseTest is an abstract class with no build_phase()

```

class BaseTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        bfm = TinyAluBfm()
        scoreboard = Scoreboard()
        await bfm.reset()
        bfm.start_tasks()
        scoreboard.start_tasks()
        await self.testster.execute()
        passed = scoreboard.check_results()
        assert passed
        self.drop_objection()
  
```

Figure 5 provides our first exposure to the phase methods in the UVM. In the *uvm_component* chapter, we'll see that components inherit a set of methods with names that end in `_phase` and that the UVM calls these methods in a predefined order.

The `run_phase()` is the only coroutine phase. In `BaseTest` (and its child classes) `run_phase()` starts up the `scoreboard` and awaits the `self.testster()`.

Which raises the question, where did `self.testster` come from?

The RandomTest and MaxTest classes

As we did in testbench 2.0, we'll use class extension to create `RandomTest` and `MaxTest`. These tests inherit the `run_phase()` from `BaseTest` and override the `build_phase()` method as we see in figure 5.

Figure 6: Extending BaseTest to create tests

```
@pyuvm.test()
class RandomTest(BaseTest) :
    """Run with random operations"""
    def build_phase(self) :
        self.tester = RandomTester()

@pyuvm.test()
class MaxTest(BaseTest) :
    """Run with random operations"""
    def build_phase(self) :
        self.tester = MaxTester()
```

(We defined the `RandomTester` and `MaxTester` classes in the *Class-based testbench 2.0* chapter.)

When we run one of these tests, the UVM calls `build_phase()`, which instantiates `self.tester`, and then launches `run_phase()`, which uses the tester to drive stimulus into the TinyALU.

Notice that `BaseTest` is an abstract class. It cannot run without being extended by a class that provides `build_phase()`. This is why `BaseTest` does not use the `@pyuvm.test()` decorator. It cannot be run directly.

The result is as we saw with testbench 2.0.

Figure 7: RandomTest passes

```
INFO PASSED: 97 ADD 2e = 00c5
INFO PASSED: 2e AND e1 = 0020
INFO PASSED: 8a XOR d2 = 0058
INFO PASSED: 72 MUL ec = 6918
INFO Covered all operations
```

Figure 8 sets all the operands to maximum values.

Figure 8 maxes all the operands

```
INFO PASSED: ff ADD ff = 01fe
INFO PASSED: ff AND ff = 00ff
INFO PASSED: ff XOR ff = 0000
INFO PASSED: ff MUL ff = fe01
INFO Covered all operations
```

Summary

This chapter featured our first foray into **pyuvm**. We defined two test classes that extended `uvm_test` and used the `build_phase()` and `run_phase()` to run the random operand and maximum operand tests.

We saw that we launch UVM tests by decorating them with `@pyuvm.test()`.

The next chapter discusses the `uvm_component` class and phases.

uvm_component

The `uvm_component` class is the backbone of the UVM. All the pieces of the testbench are UVM components or their extensions. For example `uvm_test` extends `uvm_component`.

When defining a `uvm_component`, you must override one or more phase functions defined in the base `uvm_component` class. The UVM calls these phase functions in the same order in all components.

If a given component does not override a phase function, then that component does nothing in that phase. Here are the phases in the order that the UVM calls them. Most testbenches must use the `build_phase()`, `connect_phase()`, and `run_phase()`.

1. build_phase(self)

This function instantiates objects in the component. Typically, the function creates child components who recognize the creator as their parent. Once `build_phase()` completes, the UVM calls the `build_phase` of the children. The UVM builds a hierarchy of components from the top down.

2. connect_phase(self)

We'll see later that components can send data to each other if connected. This phase connects the components created in the `build_phase()`. Unlike other phases, the `connect_phase()` runs first on the leaf-level components in the testbench and works its way up. This makes it a *bottom up* phase.

3. end_of_elaboration_phase(self)

Building and connecting the components is called *elaboration*. Once we've elaborated the testbench, we can execute operations that require the finished component hierarchy. For example, we may want to change the logging level of a child component and all its children. We do that in this function.

4. start_of_simulation_phase(self)

This function is similar to `end_of_elaboration_phase()` in that it runs after all the components have been instantiated and connected, but just before `pyuvvm` starts the `run_phase()` coroutines.

5. run_phase(self)

The `run_phase()` is the only coroutine in a UVM component. The UVM forks all the `run_phase()` coroutines in the hierarchy at the same simulation time.

Components keep the simulation in the run phase by calling `self.raise_objection()`. Any number of components can call `self.raise_objection()`. The simulation will stay in the run phase until every component that called `self.raise_objection()` calls `self.drop_objection()`.

6. extract_phase(self)

This function runs immediately after the simulation completes. You can use it to prepare simulation data to be checked.

7. check_phase(self)

Use this function to check that your simulation results are correct and complete.

8. report_phase(self)

This is where we report the results of the `extract_phase()` and `check_phase()` functions.

9. final_phase(self)

This method prepares for the end of the simulation.

Running the phases

`PhaseTest` prints a message in each phase to demonstrate the order of the phases. It is a `uvm_test` so, it inherits the phases from `uvm_component`.

```
# Figure 1: A uvm_test demonstrating the phase methods

@pyuvm.test()
class PhaseTest(uvm_test) : # uvm_test extends uvm_component
    def build_phase(self) :
        print("1 build_phase")

    def connect_phase(self) :
        print("2 connect_phase")

    def end_of_elaboration_phase(self) :
        print("3 end_of_elaboration_phase")

    def start_of_simulation_phase(self) :
        print("4 start_of_simulation_phase")

    async def run_phase(self) :
        self.raise_objection()
        print("5 run_phase")
        self.drop_objection()

    def extract_phase(self) :
        print("6 extract_phase")

    def check_phase(self) :
        print("7 check_phase")

    def report_phase(self) :
        print("8 report_phase")

    def final_phase(self) :
        print("9 final_phase")
```

Running the test gives us the output in figure 2 as the UVM calls the phases in order.

Figure 2: pyuvvm calls the phase methods in this order

```
1 build_phase
2 connect_phase
3 end_of_elaboration_phase
4 start_of_simulation_phase
5 run_phase
6 extract_phase
7 check_phase
8 report_phase
9 final_phase
```

Building the testbench hierarchy

The `@pyuvvm.test()` decorator instantiates our test using the name `uvm_test_top`. In doing this it creates the top of the testbench hierarchy. We create the rest of the hierarchy using the `build_phase()` and naming the components as we instantiate them. As an example, we'll create the hierarchy.

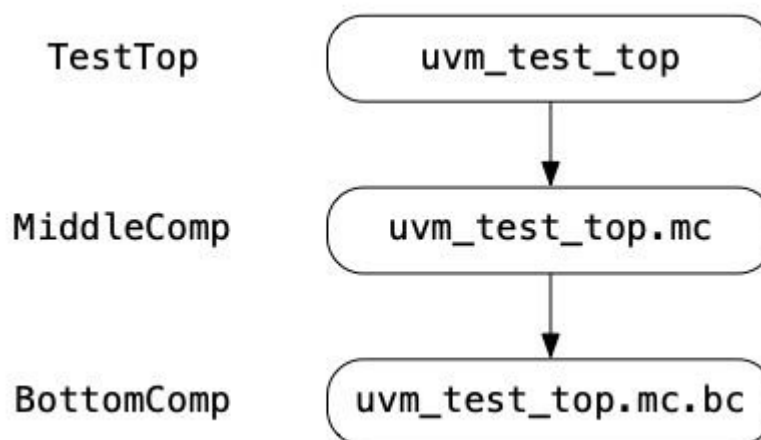


Figure 3: The testbench hierarchy

The `uvm_component` instantiation arguments.

The `uvm_component` class's `__init__()` method has two arguments:

- `name` —This string is the component's name. **pyuvvm** concatenates the name to the parent's name to create the hierarchy path.
- `parent` —This is the handle to the component's parent. Since the component instantiating the child component is the parent, this argument typically receives the `self` variable.

TestTop (`uvm_test_top`)

pyuvvm creates an instance of `TestTop` and names it `uvm_test_top`. Then it begins calling the phase methods. The first phase it calls is `build_phase()`. The `build_phase()` method in `uvm_test_top` instantiates a `MiddleComp` component and passes it its name (`"mc"`) and its parent. Since the object running `build_phase()` is the parent, we pass `self` as the parent.

```

# Figure 4: pyuvm instantiates TestTop as uvm_test_top
# The test is always named uvm_test_top
@pyuvm.test()
class TestTop(uvm_test) :
    def build_phase(self) :
        self.logger.info(f"{self.get_name()} } build_phase")
        self.mc = MiddleComp("mc", self)

    def final_phase(self) :
        self.logger.info("final phase")

```

When this `build_phase()` completes **pyuvm** calls `build_phase` in `self.mc`. The `final_phase()` will run at the end of the test. `TestTop` inherited `self.get_name()` from `uvm_object`. The method returns the component name, `uvm_test_top` in this case.

MiddleComp (uvm_test_top.mc)

The `MiddleComp` has a `build_phase` that instantiates a `BottomComp` by passing it its name ("bc") and its parent (`self`). We give the components the same name as their variable as a convention. They can be named anything.

```

# Figure 5: The middle component is instantiated by
# uvm_test_top as "mc" and instantiates "bc".

class MiddleComp(uvm_component) :
    def build_phase(self) :
        self.bc = BottomComp(name="bc", parent=self)

    def end_of_elaboration_phase(self) :
        self.logger.info(f"{self.get_name()} } end of elaboration phase")

```

When the `build_phase()` completes, **pyuvm** runs the `build_phase()` in `self.bc`. It will run the `end_of_elaboration_phase()` once the testbench is completely built.

BottomComp (uvm_test_top.mc.bc)

The `BottomComp`, being the bottom, has no `build_phase()`. It has only a `run_phase()` which **pyuvm** will call after the `end_of_elaboration_phase()`.

```

# Figure 6: The bottom component is instantiated by
# the middle component and is at "uvm_test_top.mc.bc"

class BottomComp(uvm_component) :
    async def run_phase(self) :
        self.raise_objection()
        self.logger.info(f"{self.get_name()} } run phase")
        self.drop_objection()

```


Running the simulation

We see the log messages printed in phase order when we run the simulation. Each log message contains the line in the code where the message was logged and the UVM hierarchy between the square brackets.

Figure 7: Note the testbench hierarchy in square brackets

```
INFO testbench.py(50) [uvm_test_top] : uvm_test_top build_phase
INFO testbench.py(65) [uvm_test_top.mc] : mc end of elaboration phase
INFO testbench.py(74) [uvm_test_top.mc.bc] : bc run phase
INFO testbench.py(54) [uvm_test_top] : final phase
```

Summary

In this chapter, we learned about the `uvm_component` class. We saw that UVM components inherit common phase methods and that the UVM calls these phases in a specific order.

We saw that we override phase methods to define a component's behavior and that the `build_phase()` builds a hierarchy of components.

Next, we'll learn about the UVM environment (`uvm_env`). This extension of `uvm_component` provides a place to instantiate components.

uvm_env testbench: 4.0

Testbench 4.0 converts the classes from testbench 3.0 into UVM components and instantiates those components inside a container class called an environment, created by extending `uvm_env`. We will do this in three steps:

1. Define UVM components for the testers and scoreboard.
2. Instantiate the testers and scoreboard in environments.
3. Instantiate the environments in tests.

The classes follow the same structure as in testbench 3.0 in that we'll have a base version of the tester and environment which we extend to create a random operand version and a max operand version.

Converting the testers to UVM components

The UML diagram for the testers shows that the `BaseTester` overrides `start_of_simulation_phase()` and `run_phase()`. The `RandomTester` and `MaxTester` inherit these and add `get_operands()`.

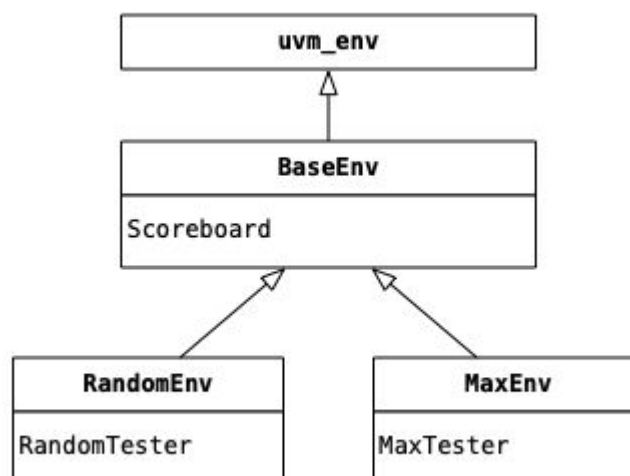


Figure 1: Environment UML diagram

BaseTester

Here is the code for `BaseTester`. The UVM phases tell us where to put the functionality we need. The `BaseTester` does two things:

1. Starts the `TinyAluBfm` tasks. We do this in the `start_of_simulation()` phase. (We could have done it in the `end_of_elaboration_phase()`. It is a programmer's choice.)
2. Loops through the operations, create commands, and send them to the TinyALU. We do this in the `run_phase()`, which pyuvvm starts as a task.

```
# Figure 2: The BaseTester implements phases common
# to all testers
```

```
class BaseTester(uvm_component) :
```

```

def start_of_simulation_phase(self) :
    TinyAluBfm() .start_tasks()

async def run_phase(self) :
    self.raise_objection()
    self.bfm = TinyAluBfm()
    ops = list(Ops)
    for op in ops:
        aa, bb = self.get_operands()
        await self.bfm.send_op(aa, bb, op)
    # send two dummy operations to allow
    # last real operation to complete
    await self.bfm.send_op(0, 0, 1)
    await self.bfm.send_op(0, 0, 1)
    self.drop_objection()

```

The `BaseTester` calls `self.raise_objection()` and `self.drop_objection()` to tell the UVM when it can end the run phase.

RandomTester and MaxTester

`RandomTester` and `MaxTester` are identical to their 2.0 versions.

```

# Figure 3: RandomTester and MaxTester don't need to change
# as BaseTester is a uvm_component

class RandomTester(BaseTester) :
    def get_operands(self) :
        return random.randint(0, 255) , random.randint(0, 255)

class MaxTester(BaseTester) :
    def get_operands(self) :
        return 0xFF, 0xFF

```

__init__() and UVM components

Notice that the `BaseTester` does not override the `__init__()` method. Instead, it simply uses the one it inherited from `uvm_component`. UVM components typically do not override `__init__()`. If we override `__init__()`, we're probably not using the UVM correctly.

The only reason to override `__init__()` is to create a class using arguments beyond `name` and `parent`. But, as we'll see, there are other ways to get data into a UVM component, and there are methods such as `end_of_elaboration_phase()` and `start_of_simulation_phase()` that initialize a component.

If you do override `__init__()` you must call `super().__init__(name, parent)`. Otherwise, pyuvvm will fail.

Scoreboard

The `Scoreboard` class uses UVM methods to set up its monitoring tasks and to check the results after the simulation. First, it defines the coroutines that gather data from the TinyALU.

Figure 4: Defining the Scoreboard data-gathering coroutines

```
class Scoreboard(uvm_component) :  
  
    async def get_cmd(self) :  
        while True:  
            cmd = await self.bfm.get_cmd()  
            self.cmds.append(cmd)  
  
    async def get_result(self) :  
        while True:  
            result = await self.bfm.get_result()  
            self.results.append(result)
```

The `Scoreboard` in testbench 3.0 used the `__init__()` method to initialize itself. This version uses `start_of_simulation_phase()` to initialize itself and to start its monitoring tasks.

Figure 5: Using the `start_of_simulation_phase()` to launch the monitoring tasks

```
def start_of_simulation_phase(self) :  
    self.bfm = TinyAluBfm()  
    self.cmds = []  
    self.results = []  
    self.cvg = set()  
    cocotb.start_soon(self.get_cmd())  
    cocotb.start_soon(self.get_result())
```

`Scoreboard` spends the simulation gathering data from the TinyALU and storing it in the `self.cmds` and `self.results` lists. It also fills the `cvg` set with all the operations it sees. Once the simulation has finished, it checks the results using the `check_phase()`.

Figure 6: Checking results after the `run_phase()` completes

```
def check_phase(self) :  
    passed = True  
    for cmd in self.cmds:  
        aa, bb, op_int = cmd  
        op = Ops(op_int)  
        self.cvg.add(op)  
        actual = self.results.pop(0)  
        prediction = alu_prediction(aa, bb, op)  
        if actual == prediction:  
            self.logger.info(  
                f"PASSED: {aa:02x} {op.name} {bb:02x} =  
{actual:04x}")
```

```

else:
    passed = False
    self.logger.error(
        f"FAILED: {aa:02x} {op.name} {bb:02x} = {actual:04x}"
        f" - predicted {prediction:04x}")
if len(set(Ops) - self.cvg) > 0:
    self.logger.error(
        f"Functional coverage error. Missed: {set(Ops) -self.cvg}")
    passed = False
else:
    self.logger.info("covered all operations")
assert passed

```

This 4.0 `check_phase()` is different than the 3.0 version in two ways:

1. Instead of using the `logger` variable to log messages, this version uses `self.logger`, an object inherited from `uvm_component`. The chapter Logging delves into `self.logger`.
2. Unlike the `Scoreboard` in testbench 3.0, this version uses `assert passed` to tell cocotb whether the test passed. If the `passed` is false, the `assert` statement raises an `AssertionError` exception. **cocotb** catches that exception and marks the test as having failed.

Using an environment

Environments allow us to instantiate a set of components that work together. For example, it would make little sense to have a tester without a scoreboard. So, we will instantiate our new components in an environment that keeps them together. First, we create a class structure that mirrors the testers.

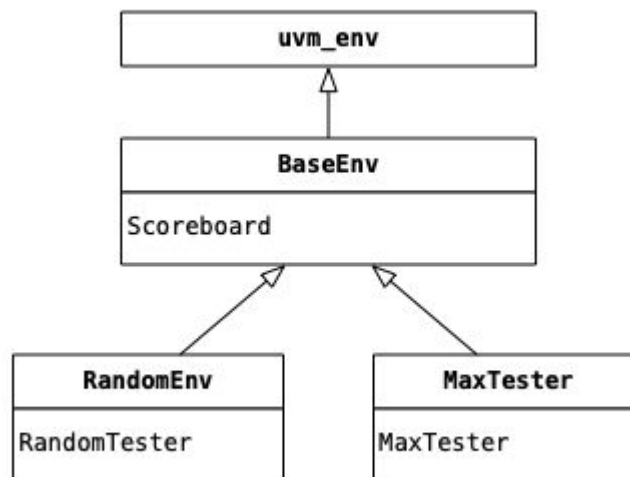


Figure 7: Environment class structure

In figure 8, we see that the `BaseEnv` instantiates the `Scoreboard` because all testers need a scoreboard. We extend `BaseEnv` to `RandomEnv` and `MaxEnv` and instantiate the correct tester. Here is the `BaseEnv`.

Figure 8: All environments need the scoreboard

```
class BaseEnv(uvm_env) :  
    """Instantiate the scoreboard"""  
  
    def build_phase(self) :  
        self.scoreboard = Scoreboard("scoreboard", self)
```

Now we'll extend `BaseEnv` and add the `RandomTester`. We must call `super().build_phase()` to get the scoreboard instantiated.

Figure 9: The `RandomEnv` instantiates the `RandomTester`
The `MaxEnv` instantiates the `MaxTester`

```
class RandomEnv(BaseEnv) :  
    """Generate random operands"""  
  
    def build_phase(self) :  
        super().build_phase()  
        self.testers = RandomTester("testers", self)  
  
class MaxEnv(BaseEnv) :  
    """Generate maximum operands"""  
    def build_phase(self) :  
        super().build_phase()  
        self.testers = MaxTester("testers", self)
```

Now our test classes simply instantiate the correct environment.

Creating RandomTest and MaxTest

The test classes for the testbench are straightforward. They do nothing but instantiate the correct environment.

```
# Figure 10: Instantiating the right environment in each test  
@pyuvm.test()  
class RandomTest(uvm_test) :  
    """Run with random operands"""  
    def build_phase(self) :  
        self.env = RandomEnv("env", self)  
  
@pyuvm.test()  
class MaxTest(uvm_test) :  
    """Run with max operands"""  
    def build_phase(self) :  
        self.env = MaxEnv("env", self)
```

This gives us the expected testbench output with `check_phase()` running after the simulation has been completed.

```
# Figure 11: The test running using environments
```

```
# Random test
```

```
INFO PASSED: 5b ADD 7b = 00d6
```

```
INFO PASSED: 3d AND 71 = 0031
```

```
INFO PASSED: 79 XOR b6 = 00cf
```

```
INFO PASSED: 88 MUL 12 = 0990
```

```
INFO Covered all operations
```

```
# Max operand test
```

```
INFO PASSED: ff ADD ff = 01fe
```

```
INFO PASSED: ff AND ff = 00ff
```

```
INFO PASSED: ff XOR ff = 0000
```

```
INFO PASSED: ff MUL ff = fe01
```

```
INFO Covered all operations
```

Summary

This chapter created testbench 4.0 by converting our testbench classes to UVM components and instantiating those components in a UVM environment. `RandomTest` and `MaxTest` had only to instantiate the correct environment.

We will take a break from upgrading the testbench to examine other features the UVM provides, such as logging and the configuration database.

Logging

Now that we've learned to create a UVM testbench hierarchy, we can discuss UVM features that take advantage of that hierarchy. The first of these is logging.

Large testbenches generate so much information that we need a simple way to filter and direct output. The SystemVerilog UVM defines a "reporting" system to address this issue, but **pyuvvm** does not implement this system. Instead, **pyuvvm** leverages the Python **logging** module and provides methods to control Python logging using the UVM testbench hierarchy.

Creating log messages

The UML diagram in figure 1 shows us that `uvm_component` extends `uvm_report_object`.

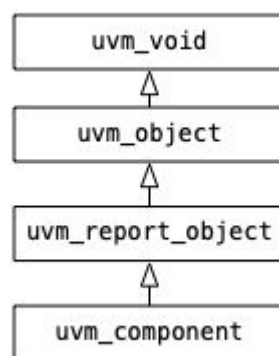


Figure 1: uvm_component UML

The `uvm_report_object` class creates an object named `self.logger` that we use to create logging messages.

In testbench 4.0, we called `self.logger.info()` to create information messages when tests passed and `self.logger.error()` to create error messages when tests failed. These are two of six logging levels defined by the **logging** module.

We create messages using logging methods with the same name as the logging levels. For example, `self.logger.error()` creates a message with an `ERROR` logging level.

Here is a component that logs messages of all levels.

Figure 2: Logging messages of all levels

```
class LogComp(uvm_component) :
    async def run_phase(self) :
        self.raise_objection()
        self.logger.debug("This is debug")
        self.logger.info("This is info")
        self.logger.warning("This is warning")
        self.logger.error("This is error")
        self.logger.critical("This is critical")
        self.drop_objection()

class LogTest(uvm_test) :
```



```
def build_phase(self) :
    self.comp = LogComp("comp", self)

--
INFO testbench.py(10) [uvm_test_top.comp] : This is info
WARNING testbench.py(11) [uvm_test_top.comp] : This is warning
ERROR testbench.py(12) [uvm_test_top.comp] : This is error
CRITICAL testbench.py(13) [uvm_test_top.comp] : This is critical
```

Logging Levels

The `Logging` module defines six logging levels, shown in figure 3. We filter log messages using logging levels. Each level has an associated numeric value, and each `self.Logger` object has a logging level that filters messages.

Logging Level	Numeric Value
CRITICAL	50
ERROR	40
WARNING	30
INFO	20
DEBUG	10
NOTSET	0

Figure 3: Logging levels

By default, the `self.Logger` logging level is `INFO`, which means that a log entry must have a logging level greater than or equal to 20 to get logged. The example in figure 2 called `self.Logger.debug()`, but we don't see the `DEBUG` message because the debug message has a logging level of 10.

Setting Logging Levels

Every UVM component comes with its own `self.logger` data member whose logging level is `logging.INFO`. You can change the logging level with two methods:

- `self.set_logging_level(level)` —This sets the logging level for the `self.logger` in the component.
- `self.set_logging_level_hier(level)` —This sets the logging level for the `self.logger` object in the component and all of the components below it in the UVM hierarchy.

We can only set logging to a hierarchy after the hierarchy has been built, so we call `self.set_logging_level_hier()` in the `end_of_elaboration_phase()` or `start_of_simulation_phase()` methods.

In figure 2, `LogComp` logged messages of all six levels, but the debug message did not print because the logging level was set to INFO.

The `DebugTest` in figure 4 extends `LogTest` but changes the logging levels in its `end_of_elaboration_phase()` method. Now the debug message prints.

```
# Figure 4: Setting the logging level in
# the end_of_elaboration_phase() method

@pyuvm.test()
class DebugTest(LogTest) :
    def end_of_elaboration_phase(self) :
        self.set_logging_level_hier(DEBUG)

---
DEBUG testbench.py(9) [uvm_test_top.comp] : This is debug
INFO testbench.py(10) [uvm_test_top.comp] : This is info
WARNING testbench.py(11) [uvm_test_top.comp] : This is warning
ERROR testbench.py(12) [uvm_test_top.comp] : This is error
CRITICAL testbench.py(13) [uvm_test_top.comp] : This is critical
```

Logging Handlers

Logging handlers tell the logger where to send the message. The `logging` module defines many handlers. We are going to examine two of them. You must import `logging` to create a handler.

logging.StreamHandler

By default, a `StreamHandler` writes messages to the screen using `sys.stderr`. `pyuvm` modifies this behavior. By default `self.logger` contains a `StreamHandler` that writes to `sys.stdout`.

logging.FileHandler

This handler writes the logs messages to a file. When you create a `FileHandler`, you give it the filename and an optional writing mode like this: `file_handler = logging.FileHandler("log.txt", mode="w")`.

The two most common modes for logging:

- "a" — (default) Append log messages to an existing file.

- "w" —Write a new file, discarding the old one.

Adding a handler

Once we have a handler, we need to add it to our component. You can do this with two UVM component methods:

- `self.add_logging_handler(handler)` —Adds a handler to the `self.logger` object in the component.
- `self.add_logging_handler_hier(handler)` —Adds a handler to the `self.logger` object in the component and all of its children.

Removing a handler

We can also remove a specific handler. The `handler` argument must hold the same handler you added previously.

- `self.remove_logging_handler(handler)` —Removes this handler from the `self.logger` object in the component.
- `self.remove_logging_handler_hier(handler)` —Removes this handler from the `self.logger` object in the component and all its children.

Removing the default StreamHandler

It is common to disable logging to the screen when running large simulations. Each UVM component comes with a `StreamHandler` object in its `self.logger` object. You can remove that `StreamHandler` object using the following methods:

- `self.remove_streaming_handler()` —Removes the `StreamHandler` from `self.logger` in the component.
- `self.remove_streaming_handler_hier()` —Removes the `StreamHandler` from the `self.logger` in the component and in all its children.

Here is an example of logging messages only to a file.

```
# Figure 5: Writing log entries to a file

class FileTest(LogTest) :
    def end_of_elaboration_phase(self) :
        file_handler = logging.FileHandler("log.txt", mode="w")
        self.add_logging_handler_hier(file_handler)
        self.remove_streaming_handler_hier()

--
INFO Starting test: "log_to_file"
    Description: Write log messages to file with no streaming
INFO Test Passed: log_to_file

% cat log.txt
INFO: testbench.py(10) [uvm_test_top] : This is info
WARNING: testbench.py(11) [uvm_test_top] : This is warning
ERROR: testbench.py(12) [uvm_test_top] : This is error
CRITICAL: testbench.py(13) [uvm_test_top] : This is critical
```

Disabling logging

To completely disable logging, use these methods:

- `self.disable_logging()` —Disables all logging in this component.
- `self.disable_logging_hier()` —Disables all logging in this component and its children.

Here we create blessed silence.

```
# Figure 6: Disabling log files

@pyuvm.test()
class NoLog(LogTest) :
    def end_of_elaboration_phase(self) :
        self.disable_logging_hier()

--
INFO NoLog passed
```

Changing the log message format

pyuvm provides a default logging format that looks like the format in the SystemVerilog UVM reporting system. You can change the format by creating a Python `Logging.Formatter` object and adding it to a new handler. Then you replace the default handler with your handler. Here are the steps:

1. Create a new handler.
2. Create a new formatter.
3. Add the formatter to the handler.
4. Use the `remove_streaming_handler()` method to remove the default streaming handler.
5. Use the `add_logging_handler()` method to add your new handler.

If you create a handler and do not provide a formatter, `add_logging_handler()` will use the default **pyuvm** formatter.

Summary

This chapter discussed logging in Python UVM, which uses the Python `Logging` module rather than implementing the UVM reporting system. The two systems are similar in that they can filter and direct logging.

Each `uvm_component` has a data member named `self.logger` that you use to log messages. **pyuvm** allows you to manipulate handlers on `self.logger`. You can disable all logging with the `disable_logging()` method.

ConfigDB()

In the `uvm_component` chapter, we learned to pass a `uvm_component` a name and a parent handle when we instantiate it. The UVM uses this information to create a testbench hierarchy.

The UVM testbench hierarchy enables reuse. We can instantiate components, environments, or even entire tests in another testbench without modifying it. The component works because properly written UVM code does not rely upon its location in a testbench hierarchy.

This rule creates a challenge when sharing data across a testbench. In testbench 4.0, we could share the `TinyAluBfm` handle with all components because `TinyAluBfm` is a singleton class. All calls to `TinyAluBfm()` returned a handle to the same object.

The singleton strategy will not work in a testbench with two TinyALUs because we'd need two BFM objects, but all calls to `TinyAluBfm()` would return handles to only one (it is a singleton, after all.)

We need a way to store data so that components with different paths will get different objects without interfering with each other. The `ConfigDB()` solves this problem.

A hierarchy-aware dictionary

We retrieve data from a dictionary by passing it a key and receiving the data stored at that key. We retrieve data from the `ConfigDB()` by passing our hierarchical path alongside the key. The `ConfigDB()` finds all the data whose path matches ours and then uses the key to retrieve the data.

The ConfigDB().get() method

We get data from the `ConfigDB()` using its `get()` method. For example, let's create a component that reads a message from the `ConfigDB()` and logs it. The code in figure 1 gets a message to log from the `ConfigDB()`.

Figure 1: Logging a message we get from the ConfigDB

```
class MsgLogger(uvm_component) :  
  
    async def run_phase(self) :  
        self.raise_objection()  
        msg = ConfigDB().get(self, "", "MSG")  
        self.logger.info(msg)  
        self.drop_objection()
```

The `ConfigDB().get()` method gets a string out of the `ConfigDB()` using the path to this component and the "MSG" key. It passes three arguments:

- `self` —This is called the *context object*. Typically we pass `self` as the context object. `Config().get()` gets the UVM hierarchy path to the context object using the `get_full_path()` method.
- `""` —The empty string could contain a path to be appended to the context object's path. We would use this if we had child components and wanted to create a path to them. We have no additional path information, so we pass `""`.
- `"MSG"` —This is the key that retrieves the data.

The `get()` method creates a path using `self` and `""`, then it checks to see if the `ConfigDB()` contains data stored with a path that matches this path. If there is a match, it uses `"MSG"` to retrieve data.

This raises the question of storing data.

The `ConfigDB().set()` method

The `MsgLogger` can be placed in any UVM testbench. Let's create a small testbench to test it. This testbench contains an environment and a test. The environment instantiates two `MsgLogger` objects.

Figure 2: Instantiating two loggers in the environment

```
class MsgEnv(uvm_env) :
    def build_phase(self) :
        self.loga = MsgLogger("loga", self)
        self.logb = MsgLogger("logb", self)
```

We want these two loggers to log different messages so the `MsgTest` stores different messages in the `ConfigDB()` using the `set()` method.

Figure 3: Giving loga and logb different messages

```
@pyuvm.test()
class MsgTest(uvm_test) :

    def build_phase(self) :
        self.env = MsgEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
        ConfigDB().set(self, "env.logb", "MSG", "LOG B msg")
```

The `set()` method has four arguments similar to the `get()` arguments:

- `self` —This is the `context object`. It works like the context object in `get().ConfigDB().set()` gets the full path to this object. In this case, the full path is `uvm_test_top`.
- `"env.loga" / "env.logb"` —We create the `set()` path by concatenating these strings to the context object's full path. The resulting paths are `uvm_test_top.env.loga` and `uvm_test_top.env.logb`. The data gets stored using this path.
- `"MSG"` —The `key.MsgLogger` looks for the `"MSG"` key. The data it finds at the key can depend upon its place in the component hierarchy.
- `"LOG A msg" / "LOG B msg"` —These are the messages for each logger.

When we run the test, the `get()` methods use a path such as `uvm_test_top.env.loga` to search the `ConfigDB`. If it finds a matching path, it uses the `"MSG"` key to get the `data.loga` and `logb` each get a different message this way.

```
# Figure 4: The loga and logb components have different
# things to say

--

INFO testbench.py(10) [uvm_test_top.env.loga] : LOG A msg
INFO testbench.py(10) [uvm_test_top.env.logb] : LOG B msg
```

Wildcards

We store data in the `ConfigDB()` using a path and a key. All the components whose paths match the path can use the key. Our example showed us storing "MSG" using a fully qualified path, for example, 'uvm_test_top.env.loga'. But, we also need to handle the case where we want multiple components to see our data. For that, we need wildcards.

Wildcard paths in **pyuvvm** work like wildcards on a Linux command line. The "*" character matches everything. For example, consider the case where we have components `talka` and `talkb` who should get the string "TALK TALK" at "MSG". Figure 5 shows an environment that instantiates these components as well as `loga` and `logb`.

```
# Figure 5: Adding talka and talkb to the environment

class MultiMsgEnv(MsgEnv) :
    def build_phase(self) :
        self.talka = MsgLogger("talka", self)
        self.talkb = MsgLogger("talkb", self)
        super().build_phase()
```

Now we create a test with different "MSG" strings for different paths. It uses a wildcard ("*") to give `talka` and `talkb` the same "MSG"

```
# Figure 6: Using a wildcard to store a message

@pyuvvm.test()
class MultiMsgTest(uvm_test) :
    def build_phase(self) :
        self.env = MultiMsgEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
        ConfigDB().set(self, "env.logb", "MSG", "LOG B msg")
        ConfigDB().set(self, "env.t*", "MSG", "TALK TALK")
```

We see that `talka` and `talkb` say "TALK TALK" while `loga` and `logb` are unchanged.

```
# Figure 7: The "talk*" components get the right message

INFO testbench.py(10) [uvm_test_top.env.talka] : TALK TALK
INFO testbench.py(10) [uvm_test_top.env.talkb] : TALK TALK
INFO testbench.py(10) [uvm_test_top.env.loga] : LOG A msg
INFO testbench.py(10) [uvm_test_top.env.logb] : LOG B msg
```

Global data

We often need to store data so that any component can get it. For example, we might have done this with a handle to the `TinyAluBfm()` in testbench 4.0. Global data can also act as a default when a `get()` call cannot find a matching path.

Figure 8: Storing data so any object can see it

```
ConfigDB().set(None, "*", "KEY", <data>)
```

When the first argument is `None` the `set()` method uses `uvm_test_top` as the context path. Then it appends `"*"` to it. All paths in the testbench hierarchy match this pattern since all paths start with `uvm_test_top`.

In the `GlobalEnv`, we add `gtalk` to the components.

Figure 9: Adding the gtalk component to the environment

```
class GlobalEnv(MultiMsgEnv) :
    def build_phase(self) :
        self.gtalk = MsgLogger("gtalk", self)
        super().build_phase()
```

The `GlobalTest` adds a new "MSG" using the global path.

Figure 10: Storing a global message

```
class GlobalTest(uvm_test) :
    def build_phase(self) :
        self.env = GlobalEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
        ConfigDB().set(self, "env.logb", "MSG", "LOG B msg")
        ConfigDB().set(self, "env.t*", "MSG", "TALK TALK")
        ConfigDB().set(None, "*", "MSG", "GLOBAL")
```

The result is as expected, `gtalk` picked up the global message.

Figure 11: Global is matched only if all others are not

```
INFO testbench.py(10) [uvm_test_top.env.gtalk] : GLOBAL
INFO testbench.py(10) [uvm_test_top.env.talka] : TALK TALK
INFO testbench.py(10) [uvm_test_top.env.talkb] : TALK TALK
INFO testbench.py(10) [uvm_test_top.env.loga] : LOG A msg
INFO testbench.py(10) [uvm_test_top.env.logb] : LOG B msg
```

Longest path wins

In the global example, we stored data with the `uvm_test_top.*`, which matches all `get()` requests. So if that path matches all get requests, why didn't it match `uvm_test_top.env.talka` or `uvm_test_top.env.logb` ?

The `ConfigDB().get()` solves the problem by matching the most specific path it finds. The `get()` command in `loga` matched `uvm_test_top.*` and `uvm_test_top.env.loga` . The longer, more specific path won.

Parent/child conflicts

`ConfigDB().set()` uses the supplied context object and a string to create the path stored along with the key and the data. This system allows a component to control which of its children gets a piece of data. But, it also allows conflicts between a parent and child object. Consider this code:

```
# Figure 12: Creating a ConfigDB conflict between
# parent and child at env.loga. Which message prints?

class ConflictEnv(uvm_env) :
    def build_phase(self) :
        self.loga = MsgLogger("loga", self)
        ConfigDB().set(self, "loga", "MSG", "CHILD RULES!")

class ConflictTest(uvm_test) :
    def build_phase(self) :
        self.env = ConflictEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "PARENT RULES!")
```

The `ConflictEnv` combines its `self` variable and "loga" to create the path "`uvm_test_top.env.loga`". Then the `ConflictTest` uses its `self` variable and the string "`env.loga`" to create the path "`uvm_test_top.env.loga`". Who wins this conflict?

```
# Figure 13: The parent wins

INFO testbench.py(10) [uvm_test_top.env.loga] : PARENT RULES!
```

The UVM specifies that when you call the `ConfigDB().set()` method in the build phase, the parent value wins in a conflict. However, if you call `ConfigDB().set()` outside the build phase, the data gets overwritten.

Summary

This chapter introduced us to the `ConfigDB()`. We saw how the `ConfigDB()` allows us to store different objects for different instances of the same UVM component.

The `ConfigDB()` can get complicated in a large testbench. In the next chapter, we'll learn about methods that control the `ConfigDB()` and help us debug it.

Debugging the ConfigDB()

Problems using the `ConfigDB()` come in two classes. Either you can't find the data or you get the wrong data. This chapter demonstrates how to catch missing data, trace `ConfigDB()` operations, and print the `ConfigDB()` state.

Missing data

If the `ConfigDB()` does not contain a key at the path we presented, it raises the `UVMConfigItemNotFound` exception. We forget to provide a "MSG" for `logb`. In figure 1, `MsgEnv` instantiates two loggers.

```
# Figure 1: Instantiate two loggers

class MsgEnv(uvm_env) :
    def build_phase(self) :
        self.loga = MsgLogger("loga", self)
        self.logb = MsgLogger("logb", self)
```

In figure 2, `MsgTest` fails to give `logb` a "MSG". As a result, `ConfigDB()` complains that it cannot find the provided UVM testbench path. We expect the test to fail, so we pass the expected exception to the `@pyuvm.test` decorator.

```
# Figure 2: Provide a message for only one logger

@pyuvm.test(expect_error=UVMConfigItemNotFound)
class MsgTest(uvm_test) :

    def build_phase(self) :
        self.env = MsgEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
    --
UVMConfigItemNotFound: "uvm_test_top.env.logb" is not in ConfigDB() .
```

In figure 3, `MsgTestAlmostFixed` tries to fix the problem by adding an entry for "`env.logb`". Sadly, it spells the key wrong ("MESG" vs "MSG") generating the same exception with a different message. The component is in the `ConfigDB()` but we cannot find "MESG".

```
# Figure 3: Misspelling a ConfigDB() key

@pyuvm.test(expect_error=UVMConfigItemNotFound)
class MsgTestAlmostFixed(uvm_test) :

    def build_phase(self) :
        self.env = MsgEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
        ConfigDB().set(self, "env.logb", "MESG", "LOG B msg")
    --
UVMConfigItemNotFound: "component uvm_test_top.env.logb has no key: MSG"
```

Catching exceptions

We no longer want to get ugly exceptions, so let's create the `NiceMsgLogger` that catches the `UVMConfigItemNotFound` exception, logs a warning, and sets a default.

Figure 4: Catching ConfigDB() exceptions

```
class NiceMsgLogger(uvm_component) :  
  
    async def run_phase(self) :  
        self.raise_objection()  
        try:  
            msg = ConfigDB().get(self, "", "MSG")  
        except UVMConfigItemNotFound:  
            self.logger.warning("Could not find MSG. Setting to default")  
            msg = "No message for you!"  
        self.logger.info(msg)  
        self.drop_objection()
```

In figure 5, we run the same tests with the `NiceMsgLogger`, and no longer see ugly exceptions.

Figure 5: Exception dumps replaced by warning messages

<path> = "uvm_test_top.env"

```
INFO    testbench.py(57) [<path>.loga] : LOG A msg  
WARNING testbench.py(55) [<path>.logb] : Could not find MSG. Setting to default  
INFO    testbench.py(57) [<path>.logb] : No message for you!
```

Printing the ConfigDB

We can debug missing data errors by printing the `ConfigDB()`. We can either print the `ConfigDB()` using the `print()` function, or we can create a string from the `ConfigDB()` using the `str()` constructor. In our first example we print the `ConfigDB()` with the missing `logb MSG` data. Using the `end_of_elaboration_phase()` method ensures that all the `build_phase()` methods have run.

Figure 6: Printing the ConfigDB()

```
class NiceMsgTest(uvm_test) :  
  
    def build_phase(self) :  
        self.env = NiceMsgEnv("env", self)  
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")  
  
    def end_of_elaboration_phase(self) :  
        print(ConfigDB())  
  
--  
PATH : KEY : DATA  
uvm_test_top.env.loga: MSG : {999: 'LOG A msg'}
```

Figure 6 shows the path, the key, and the data. We'll discuss the 999 number below. But we see that we have Log A msg but no data for the logb path. In figure 7, NiceMsgTestAlmostFixed misspells the key ("MESG" not "MSG"), but prints ConfigDB() so we can see the error.

```
# Figure 7: Debugging a ConfigDB() error using print()
```

```
class NiceMsgTestAlmostFixed(uvm_test) :
def build_phase(self) :
self.env = NiceMsgEnv("env", self)
ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
ConfigDB().set(self, "env.logb", "MESG", "LOG B msg")
def end_of_elaboration_phase(self) :
print(ConfigDB() )
--
PATH : KEY : DATA
uvm_test_top.env.loga: MSG : {999: 'LOG A msg'}
uvm_test_top.env.logb: MESG : {999: 'LOG B msg'}
```

We see an entry for uvm_test_top.env.logb, but it has the wrong key name ("MESG"). You can debug most ConfigDB() problems this way.

Debugging parent/child conflict

In the ConfigDB() chapter, we discussed the case where the parent and child both store data at the same path. The UVM specification says that if we call ConfigDB().set() in the build_phase() that the parent data wins. We can see these conflicts and how the parent wins by printing ConfigDB().

```
# Figure 8: Both parent and child reference env.loga
```

```
class ConflictTest(uvm_test) :
def build_phase(self) :
self.env = ConflictEnv("env", self)
ConfigDB().set(self, "env.loga", "MSG", "PARENT RULES!")

def end_of_elaboration_phase(self) :
print(ConfigDB())
```

In figure 9, we see that there is one entry for uvm_test_top.env.loga with the "MSG" key, but there are two values. The UVM specification says that the ConfigDB() should store a level number with each value counting down from 1000 at the top level. The parent value got the number 999, and the child value got 998, so the parent won. We see this when we print ConfigDB().

```
# Figure 9: The parent wins in the build_phase() method
```

```
PATH : KEY : DATA
uvm_test_top.env.loga : MSG : {999: 'PARENT RULES!', 998: 'CHILD RULES!'}
INFO testbench.py(12) [uvm_test_top.env.loga] : PARENT RULES!
```

The parent value is higher than the child value, overruling it.

Tracing ConfigDB() operations

Printing the `ConfigDB()` shows the state of the database, not how it got into that state. Tracing provides this information.

We trace `ConfigDB()` operations by setting its `is_tracing` variable to `True`. In this example we set `ConfigDB().is_tracing` to `True` and then see the set and get operations.

Figure 10: Tracing ConfigDB() operations using is_tracing

```
class GlobalTest(uvm_test) :
    def build_phase(self) :
        ConfigDB().is_tracing = True
        self.env = GlobalEnv("env", self)
        ConfigDB().set(self, "env.loga", "MSG", "LOG A msg")
        ConfigDB().set(self, "env.logb", "MSG", "LOG B msg")
        ConfigDB().set(self, "env.t*", "MSG", "TALK TALK")
        ConfigDB().set(None, "*", "MSG", "GLOBAL")
```

The `ConfigDB()` sends its operations to the log and identifies them with the `CFGDB/SET` and `CFGDB/GET` labels. First, we set the data. Notice that when we pass `None` to `set()` that the context is empty.

Figure 11: Watching ConfigDB() set data

```
INFO CFGDB/SET Context: uvm_test_top -- uvm_test_top.env.loga MSG=LOG A msg
INFO CFGDB/SET Context: uvm_test_top -- uvm_test_top.env.logb MSG=LOG B msg
INFO CFGDB/SET Context: uvm_test_top -- uvm_test_top.env.t* MSG=TALK TALK
INFO CFGDB/SET Context: -- * MSG=GLOBAL
```

The `CFGDB/GET` messages show the `get()` context and where `get()` found the data. Then it shows the data being returned. In this case, the `MsgLogger` logs the data immediately.

Figure 12: Watching ConfigDB() get data

```
INFO CFGDB/GET Context: uvm_test_top.env.gtalk -- uvm_test_top.env.gtalk MSG=GLOBAL
INFO testbench.py(12) [uvm_test_top.env.gtalk] : GLOBAL
INFO CFGDB/GET Context: uvm_test_top.env.ta1ka -- uvm_test_top.env.ta1ka MSG=TALK TALK
INFO testbench.py(12) [uvm_test_top.env.ta1ka] : TALK TALK
INFO CFGDB/GET Context: uvm_test_top.env.talkb -- uvm_test_top.env.talkb MSG=TALK TALK
INFO testbench.py(12) [uvm_test_top.env.talkb] : TALK TALK
INFO CFGDB/GET Context: uvm_test_top.env.loga -- uvm_test_top.env.loga MSG=LOG A msg
INFO testbench.py(12) [uvm_test_top.env.loga] : LOG A msg
INFO CFGDB/GET Context: uvm_test_top.env.logb -- uvm_test_top.env.logb MSG=LOG B msg
INFO testbench.py(12) [uvm_test_top.env.logb] : LOG B msg
```

Summary

This chapter demonstrated the `ConfigDB()` debug features. The `ConfigDB()` raises a `UVMConfigItemNotFound` exception when it cannot find the path or the key within the path. The `print()` function converts the `ConfigDB()` to a string and prints it, or you can create a string with the `str()` constructor by passing it `ConfigDB()` .

The `is_tracing` variable controls whether the `ConfigDB()` logs operations.

The UVM factory

In the *Design patterns* chapter, we discussed the *Factory pattern*, which allows you to instantiate an object without hardcoding the object type in the source code. For example, we used that pattern to create weapons in a Space War game.

The UVM provides a factory to use in our testbenches. Unlike the Space Game factory, the UVM factory allows us to override one class with another.

Remember that testbench 4.0 defined different environments for random tests and max tests. We'll see that the UVM factory allows us to use the same environment for both tests.

The create() method

Figure 1 creates a tiny component to illustrate factory behavior. Here is `TinyComponent`. It logs a message and exits.

Figure 1: A tiny example component

```
class TinyComponent(uvm_component) :
    async def run_phase(self) :
        self.raise_objection()
        self.logger.info("I'm so tiny!")
        self.drop_objection()
```

In figure 2, `TinyTest` instantiates the component by calling the class as we've done in all previous examples.

```
# Figure 2: Instantiating the component by calling the class directly

@pyuvm.test()
class TinyTest(uvm_test) :
    def build_phase(self) :
        self.tc = TinyComponent("tc", self)
```

Figure 3 shows the log message we expect.

```
# Figure 3: The expected log messages (with timestamps removed)
INFO running TinyTest (1/9)
INFO testbench.py(11) [uvm_test_top.tc] : I'm so tiny!
```

In figure 4, we modify the `TinyTest` to use the factory to create the `TinyComponent`. This test instantiates the component using the `TinyComponent.create()` class method instead of calling the class using `TinyComponent()`.

Figure 4: Instantiating a component using the UVM factory

```
@pyuvm.test()
class TinyFactoryTest(uvm_test) :
    def build_phase(self) :
        self.tc = TinyComponent.create("tc", self)
```

When calling a class method such as `create()`, do not put parenthesis after the class name. The result using `create()` is exactly the same as instantiating the object by calling the class (`TinyComponent()`).

Figure 5: We get the same result as in figure 3

```
INFO running TinyFactoryTest (2/9)
INFO testbench.py(11) [uvm_test_top.tc] : I'm so tiny!
```

Now that we're instantiating `TinyComponent` using the factory, we can override `TinyComponent` to create new behaviors.

uvm_factory()

The code in figure 4 used the `uvm_object.create()` class method to instantiate an object using the factory. We can also instantiate objects using the factory directly using the `uvm_factory()` singleton.

Creating objects using uvm_factory()

You can create any `uvm_object` or `uvm_component` directly using the `uvm_factory()` singleton. We create objects and components using different methods since objects take only their name as a creation argument, while components take their name and their parent's handle.

Note: The SystemVerilog UVM uses the word *type* where Python uses the word *class*.

- `create_object_by_type(self, requested_type, parent_inst_path="", name="")` —Creates an object whose class is `requested_type` and names it `name`. You can tell the factory the path to the component that contains the object using `parent_inst_path`.
- `create_object_by_name(self, requested_type_name, parent_inst_path="", name="")` —Creates an object whose class name is the string `requested_type_name` and names it `name`. You can tell the factory the path to the component that contains the object using `parent_inst_path`.
- `create_component_by_type(self, requested_type, parent_inst_path="", name="", parent=None)` —Creates a component whose class is `requested_type` and passes it `name` and `parent` as with any component instantiation. I don't know why the UVM specification calls for `parent_inst_path` as the information is in `parent`.
- `create_component_by_name(self, requested_type_name, parent_inst_path="", name="", parent=None)` —Creates a component whose class name is the string `requested_type_name` and passes it `name` and `parent` as with any component instantiation. Ignore `parent_inst_path`.

Figure 6 shows an example of creating a component from its name using the factory.

Figure 6: Passing a component name to the factory

```
class CreateTest(uvm_test) :
    def build_phase(self) :
        self.tc = uvm_factory().create_component_by_name(
            "TinyComponent",
            name="tc", parent=self)
--
INFO testbench.py(13) [uvm_test_top.tc] : I'm so tiny!
```

Creating objects using their class names is useful if you'd like to read a file and create objects listed there. For example, you could create a command file this way.

(I did this once in a testbench that tested numbered requirements. We had a file of requirement numbers, and the testbench read it and ran a test for each one.)

Overriding types

The UVM factory allows us to choose the object to create at run time.

The UVM factory allows you to override one class with another. If you override class `A` with class `B`, requests to create an object of class `A` will return a `B` instead.

We create overrides to change a testbench's behavior without modifying its structure. We'll see this in testbench 5.0 when we replace `RandomEnv` and `MaxEnv` with a single `AltEnv`.

Here are the override methods. Call them by using the `uvm_factory()` singleton as in `uvm_factory().set_type_override_by_type(...)`. As before the words *type* and *class* mean the same thing in **pyuvvm**:

- `set_type_override_by_type(self, original_type, override_type, replace=True)` —Override `original_type` with `override_type`. Both arguments are class objects and extensions of `uvm_object` or `uvm_component`. The `replace` argument tells **pyuvvm** to replace an existing override with this one.
- `set_type_override_by_name(self, original_type_name, override_type_name, replace=True)` — This method does the same as `set_type_override_by_type` but it takes strings as arguments with the names of the classes. The `replace` argument tells **pyuvvm** to replace an existing override with this one.

This example will override `TinyComponent` with `MediumComponent`. In figure 7, we define the `MediumComponent`.

Figure 7: Defining an example class

```
class MediumComponent(uvm_component) :
    async def run_phase(self) :
        self.raise_objection()
        self.logger.info("I'm medium size.")
        self.drop_objection()
```

Next we extend `TinyFactoryTest` to create `MediumFactoryTest`. Then, we use a factory override in the `build_phase()` before calling `super().build_phase()`. That code has not changed, but the factory override changes its behavior. Here is our `MediumFactoryTest`.

Figure 8: Overriding TinyComponent with MediumComponent

```
@pyuvm.test()
class MediumFactoryTest(TinyFactoryTest) :
    def build_phase(self) :
        uvm_factory().set_type_override_by_type(
            TinyComponent, MediumComponent)
        super().build_phase()
```

We have not copied any code, and we still got the result we want.

Figure 9: Now we get logs from the MediumComponent

```
INFO testbench.py(40) [uvm_test_top.tc] : I'm medium size.
```

The code in figure 10 overrides `TinyComponent` with `MediumComponent` using the string names.

Figure 10: Overriding types using string names

```
@pyuvm.test()
class MediumNameTest(TinyFactoryTest) :
    def build_phase(self) :
        uvm_factory().set_type_override_by_name(
            "TinyComponent", "MediumComponent")
        super().build_phase()
```

The result is the same:

Figure 11: The override is the same with strings

```
INFO testbench.py(40) [uvm_test_top.tc] : I'm medium size.
```

Factory overrides by instance

The override methods define global overrides. However, we can also create overrides that apply to only one instance in the UVM testbench hierarchy. The `uvm_factory()` provides two methods to do this. As before, *type* and *class* mean the same thing:

- `set_inst_override_by_type(self, original_type, override_type, full_inst_path)` —Overrides the original class with the override class only at a specific UVM instance, which you pass as a string.
- `set_inst_override_by_name(self, original_type_name, override_type_name, full_inst_path)` — Overrides the original class with the override class, with the class names passed as strings as is the `full_inst_path`.

In figure 12, we have two `TinyComponents` and will override one using its instance path. First, create an environment that instantiates the components, giving them the names `tc1` and `tc2`.

Figure 12: Creating instances to be overridden

```
class TwoCompEnv(uvm_env) :
    def build_phase(self) :
        self.tc1 = TinyComponent.create("tc1", self)
        self.tc2 = TinyComponent.create("tc2", self)
```

Now override `tc1` in the test.

Figure 13: Overriding an instance by type

```
class TwoCompTest(uvm_test) :
    def build_phase(self) :
        uvm_factory().set_inst_override_by_type(
            TinyComponent, MediumComponent, "uvm_test_top.env.tc1")
        self.env = TwoCompEnv("env", self)
```

The result is that `tc1` becomes a `MediumComponent` while `tc2` remains a `TinyComponent`.

Figure 14: The `tc1` instance has been overridden

```
INFO testbench.py(40) [uvm_test_top.env.tc1] : I'm medium size.
INFO testbench.py(13) [uvm_test_top.env.tc2] : I'm so tiny!
```

`inst_override_by_name()` uses the path the same way as `inst_override_by_type()` but takes strings as the original and override types.

Using the `create()` method carefully

As we saw at the beginning of the chapter, you enable the factory to override a component by using the component class's `create()` method like this: `self.tc = TinyComponent.create("tc", self)`.

Some argue that we should instantiate all our components using `create()`. They say this gives later users maximum flexibility to override anything.

I beg to differ.

As a testbench developer, I don't want a user to override a component I never intended to be overridden. Therefore, I recommend using `create()` only on objects that you intend to be overridden to maintain control of what is happening in your code.

Debugging `uvm_factory()`

The UVM describes a factory debug system that **pyuvm** augments for Python. The `uvm_factory.print()` method prints out information from the factory. It looks like this: `uvm_factory().print(all_types=1)`.

The `all_types` argument controls how much information gets printed. It has three legal values:

- 0 —Print overrides only.
- 1 —Print user-defined types and overrides. User-defined types don't start with "`uvm`".
- 2 —Print types whose names start with `uvm_`, user-defined types, and overrides.

We override `TinyComponent` with `MediumComponent` in figure 15. We can print this override in the `pyuvm` test:

```
# Figure 15: Printing only overrides

@pyuvm.test()
class PrintOverrides(MediumNameTest) :
    def final_phase(self) :
        uvm_factory() .print(0)
```

The `MediumComponent` prints its message. Then the `final_phase()` calls `uvm_factory().print(0)` to print the override. We see in figure 16 that `MediumComponent` overrides `TinyComponent` and that there are no instance overrides.

```
# Figure 16: Printing type overrides

INFO testbench.py(44) [uvm_test_top.tc] : I'm medium size.
--- overrides ---
Overrides:
TinyComponent: Type Override: MediumComponent | | Instance Overrides:
```

Figure 17 shows an example of printing an instance override.

```
# Figure 17: Printing instance overrides

@pyuvm.test()
class PrintInstanceOverrides(TwoCompTest) :
    def final_phase(self) :
        uvm_factory() .print(0)
```

This example shows only `uvm_test_top.env.tc1` being overridden with no generalized type override.

```
# Figure 18: Now we see instance overrides

INFO testbench.py(58) [uvm_test_top.env.tc1] : I'm medium size.
INFO testbench.py(15) [uvm_test_top.env.tc2] : I'm so tiny!
--- overrides ---

Overrides:
TinyComponent: Type Override: None | | Instance Overrides: uvm_test_top.env.tc1 =>
MediumComponent
```

Logging `uvm_factory()` data

While printing the factory state is valuable, logging it using the logging system would be better. We log the factory state by creating a string from the factory singleton and logging the string.

```
uvm_factory_str = str(uvm_factory() )
self.logger.info(uvm_factory_str)
```

The information in `uvm_factory_str` depends upon a variable named `uvm_factory().debug_level`. The debug level has the same definition as the `all_types` argument in the `print()` method.

- 0 —Show overrides only.
- 1 —Show user-defined types and overrides. User-defined types don't start with "`uvm_`".
- 2 —Show types whose names start with `uvm_`, user-defined types, and overrides.

In this example, we log the UVM factory in the `final_phase()` method.

Figure 19: Getting a string and logging the factory state

```
@pyuvm.test()
class LoggingOverrides(MediumNameTest) :
    def final_phase(self) :
        uvm_factory().debug_level = 0
        factory_log = "\n"+str(uvm_factory() )
        self.logger.info(factory_log)
```

Our override status is now in the log.

Figure 20: The factory state in a log entry

```
INFO testbench.py(58) [uvm_test_top.tc] : I'm medium size.
INFO testbench.py(132) [uvm_test_top] :
--- overrides ---
Overrides:
TinyComponent: Type Override: MediumComponent | | Instance Overrides:
```

Summary

In this chapter, we learned how to instantiate objects using the `create()` method when we want to enable the factory to override the class.

We learned how to override types using the class objects as arguments and how to override types using strings that contain the class names. We also learned to limit overrides to one point in the UVM hierarchy.

Finally, we learned how to print out the state of the factory and see the overrides.

UVM factory testbench: 5.0

In testbench 4.0, we created a class hierarchy containing a `BaseEnv`, a `RandomEnv`, and `MaxEnv`. `RandomEnv` instantiated the `RandomTester` and `MaxEnv` instantiated the `MaxTester`.

In testbench 5.0, we're eliminating the three kinds of environments and creating only one environment named `AluEnv`. Then, we'll use factory overrides to create tests.

AluEnv

In figure 1, the `AluEnv` instantiates the scoreboard and tester. Then it starts the `TinyAluBfm()` DUT-communication tasks.

Figure 1: Using the factory to instantiate the BaseTester

```
class AluEnv(uvm_env) :
    """Instantiate the scoreboard"""

    def build_phase(self) :
        self.scoreboard = Scoreboard("scoreboard", self)
        self.tester = BaseTester.create("tester", self)

    def start_of_simulation_phase(self) :
        TinyAluBfm().start_tasks()
```

We instantiate the `BaseTester` using the `create()` method so that the factory can override the instance with other classes.

The `BaseTester` is an abstract base class, and so it raises a `RuntimeError` exception if a user tries to run the test without an override.

Figure 2: The abstract class BaseTester raises an error if instantiated

```
class BaseTester(uvm_component) :
    def get_operands(self) :
        raise RuntimeError("You must extend BaseTester and override
get_operands().")

    async def run_phase(self) :
        self.raise_objection()
        self.bfm = TinyAluBfm()
        ops = list(Ops)
        for op in ops:
            aa, bb = self.get_operands()

# The code beyond this point never runs due to the exception
```

RandomTest

The `RandomTest` overrides the `BaseTester` with the `RandomTester`. The component path `uvm_test_top.env.scoreboard` has been snipped to fit on the page.

Figure 3: Overriding BaseTester with RandomTester

```
@pyuvm.test()
class RandomTest(uvm_test) :
    """Run with random operands"""
    def build_phase(self) :
        uvm_factory().set_type_override_by_type(BaseTester, RandomTester)
        self.env = AluEnv("env", self)

--
INFO testbench.py(74) [<snip>] : PASSED: ff ADD f5 = 01f4
INFO testbench.py(74) [<snip>] : PASSED: 66 AND 72 = 0062
INFO testbench.py(74) [<snip>] : PASSED: 78 XOR 62 = 001a
INFO testbench.py(74) [<snip>] : PASSED: 68 MUL 88 = 3740
INFO testbench.py(87) [<snip>] : Covered all operations
```

MaxTest

`MaxTest` works the same way as `RandomTest`. It overrides `BaseTester` and instantiates the `AluEnv`.

Figure 4: Overriding BaseTester with MaxTester

```
class MaxTest(uvm_test) :
    """Run with max operands"""
    def build_phase(self) :
        uvm_factory().set_type_override_by_type(BaseTester, MaxTester)
        self.env = AluEnv("env", self)

--
INFO testbench.py(74) [<snip>] : PASSED: ff ADD ff = 01fe
INFO testbench.py(74) [<snip>] : PASSED: ff AND ff = 00ff
INFO testbench.py(74) [<snip>] : PASSED: ff XOR ff = 0000
INFO testbench.py(74) [<snip>] : PASSED: ff MUL ff = fe01
INFO testbench.py(87) [<snip>] : Covered all operations
```

Summary

This chapter simplified testbench 4.0 using the factory. Testbench 5.0 has a single environment class that instantiates `BaseTester`, allowing `RandomTester` and `MaxTester` to override the `BaseTester` create tests.

Component communications

In the *cocotb Queue* chapter, we learned that **cocotb** allows parallel tasks to communicate using the `cocotb.queue.Queue` class. The UVM builds upon `Queue` using a standard named *Transaction Level Modeling 1.0 (TLM 1.0)*. TLM 1.0 was initially developed to allow SystemC transaction-level models to communicate. The idea was that you would design a system using C modules that communicated through TLM 1.0 and then replace the C modules with RTL modules as you implemented the design. TLM also works well for testbench components.

Why use TLM 1.0?

One may think that the Python `Queue` class is all one needs to define robust coroutine communication, but it solves only part of the problem.

The `Queue` solves the problem of allowing one coroutine to wait for data from another coroutine, and it stores the data being transferred. But, it doesn't solve the problem of creating a completely self-contained UVM component that can be reused in any testbench.

TLM 1.0 solves the problem of creating self-contained UVM components by defining the following:

1. A list of operations including *put*, *get*, and *peek*.
2. The two styles of operation: *blocking* and *nonblocking*.
3. A list of classes named *port* and *export* that combine the operations and their style.

Operations

The UVM 1800.2 specification names more than three communication operations, but we are only going to study the three popular ones:

- **put** —As with the `Queue`, the *put* operation sends a datum somewhere.
- **get** —As with the `Queue`, the *get* operation retrieves a datum from somewhere.
- **peek** —The `peek()` operation is unique to TLM 1.0 and is not in the `Queue`. It returns the datum that *would* have been retrieved by a *get* but leaves the datum in place.

Styles of operations

As with the `Queue`, TLM 1.0 defines two styles of operations:

- **blocking** —The coroutine blocks if the operation cannot be performed because there is either no datum or no room for a datum.
- **nonblocking** —The function does not block but returns a success flag to indicate whether the *put*, *get*, or *peek* was successful.

Ports

The UVM encapsulates the operations and their styles in *port* classes. It names the classes using the operations and the styles in all possible combinations. For example:

- `uvm_blocking_put_port` —Provides a `put()` coroutine that blocks if the data cannot be placed.
- `uvm_nonblocking_get_port` —Provides a `try_get()` function that returns a flag to indicate success.
- `uvm_blocking_peek_port` —Provides a `peek()` coroutine that blocks if there is no datum to peek.

The basic UVM ports provide one function such as the `put()` in `uvm_blocking_put_port` and `try_get()` in `uvm_nonblocking_get_port`. This allows the developer to limit the possible operations and styles in a component. However, there are times when a developer wants ports that take all styles. So the UVM provides generalized ports that provide all styles of operation.

- `uvm_put_port` —Provides the `put()` coroutine and the `try_put()` function.
- `uvm_get_port` —Provides the `get()` coroutine and the `try_get()` function.
- `uvm_peek_port` —Provides the `peek()` coroutine and the `try_peek()` function.

This example uses ports to create producer/consumer components. Figure 1 shows a `BlockingProducer` component that instantiates a `uvm_blocking_put_port` and then awaits `self.bpp.put()` to put data into the port and blocks if the port is full.

```
# Figure 1: A producer that blocks on a full FIFO

class BlockingProducer(uvm_component) :
    def build_phase(self) :
        self.bpp = uvm_blocking_put_port("bpp", self)

    async def run_phase(self) :
        self.raise_objection()
        for nn in range(3) :
            await self.bpp.put(nn)
            self.logger.info(f"Put {nn}")
        self.drop_objection()
```

The figure 1 code puts the data into the port and blocks if the port is full. The port must be connected to something, but we don't worry about that. Our parent component needs to connect our port to something.

The `BlockingConsumer` in figure 2 instantiates a `uvm_blocking_get_port` and awaits the `get()` coroutine to get data from the port and blocks if the port is empty.

```
# Figure 2: A consumer that blocks on an empty FIFO

class BlockingConsumer(uvm_component) :
    def build_phase(self) :
        self.bgp = uvm_blocking_get_port("bgp", self)

    async def run_phase(self) :
        while True:
            nn = await self.bgp.get()
            self.logger.info(f"Got {nn}")
```

We've mentioned that components using ports can put or get data to and from "wherever it is connected." Ports connect to another class of objects named *exports*.

Exports

Our port needs to be connected to an object that will retrieve or provide data. This object is called an *export*, and the UVM defines export classes that match the port classes, for example:

- `uvm_blocking_put_export`
- `uvm_nonblocking_get_export`
- `uvm_blocking_peek_export`

TLM 1.0 also defines exports that can be used for both blocking and nonblocking communication:

- `uvm_put_export`
- `uvm_get_export`
- `uvm_peek_export`

Users typically leverage export objects defined in classes such as the `uvm_tlm_fifo`.

uvm_tlm_fifo

The `uvm_tlm_fifo` class encapsulates a Python `Queue` and defines export classes to communicate with the `Queue`. The `uvm_tlm_fifo` provides all the combinations of put, get, peek, blocking, and nonblocking exports.

This example `uvm_test` connects the `BlockingProducer` to the `BlockingConsumer` through a `uvm_tlm_fifo`. Figure 3 contains a diagram of the connections in `BlockingTest`.

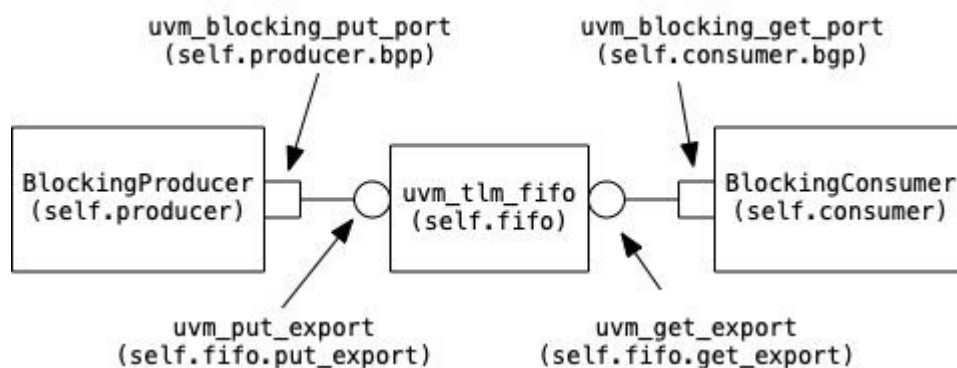


Figure 3: BlockingTest TLM diagram

Figure 3 shows the following:

- `BlockingProducer` is instantiated as `self.producer` and contains a `uvm_blocking_put_port` named `bpp`.
- `BlockingConsumer` is instantiated as `self.consumer` and contains a `uvm_blocking_get_port` named `bgp`.
- The `self.fifo` object contains a `uvm_put_export` named `put_export`. `self.producer.bpp` connects to `self.fifo.put_export`.
- The test instantiates `uvm_tlm_fifo` as `self.fifo`, which contains a `uvm_get_export` named `get_export`. `self.consumer.bgp` connects to `self.fifo.get_export`.

The code in figure 4 instantiates the objects in `build_phase()` and connects the ports to the exports in `connect_phase()`.

Figure 4: The connect_phase() in action

```
@pyuvm.test()
class BlockingTest(uvm_test) :
    def build_phase(self) :
        self.producer = BlockingProducer("producer", self)
        self.consumer = BlockingConsumer("consumer", self)
        self.fifo = uvm_tlm_fifo("fifo", self)

    def connect_phase(self) :
        self.producer.bpp.connect(self.fifo.put_export)
        self.consumer.bgp.connect(self.fifo.get_export)
```

Like all components, the `uvm_tlm_fifo` takes a name (" fifo ") and the handle to its parent (`self`). The `uvm_tlm_fifo()` call could also take a third argument named `size` that defaults to `1` . The `size` argument in `uvm_tlm_fifo` is analogous to the `maxsize` argument in `Queue` , though `maxsize` defaults to `0` . Since we used the default for `size` , `self.fifo` can only hold one object at a time.

This is the first time we've used the `connect_phase()` function. The function calls the `connect()` method in ports and passes them exports. For example, we call `self.producer.bpp.connect()` and pass it `self.fifo.put_export`.

Notice that the producer instantiated a `uvm_blocking_put_port` and the consumer instantiated a `uvm_blocking_get_port` , yet we connected them to a `uvm_put_export` and a `uvm_get_export` . These exports support both blocking and nonblocking communication.

The result in figure 5 is as we expect. The `Producer` produces numbers, and the `Consumer` consumes them.

Figure 5: Running the producer and consumer

```
INFO testbench.py(14) [uvm_test_top.producer] : Put 0
INFO testbench.py(25) [uvm_test_top.consumer] : Got 0
INFO testbench.py(14) [uvm_test_top.producer] : Put 1
INFO testbench.py(25) [uvm_test_top.consumer] : Got 1
INFO testbench.py(14) [uvm_test_top.producer] : Put 2
INFO testbench.py(25) [uvm_test_top.consumer] : Got 2
```

Nonblocking communication in pyuvm

We learned about blocking and nonblocking communication in the *cocotb Queue* chapter. We saw that `Queue` objects had blocking `put()` and `get()` coroutines that would block if the queue was full or empty. `Queue` objects also had nonblocking functions named `put_nowait()` and `get_nowait()` that would raise a `QueueFull` or `QueueEmpty` exception if the queue was full or empty, but would not block.

The SystemVerilog also implements nonblocking communication, using functions named `try_put()` and `try_get()` . Because SystemVerilog has no exception mechanism, `try_put()` and `try_get()` return a `bit` that is set to `1` on success and `0` on failure.

pyuvm implements these functions with the following modifications:

- The functions return `bool` value that is `True` for success and `False` for failure.
- Since one cannot return a value in the argument list in Python, `try_get()` returns a tuple containing `(success, datum)`.

Let's create the classes `NonBlockingProducer` and `NonBlockingConsumer` and instantiate them in `NonBlockingTest`.

Figure 6 shows the `NonBlockingProducer` class. It instantiates a `uvm_nonblocking_put_port` in the `build_phase()` and then uses a `while` loop to keep trying to put data into the port until it succeeds. If it fails to put the datum, it waits for 1 ns of simulation time before trying again.

```
# Figure 6: A producer that does not block on a full FIFO

class NonBlockingProducer(uvm_component) :
    def build_phase(self) :
        self.nbpp = uvm_nonblocking_put_port("nbpp", self)

    async def run_phase(self) :
        self.raise_objection()
        for nn in range(3) :
            self.logger.info(f"Putting: {nn}")
            success = False
            while not success:
                success = self.nbpp.try_put(nn)
                if success:
                    self.logger.info(f"Put {nn}")
                else:
                    self.logger.info("FIFO full")
                    await Timer(1, units="ns")
            await Timer(3, units="ns")
        self.drop_objection()
```

The `NonBlockingConsumer` uses a similar `while` loop to get the datum. If the queue is empty, `success` returns `False`, and the `while` loop waits 3 ns of simulation time before trying again.

```
# Figure 7: A consumer that does not block on an empty FIFO

class NonBlockingConsumer(uvm_component) :
    def build_phase(self) :
        self.nbgp = uvm_nonblocking_get_port("nbgp", self)

    async def run_phase(self) :
        while True:
            success = False
            while not success:
                success, nn = self.nbgp.try_get()
            if success:
                self.logger.info(f"Got {nn}")
            else:
                self.logger.info("FIFO empty")
                await Timer(3, units="ns")
```

Note: `try_put()` and `try_get()` are functions, not coroutines. If you `await` them, you get confusing errors about a coroutine being incompatible with tuples.

The `NonBlockingTest` instantiates these components and connects them as we did in `BlockingTest`.

Figure 8: Connecting the nonblocking components

```
@pyuvm.test()
class NonBlockingTest(uvm_test) :
    def build_phase(self) :
        self.producer = NonBlockingProducer("producer", self)
        self.consumer = NonBlockingConsumer("consumer", self)
        self.fifo = uvm_tlm_fifo("fifo", self)

    def connect_phase(self) :
        self.producer.nbpp.connect(self.fifo.put_export)
        self.consumer.nbgp.connect(self.fifo.get_export)
```

The test gives us the result in figure 9. The timestamps show the `Timer` delays. We snip the `uvm_test_top` part of the component path to fit on the page.

Figure 9: Nonblocking communication

```
1.00ns INFO testbench.py(52) [<snip>.producer] : Putting: 0
1.00ns INFO testbench.py(57) [<snip>.producer] : Put 0
1.00ns INFO testbench.py(52) [<snip>.producer] : Putting: 1
1.00ns INFO testbench.py(59) [<snip>.producer] : FIFO full
1.00ns INFO testbench.py(75) [<snip>.consumer] : Got 0
1.00ns INFO testbench.py(77) [<snip>.consumer] : FIFO empty
2.00ns INFO testbench.py(57) [<snip>.producer] : Put 1
2.00ns INFO testbench.py(52) [<snip>.producer] : Putting: 2
2.00ns INFO testbench.py(59) [<snip>.producer] : FIFO full
3.00ns INFO testbench.py(59) [<snip>.producer] : FIFO full
4.00ns INFO testbench.py(75) [<snip>.consumer] : Got 1
4.00ns INFO testbench.py(77) [<snip>.consumer] : FIFO empty
4.00ns INFO testbench.py(57) [<snip>.producer] : Put 2
7.00ns INFO testbench.py(75) [<snip>.consumer] : Got 2
7.00ns INFO testbench.py(77) [<snip>.consumer] : FIFO empty
```

Debugging uvm_tlm_fifo

In the chapter on logging, we learned about logging levels. The **logging** module defines levels ranging from 10 to 50. In addition, **pyuvm** adds another level for FIFO debugging.

Logging Level	Numeric Value
CRITICAL	50
WARNING	40
ERROR	30
INFO	20
DEBUG	10
FIFO_DEBUG	5
NOTSET	0

Figure 10: The FIFO_DEBUG logging level

We see the FIFO operations when we set the logging level for a component to `FIFO_DEBUG`. Logging FIFO activity results in a lot of messages, so it is best to target only one component when setting this level. Notice that we use `set_logging_level_hier()` since a `uvm_tlm_fifo` contains other components.

Figure 11: Enabling log messages from `uvm_tlm_fifo`

```
@pyuvm.test()
class LoggedBlockingtest(BlockingTest) :
    def end_of_elaboration_phase(self) :
        self.fifo.set_logging_level_hier(FIFO_DEBUG)
--
8.00ns FIFO_DEBUG [<snip>.fifo.put_export] : blocking put: 0
8.00ns FIFO_DEBUG [<snip>.fifo.put_export] : success put 0
8.00ns INFO testbench.py(14) [<snip>.producer] : Put 0
8.00ns FIFO_DEBUG [<snip>.fifo.put_export] : blocking put: 1
8.00ns FIFO_DEBUG [<snip>.fifo.get_export] : Attempting blocking get
8.00ns FIFO_DEBUG [<snip>.fifo.get_export] : got 0
8.00ns INFO testbench.py(25) [<snip>.consumer] : Got 0
<etc>
```

Summary

This chapter taught us how UVM components communicate using ports, exports, and TLM FIFOs. UVM components instantiate ports and call communication methods such as `put()` and `try_get()`.

We learned how to connect ports to exports in the `connect_phase()`. The `uvm_tlm_fifo` provides exports that match all the ports defined in the UVM.

Finally, we learned how to enable FIFO logging messages using the `FIFO_DEBUG` logging level.

Analysis ports

In the *Component communications* chapter, we examined communication between a single producer and a single consumer. This chapter will show how a single producer can broadcast data to many consumers.

We use this feature in testbenches to send monitored data to be analyzed by multiple components. For example, one component may check that the DUT is working correctly, while another component gathers coverage data.

Components that capture and analyze data from a testbench are said to make up the testbench's *analysis layer*. Analysis layer components receive data from the testbench and never block testbench operation. They are passive observers.

The uvm_analysis_port

The UVM implements the analysis layer using the `uvm_analysis_port` class. An analysis port cannot block, so it does not have a `put()` method. Instead, it has a `write()` method.

The UVM analysis system implements the *subscriber* design pattern. In the subscriber pattern, one object publishes data to zero or more subscribers (as with Twitter, for example.) The publisher in UVM instantiates an analysis port, and a higher-level component connects it to zero or more analysis exports.

For example, let's create a UVM component that generates ten random numbers and writes them to an analysis port. The `NumberGenerator` instantiates an analysis port named `self.ap` in the `build_phase()`. Then it generates ten random numbers in the `run_phase()` and calls `self.ap.write()` to send it to the subscribers.

```
# Figure 1: A random number generator

class NumberGenerator(uvm_component) :
    def build_phase(self) :
        self.ap = uvm_analysis_port("ap", self)

    async def run_phase(self) :
        self.raise_objection()
        for _ in range(10) :
            nn = random.randint(1, 10)
            print(nn, end=" ")
            self.ap.write(nn)
        print("")
        self.drop_objection()
```

The `write()` method is a function, and so it never blocks. If there are no subscribers, then nothing happens.

The `connect_phase()` function connects `uvm_analysis_port` objects to `uvm_analysis_export` objects. We pass a handle to the `uvm_analysis_export` object to the `uvm_analysis_port.connect()` method.

There are three ways to get a handle to a `uvm_analysis_export`:

1. Extend the `uvm_analysis_export` class and instantiate the class. It's best to name the instantiated object with the `_export` suffix so as to be clear that it is an export.

2. Instantiate a `uvm_tlm_analysis_fifo` which contains a `uvm_analysis_export` object named `analysis_export`.
3. Extend the `uvm_subscriber` class, which contains an object named `analysis_export`.

We'll examine each of these approaches.

Extend the `uvm_analysis_export` class

The `uvm_analysis_export` class is an abstract class that defines a `write()` method but leaves it empty. An analysis export class must have a `write()` method that takes a single object as an argument (in addition to `self`.)

Let's create an analysis export class named `Adder` that sums up the numbers it gets from the `NumberGenerator`. The required `write()` method adds the numbers it receives to `self.sum`. The `report_phase()` logs the sum.

```
# Figure 2: A component to add random numbers

class Adder(uvm_analysis_export) :
    def start_of_simulation_phase(self) :
        self.sum = 0

    def write(self, nn) :
        self.sum += nn

    def report_phase(self) :
        self.logger.info(f"Sum: {self.sum}")
```

We instantiate the `NumberGenerator` and `Adder` in `AdderTest` and connect the `self.sum_export` to the number generator's analysis port in the `connect_phase()`.

```
# Figure 3: Connecting a uvm_analysis_export to
# a uvm_analysis_port

@pyuvm.test()
class AdderTest(uvm_test) :
    def build_phase(self) :
        self.num_gen = NumberGenerator("num_gen", self)
        self.sum_export = Adder("sum", self)

    def connect_phase(self) :
        self.num_gen.ap.connect(self.sum_export)

--
1 4 5 9 2 1 4 9 1
INFO testbench.py(33) [uvm_test_top.sum] : Sum: 36
```

The generator created the numbers, and its subscriber summed them.

Forgetting to override write()

Since `uvm_tlm_analysis_export` is an abstract class, SystemVerilog gives you a syntax error if you forget to override the `write()` method. Python gives you a runtime error defined in **pyuvm**.

Figure 4: Failed to override the write() method

```
pyuvm.error_classes.UVMTLMConnectionError: If you extend
uvm_analysis_export, you must override the
write method.
```

Extend the uvm_subscriber class

Extending the `uvm_subscriber` is like extending the `uvm_analysis_export` in that you must override the `write()` method. The difference is that the `uvm_subscriber` creates an `analysis_export` data member in its `__init__()` method.

In figure 5, we extend the `uvm_subscriber` to write `Median`. The `Median` class creates a list in the `start_of_simulation_phase()` method. Then it fills the list in the `write()` method. It uses the **statistics** package to log the median number.

Figure 5: Using the statistics package to find the median

```
import statistics

class Median(uvm_subscriber) :

    def start_of_simulation_phase(self) :
        self.numb_list = []

    def write(self, nn) :
        self.numb_list.append(nn)

    def report_phase(self) :
        self.logger.info(f"Median: {statistics.median(self.numb_list) }")
```

In figure 6, we instantiate the `Median` and connect it in the `MedianTest`. Notice that `self.median` is different than `self.sum_export` in that we connect the `self.median.analysis_export` data member.

Figure 6: A test that provides the sum and median

```
@pyuvm.test()
class MedianTest(uvm_test) :
    def build_phase(self) :
        self.num_gen = NumberGenerator("num_gen", self)
        self.sum_export = Adder("sum", self)
        self.median = Median("median", self)

    def connect_phase(self) :
        self.num_gen.ap.connect(self.sum_export)
```

```

        self.num_gen.ap.connect(self.median.analysis_export)

--
3 5 2 9 10 9 2 2 10
INFO testbench.py(35) [uvm_test_top.sum] : Sum: 52
INFO testbench.py(61) [uvm_test_top.median] : Median: 5

```

Here we see the analysis port's superpower. It can connect to any number of analysis exports, and they'll all get a copy of the data. Next we'll demonstrate the third way to connect to an analysis port.

Instantiate a `uvm_tlm_analysis_fifo`

The `uvm_tlm_analysis_fifo` extends the `uvm_tlm_fifo` discussed in the *Component communications* chapter.

The `uvm_tlm_analysis_fifo` adds the following to `uvm_tlm_fifo` :

1. It contains an `analysis_export` object that you can connect to an analysis port.
2. It is of infinite depth by default.

This allows the analysis FIFO to store all the data from a simulation and analyze it in the `check_phase()` method. You can use a get port to remove the data from the FIFO when you want to check it.

The `Average` component stores all the data in the analysis FIFO and averages it after the run phase ends. We instantiate a `uvm_tlm_analysis_fifo` to receive the data. The `non_blocking_get_port` allows us to loop through the data in the FIFO and average it.

```

# Figure 7: A component that averages random numbers

class Average(uvm_component) :
    def build_phase(self) :
        self.fifo = uvm_tlm_analysis_fifo("fifo", self)
        self.nbgp = uvm_nonblocking_get_port("nbgp", self)

    def connect_phase(self) :
        self.nbgp.connect(self.fifo.get_export)
        self.analysis_export = self.fifo.analysis_export

    def report_phase(self) :
        success = True
        sum = 0
        count = 0
        while success:
            success, nn = self.nbgp.try_get()
            if success:
                sum += nn
                count += 1
        self.logger.info(f"Average: {sum/count:0.2f}")

```

Providing the analysis_export data attribute

In figure 7, We created the `Average` class by instantiating a `uvm_tlm_analysis_fifo` as `self.fifo`. The `self.fifo` object provides an `analysis_export`.

The user could connect the analysis port to the FIFO's analysis export, but we don't want users to access objects inside our `Average` class. To avoid this, we copy `self.fifo.analysis_export` to `self.analysis_export` so that the user can pass `self.avg.analysis_export` to the analysis port's `connect()` method.

The `AverageTest` in figure 8 instantiates the `Adder`, `Median`, and `Average` classes in the build phase. Then it connects `self.sum_export` directly to `self.num_gen.ap` and it connects `self.avg.analysis_export` to the same analysis port.

Figure 8: A test that adds and averages random numbers

```
class AverageTest(uvm_test) :
    def build_phase(self) :
        self.num_gen = NumberGenerator("num_gen", self)
        self.sum_export = Adder("sum", self)
        self.median = Median("median", self)
        self.avg = Average("avg", self)

    def connect_phase(self) :
        self.num_gen.ap.connect(self.sum_export)
        self.num_gen.ap.connect(self.median.analysis_export)
        self.num_gen.ap.connect(self.avg.analysis_export)

--
3 7 9 8 10 4 5 10 10
INFO testbench.py(35) [uvm_test_top.sum] : Sum: 66
INFO testbench.py(61) [uvm_test_top.median] : Median: 8
INFO testbench.py(97) [uvm_test_top.avg] : Average: 7.33
```

Summary

In this chapter, we learned about analysis ports and exports. Analysis ports contain a `write()` function instead of a `put()` coroutine or a `try_put()` function. They never block.

We attach analysis ports to analysis exports. There are three ways to get an analysis export:

1. Extend `uvm_analysis_export` and override the `write()` method.
2. Extend `uvm_subscriber` and override the `write()` method. Then connect it using its `analysis_export` data member.
3. Instantiate a `uvm_tlm_analysis_fifo` to get an object that contains an `analysis_export` data member.

Components in testbench 6.0

Now that we've learned about the configuration database, the factory, logging, and component communication, we are ready to create testbench 6.0.

We will use port objects to create components with a single purpose, and then we'll connect these components using FIFOs to create the testbench.

We're going to create 6.0 in two steps. In this chapter, we'll refactor the components so that each one does a single job and uses a port to communicate with the rest of the testbench. Then, in the next chapter, we'll connect the components using `uvm_tlm_fifo` objects.

The testers

As in testbench 5.0, the tester class hierarchy consists of a `BaseTester` and extensions named `RandomTester` and `MaxTester`.

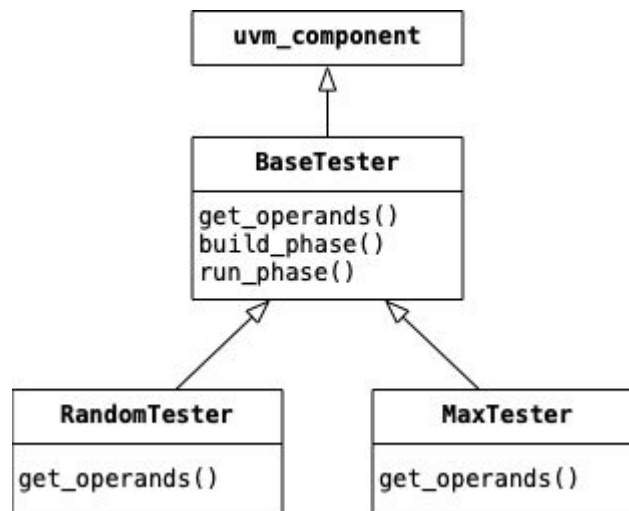


Figure 1: Testers' component class structure

The class structure in 6.0 is the same as 5.0. We have a `BaseTester` extended by `RandomTester` and `MaxTester`.

BaseTester

Instead of calling `self.bfm.send_op(aa, bb, op)` the 6.0 `BaseTester` puts the command data into a tuple and writes it to a blocking put port.

```
# Figure 2: The BaseTester using TLM 1.0

class BaseTester(uvm_component) :

    def build_phase(self) :
        self.pp = uvm_put_port("pp", self)

    async def run_phase(self) :
        self.raise_objection()
        self.bfm = TinyAluBfm()
        ops = list(ops)
```

```

        for op in ops:
            aa, bb = self.get_operands()
            cmd_tuple = (aa, bb, op)
            await self.pp.put(cmd_tuple)
            await ClockCycles(signal=cocotb.top.clk, num_cycles=10,
                               rising=False)
            self.drop_objection()

    def get_operands(self) :
        raise RuntimeError("You must extend BaseTester and override it.")

```

The figure 2 version of the `BaseTester` raises an exception if you try to instantiate it directly rather than using one of its child classes. The `ClockCycles` trigger waits for ten clocks and demonstrates how you can access DUT signals from anywhere in your testbench using `cocotb.top`. The ten clocks allow the multiply operation to get through the FIFOs and complete.

RandomTester and MaxTester

The `RandomTester` and `MaxTester` extend `BaseTester` and override the `get_operands()` method.

```

# Figure 3: The RandomTester and MaxTester
# override get_operands() to do their jobs

class RandomTester(BaseTester) :
    def get_operands(self) :
        return random.randint(0, 255) , random.randint(0, 255)

class MaxTester(BaseTester) :
    def get_operands(self) :
        return 0xFF, 0xFF

```

Since the `BaseTester` no longer talks to the TinyALU directly, we'll need a `Driver` class.

Driver

The `Driver` gets commands from a blocking get port and calls `bfm.send_op()` to send them to the TinyALU. The `build_phase()` gets the `TinyAluBfm()` singleton and instantiates a `uvm_get_port`. The `uvm_get_port` provides blocking and nonblocking methods.

Its `run_phase()` resets the TinyALU and starts the BFM tasks. Then it loops forever, getting commands from the get port and sending them to the BFM.

```

# Figure 4: The Driver class takes commands and sends them
# to the TinyALU using the BFM.

class Driver(uvm_driver) :
    def build_phase(self) :
        self.bfm = TinyAluBfm()
        self.gp = uvm_get_port("gp", self)

    async def run_phase(self) :

```

```

await self.bfm.reset()
self.bfm.start_tasks()
while True:
    aa, bb, op = await self.gp.get()
    await self.bfm.send_op(aa, bb, op)

```

The blocking behavior synchronizes the testbench with the DUT.

Monitor

The `Monitor` class demonstrates a Python feature that does not exist in SystemVerilog—the ability to dynamically access a method in an object using the method's name.

Recall that the `TinyAluBfm` provides `get_cmd()` and `get_result()` methods. In SystemVerilog, we would have to create two `Monitor` classes, one that calls `get_cmd()` and another that calls `get_result()`. In Python, we can write a single class that takes the method's name as an instantiation argument. We instantiate it like this.

```

# Figure 5: Instantiating Monitor() and passing different
# function names to get different data

self.cmd_monitor = Monitor("cmd_monitor", self, "get_cmd")
self.result_monitor = Monitor("result_monitor", self, "get_result")

```

Since `Monitor` has an additional argument in its instantiation, we must override its `__init__()` method. The `Monitor.__init__()` immediately calls `super().__init__(name, parent)` as is required. Then it stores `method_name`.

```

# Figure 6: The Monitor() class takes the method name
# as an instantiation argument

class Monitor(uvm_monitor) :
    def __init__(self, name, parent, method_name) :
        super().__init__(name, parent)
        self.method_name = method_name

```

The `build_phase()` instantiates a `uvm_analysis_port` and gets the `TinyAluBfm()` singleton. Then, it uses `getattr(self.bfm, self.method_name)` to get the handle to the monitoring method from the BFM.

```

# Figure 8: Getting the datum and writing
# it to the analysis port

async def run_phase(self) :
    while True:
        datum = await self.get_method()
        self.ap.write(datum)

```

Coverage

Testbench 6.0 breaks the `Scoreboard` into two classes. One, still named `Scoreboard`, compares actual results to predicted results. The other `Coverage` uses a set to ensure we've tested all the operations.

`Coverage` extends `uvm_analysis_export`, so you can directly connect it to an analysis port. However, you must override the `write()` method. This `write` method in `Coverage` extracts the op from the `cmd` tuple and adds it to the `cvg` set.

Figure 9: Extending `uvm_analysis_export` directly

```
class Coverage(uvm_analysis_export) :

    def start_of_simulation_phase(self) :
        self.cvg = set()

    def write(self, cmd) :
        _, _, op = cmd
        self.cvg.add(ops(op) )

    def check_phase(self) :
        if len(set(ops) - self.cvg) > 0:
            self.logger.error(
                f"Functional coverage error. Missed: {set(ops) -self.cvg}")
            assert False
        else:
            self.logger.info("Covered all operations")
```

The `check_phase()` method ensures that we've tested all the operations. The line `assert False` tells cocotb that we have missed an operation.

The Scoreboard

Like the `Scoreboard` in testbench 5.0, the `Scoreboard` in testbench 6.0 stores commands and results during the simulation and checks them after the run phase has been completed. The difference is that the 6.0 `Scoreboard` stores simulation in two `uvm_tlm_analysis_fifo` objects. Then reads the data from the FIFOs in the `check_phase`.

Scoreboard.build_phase()

We start with a `build_phase()` that instantiates two `uvm_tlm_analysis_fifo` objects and two `uvm_get_port` objects. We'll use the get ports to read the FIFOs.


```

# Figure 10: Using uvm_tlm_analysis_fifo to provide
# multiple analysis_exports
# and creating ports to read the fifos

class Scoreboard(uvm_component) :

    def build_phase(self) :
        self.cmd_mon_fifo = uvm_tlm_analysis_fifo("cmd_mon_fifo", self)
        self.result_mon_fifo = uvm_tlm_analysis_fifo("result_mon_fifo", self)
        self.cmd_gp = uvm_get_port("cmd_gp", self)
        self.result_gp = uvm_get_port("result_gp", self)

```

Scoreboard.connect_phase()

We connect the FIFOs to the get ports by passing the FIFO's `get_export` object to the get port's `connect()` method.

```

# Figure 11: Connecting get ports to the FIFOs and
# making the FIFO analysis exports visible to the user

def connect_phase(self) :
    self.cmd_gp.connect(self.cmd_mon_fifo.get_export)
    self.result_gp.connect(self.result_mon_fifo.get_export)
    self.cmd_export = self.cmd_mon_fifo.analysis_export
    self.result_export = self.result_mon_fifo.analysis_export

```

We also copy the FIFO `analysis_export` objects to `self.cmd_export` and `self.result_export` to make it easy to connect the scoreboard to analysis ports.

Scoreboard.check_phase()

The 6.0 `check_phase()` gets the data from the `cmd_gp` get port using `try_get`. When `get_next_cmd` is `False` we break out of the loop as we have emptied the FIFO.

Once we have a command, we get the associated result. If there is no result, the testbench is broken, so we raise a `RuntimeError` exception.

```

# Figure 12: Check results after the run phase

def check_phase(self) :
    passed = True
    while True:
        got_next_cmd, cmd = self.cmd_gp.try_get()
        if not got_next_cmd:
            break
        result_exists, actual = self.result_gp.try_get()
        if not result_exists:
            raise RuntimeError(f"Missing result for command {cmd}")

<snipped checking code>

```

The rest of the code is identical to 5.0. It predicts the result and compares it to the actual result.

Summary

We've now simplified the components in the UVM testbench down to their minimum behavior. Each component either creates data and writes it to a port or gets data from a port and processes it.

We'll connect these components to create the testbench in the next chapter.

Connections in testbench 6.0

In the *Testbench components* chapter, we modularized the testbench by creating components that did one thing each and used ports to send data to another component or get data from another component. Here are the components:

- `RandomTester` —Puts TinyALU command tuples with random operands into a blocking put port.
- `MaxTester` —Puts TinyALU command tuples with `0xFF` operands into a blocking put port.
- `Driver` —Gets TinyALU commands from a blocking get port and uses `bfm.send_op()` to send them to the TinyALU DUT.
- `Monitor("get_cmd")` —Uses `bfm.get_cmd()` to get a command from the TinyALU DUT and puts the tuple into an analysis port.
- `Monitor("get_result")` —Uses `bfm.get_result()` to get a result from the TinyALU DUT and puts the result into an analysis port.
- `Coverage` —Receives commands in its `write()` method and stores the operation in a set. Then it checks that we've covered all the operations.
- `Scoreboard` —Receives commands and their results and stores them in `uvm_tlm_analysis_fifo` objects, then uses nonblocking get ports to read them and compare actual and predicted results.

Each of these components is easy to understand and easy to maintain. Next, we connect them to create a testbench. We do that in the `AluEnv` class.

The AluEnv TLM diagram

Environments instantiate components and connect them. Figure 1 shows the `AluEnv` TLM diagram.

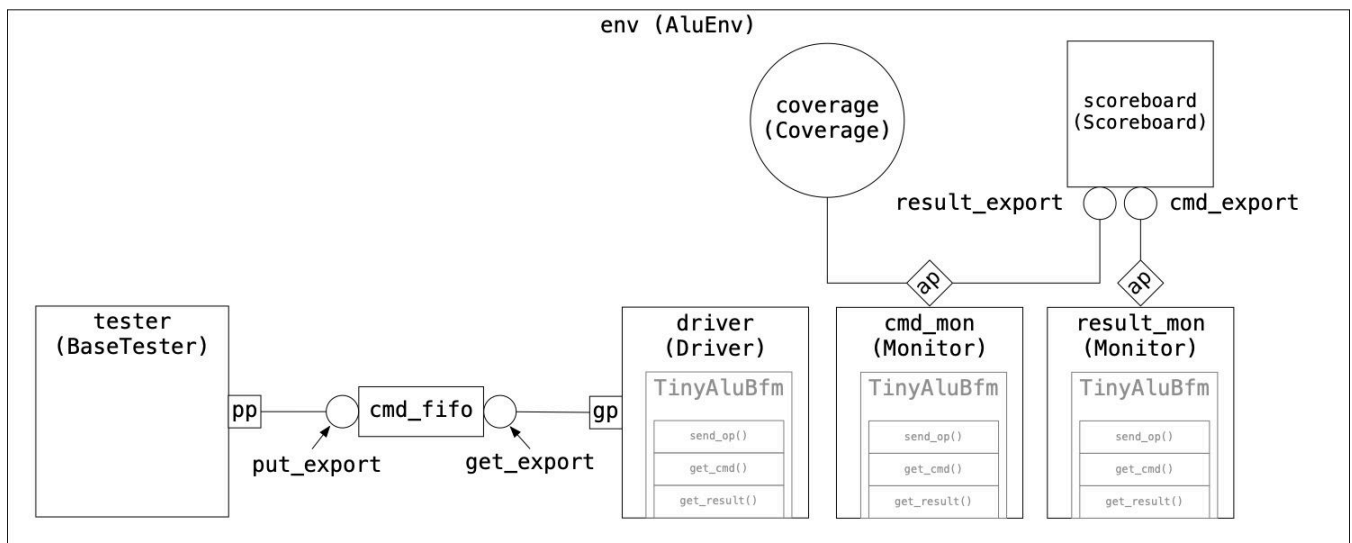


Figure 1: Component connections in AluEnv

This diagram shows how the environment connects the components. The small squares represent ports, such as the blocking put port named `pp` on the `Tester`. The circles represent exports such as the `put_export` on the `cmd_fifo`. The `Coverage` class extends `uvm_analysis_export`, so the entire object is a circle. The lines represent times that we called `connect()` on a port and passed it an export.

AluEnv.build_phase()

The `build_phase()` function in figure 2 instantiates all the components and the FIFOs. It instantiates `BaseTester` using the `create()` method so that the factory can override the `BaseTester` with `RandomTester` and `MaxTester`.

```
# Figure 2: Instantiating the stimulus layer

class AluEnv(uvm_env) :

    def build_phase(self) :
        self.tester = BaseTester.create("tester", self)
        self.driver = Driver("driver", self)
        self.cmd_fifo = uvm_tlm_fifo("cmd_fifo", self)
```

The `Scoreboard` and `Coverage` objects in figure 3 comprise the analysis layer. We will connect them to the analysis ports in the `Monitor` objects.

```
# Figure 3: Instantiating the analysis layer

self.scoreboard = Scoreboard("scoreboard", self)
self.coverage = Coverage("coverage", self)
```

The code in figure 4 instantiates the `self.cmd_mon` and `self.result_mon` using the same `Monitor` class, but pass the names of the monitoring methods. The "get_cmd" or "get_result" strings differentiate between the two `Monitor` objects.

```
# Figure 4: Instantiating the Monitor class by passing
# monitoring method names

self.cmd_mon = Monitor("cmd_monitor", self, "get_cmd")
self.result_mon = Monitor("result_monitor", self, "get_result")
```

AluEnv.connect_phase()

Each line in the `AluEnv` TLM diagram in figure 1 represents a call to the port's `connect()` method. When we pass an export to the `connect()` method, we connect the port to the export. First we connect the `tester.pp` and `driver.gp` ports to the `cmd_fifo` exports.

```
# Figure 5: Connecting ports to the FIFO exports

def connect_phase(self) :
    self.tester.pp.connect(self.cmd_fifo.put_export)
    self.driver.gp.connect(self.cmd_fifo.get_export)
```

Since the `Coverage` class extends `uvm_analysis_export` we can pass `self.coverage` directly to the `cmd_mon.ap.connect()` method, as in figure 6. Notice that we easily connect the scoreboard as we planned when we copied the FIFO `analysis_export` objects to scoreboard data attributes.

```
# Figure 6: Connecting the coverage analysis_export
# to the cmd_mon analysis port

self.cmd_mon.ap.connect(self.coverage)
self.cmd_mon.ap.connect(self.scoreboard.cmd_export)
self.result_mon.ap.connect(self.scoreboard.result_export)
```

RandomTest and MaxTest

The tests in figure 7 use the factory to override `BaseTester`. Then, they instantiate the `AluEnv`.

```
# Figure 7: RandomTest and MaxTest override the BaseTester

@pyuvm.test()
class RandomTest(uvm_test) :
    def build_phase(self) :
        uvm_factory().set_type_override_by_type(BaseTester, RandomTester)
        self.env = AluEnv("env", self)

@pyuvm.test()
class MaxTest(uvm_test) :
    def build_phase(self) :
        uvm_factory().set_type_override_by_type(BaseTester, MaxTester)
        self.env = AluEnv("env", self)
```

The two tests run and use random operands, then maximum operands. The component paths have been snipped so as to fit the output on the page.

```
# Figure 8: RandomTest results
INFO testbench.py(146) [<snip>] : PASSED: 66 ADD e6 = 014c
INFO testbench.py(146) [<snip>] : PASSED: bd AND 4a = 0008
INFO testbench.py(146) [<snip>] : PASSED: 2e XOR 2d = 0003
INFO testbench.py(146) [<snip>] : PASSED: c8 MUL d9 = a988
INFO testbench.py(108) [<snip>] : Covered all operations
```

`MaxTest` also passes with all 1's on the operand buses.

```
# Figure 9: MaxTest Results
INFO running MaxTest (2/2)
INFO testbench.py(146) [<snip>] : PASSED: ff ADD ff = 01fe
INFO testbench.py(146) [<snip>] : PASSED: ff AND ff = 00ff
INFO testbench.py(146) [<snip>] : PASSED: ff XOR ff = 0000
INFO testbench.py(146) [<snip>] : PASSED: ff MUL ff = fe01
INFO testbench.py(108) [<snip>] : Covered all operations
```

Summary

This chapter completed testbench 6.0, a UVM testbench made of simple components that communicate using ports and an environment that instantiates the components and connects them.

uvm_object in Python

It's tempting to think that testbench 6.0 is the final step in our journey to the best UVM testbench. It's modular, uses TLM communications, has logging, a `ConfigDB()`, and the `uvm_factory()`. What more could we need?

We need a way to create a testbench that doesn't need to be restructured every time we change the test stimulus. For example, we'd like to create a testbench that can read commands from a file and execute them. Or we'd like our test writers to be able to write tests using standardized testbench commands. We can't do these things if we have to rebuild the testbench each time we change stimulus.

The UVM solves this problem using UVM *sequences*, which must not be confused with Python sequences. The UVM and Python happen to use the same word for different things.

UVM sequences send commands to the DUT in a class called a `uvm_sequence_item`. Here is the `uvm_sequence_item` UML diagram.

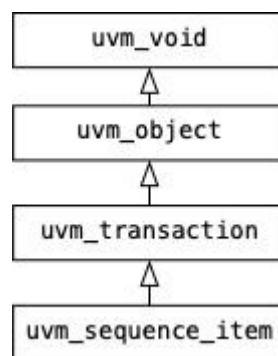


Figure 1: `uvm_sequence_item` UML

In the SystemVerilog UVM, the `uvm_object` contains convenience routines that allow programmers to print `uvm_objects` and copy or clone them easily. This chapter shows how **pyuvm** replaces these convenience routines with Python magic methods and modules from the Python ecosystem.

Creating a string from an object

The SystemVerilog UVM specifies that classes that extend `uvm_object` should implement a `convert2string()` method. This method returns a string representation of the object.

Python has the same functionality but uses the magic method `__str__()` to return a string representation of the object. Then you can use the `str()` function to create a new string from the object, print the object, or do any operation that needs a string.

In figure 2, we create a `PersonRecord` class that extends `uvm_object` and adds an ID number. The `uvm_object` class already has a `name` field that we access using `get_name()`. The `__str__()` magic method returns a string with the record information.

Figure 2: Extending uvm_object

```
class PersonRecord(uvm_object) :
    def __init__(self, name="", id_number=None) :
        super().__init__(name)
        self.id_number = id_number

    def __str__(self) :
        return f"Name: {self.get_name() }, ID: {self.id_number}"
```

The `__str__()` magic method allows us to print the record. We could also create a `str` from the record and log it. Figure 3 uses a **cocotb** test to create the object and print it.

Figure 3: Printing a uvm_object

```
@pyuvm.test()
class TestStr(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        xx = PersonRecord("Joe Shmoe", 37)
        print("Printing the record:", xx)
        logger.info("Logging the record: " + str(xx) )
        self.drop_objection()

--
Printing the record: Name: Joe Shmoe, ID: 37
INFO Logging the record: Name: Joe Shmoe, ID: 37
```

Comparing objects

The SystemVerilog UVM `compare()` method compares two objects like this.

Figure 4: SystemVerilog compares uvm_objects using compare()

```
same = xx.compare(yy)
```

`same` receives a `1` if `xx` and `yy` are considered to contain the same data. I say “are considered to contain” because different objects may have different definitions of being “the same.” For example, our `PersonRecord` class will compare only the `id_number`. In SystemVerilog, we implement this using the `do_compare()` function.

Figure 5: The programmer overrides do_compare() to implement compare()

```
function bit do_compare(yy) begin
    return this.id_number == yy.id_number
endfunction
```

In Python, we use the `__eq__()` magic method.

Figure 6: Comparing pyuvvm uvm_objects using `__eq__()`

```
class PersonRecord(uvm_object) :
    def __init__(self, name="", id_number=None) :
        super() .__init__(name)
        self.id_number = id_number

    def __eq__(self, other) :
        return self.id_number == other.id_number
```

Unlike SystemVerilog, Python allows us to use the `==` operator to compare objects and print an important bit of info.

Figure 7: Comparing uvm_objects using `==`

```
@pyuvvm.test()
class TestEq(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        batman = PersonRecord("Batman", 27)
        bruce_wayne = PersonRecord("Bruce Wayne", 27)
        if batman == bruce_wayne:
            logger.info("Batman is really Bruce Wayne!")
        else:
            logger.info("Who is Batman?")
        self.drop_objection()
    --
INFO Batman is really Bruce Wayne!
```

Copying and cloning

Python and the UVM have different systems for copying and cloning objects. I recommend using the Python **copy** module, which is the easier of the two. **pyuvvm** implements the UVM system to follow the UVM 1800.2 specification.

This example uses a `PersonRecord` (defined in figure 2) and its extension the `StudentRecord`. A person can be a student. The difference between them is that a student has a list of grades which must also be printed.

Figure 8: Extending `PersonRecord` and leveraging `__str__()`

```
class StudentRecord(PersonRecord) :
    def __init__(self, name="", id_number=None, grades=[] ) :
        super() .__init__(name, id_number)
        self.grades = grades

    def __str__(self) :
        return super() .__str__() + f" Grades: {self.grades}"
```


Copying with Python

The code in Figure 10 creates a student object named `mary` and copies it using the Python **copy** module function `copy.copy()`. Notice that `mary.grades` and `mary_copy.grades` are the same object with the same ID.

```
# Figure 9: Copying an object with copy.copy()

import copy
@pyuvm.test()
class CopyCopyTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        mary = StudentRecord("Mary", 33, [97, 82] )
        mary_copy = StudentRecord()
        mary_copy = copy.copy(mary)
        print("mary: ", mary)
        print("mary_copy:", mary_copy)
        print("-- grades are SAME id --")
        print("id(mary.grades) : ", id(mary.grades) )
        print("id(mary_copy.grades) :", id(mary_copy.grades) )
        self.drop_objection()

--
mary: Name: Mary, ID: 33 Grades: [97, 82]
mary_copy: Name: Mary, ID: 33 Grades: [97, 82]
-- grades are SAME id --
id(mary.grades) : 140369743774784
id(mary_copy.grades) : 140369743774784
```

This is a shallow copy. `mary` and `mary_copy` have a handle to the same list, so changing the list using `mary` will also change the list in `mary_copy`. Python resolves his problem using `copy.deepcopy()`.

```
# Figure 10: Making copies of all objects with copy.deepcopy()

@pyuvm.test()
class CopyDeepCopyTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        mary = StudentRecord("Mary", 33, [97, 82] )
        mary_copy = StudentRecord()
        mary_copy = copy.deepcopy(mary)
        <snipped print statements>

--
mary: Name: Mary, ID: 33 Grades: [97, 82]
mary_copy: Name: Mary, ID: 33 Grades: [97, 82]
-- grades are DIFFERENT id --
id(mary.grades) : 140369746999360
id(mary_copy.grades) : 140369746860096
```

`deepcopy()` creates a new instance of every object it copies, so changes to `mary.grades` no longer affect `mary_copy.grades`.

Using super() in methods

The `__str__()` method in figure 8 shows how we can use `super()` to leverage a method from a parent class. The `StudentRecord.__str__()` leverages `PersonRecord.__str__()` (defined in figure 2) by using the `super()` function to get the name and ID number string and then append the grades.

```
# Figure 11: Using super() to leverage the PersonRecord __str__() method

def __str__(self) :
    return super() .__str__() + f" Grades: {self.grades}"
```

We will take advantage of `super()` when we implement copying in the UVM.

Copying using the UVM

Unlike Python, SystemVerilog does not have a generic copying system, so the UVM defines one. The `uvm_object` class contains two methods that provide copying functionality:

- `uvm_object.copy(<other>)` —The `copy()` function copies the data fields from the `<other>` object into the calling object.
- `uvm_object.clone()` —The `clone()` function returns a new object that is a copy of the calling object.

Both `copy()` and `clone()` call the user-defined method `do_copy()` .

do_copy()

Both the `copy()` and `clone()` methods call a user-defined method named `do_copy()` . The `do_copy()` method implements copying. Figure 12 shows the `do_copy()` method in `PersonRecord` . It calls `super().do_copy(other)` to leverage the `do_copy()` implemented in `uvm_object` to copy the name, then it copies the data unique to itself.

```
# Figure 12: Overriding do_copy()

class PersonRecord(uvm_object) :
    def __init__(self, name="", id_number=None) :
        super() .__init__(name)
        self.id_number = id_number

    def do_copy(self, other) :
        super() .do_copy(other)
        self.id_number = other.id_number
```

When you override `do_copy()` , you should always call `super().do_copy(other)` at the beginning of your method. `StudentRecord` works the same way.

Figure 13: Using `super()` in `do_copy` to do a deep copy correctly

```
class StudentRecord(PersonRecord) :
    def __init__(self, name="", id_number=None, grades=[] ) :
        super() .__init__(name, id_number)
        self.grades = grades

    def do_copy(self, other) :
        super() .do_copy(other)
        self.grades = list(other.grades)
```

The version of `do_copy()` in figure 13 implements a deep copy by creating a new list and passing it to the `other.grades` list to initialize it. We could have copied `self.grades` to `other.grades` if we wanted to implement a shallow copy. Implementing a deep or shallow copy is your choice as the programmer.

The code in Figure 14 is an example of using `copy()`.

Figure 14: Using the `copy()` method in `uvm_object`

```
@pyuvm.test()
class CopyTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        mary = StudentRecord("Mary", 33, [97, 82] )
        mary_copy = StudentRecord()
        mary_copy.copy(mary)
        <snipped print statements>
```

Figure 15 shows an example of using `clone()`.

Figure 15: Using the `clone()` method in `uvm_object`

```
@pyuvm.test()
class CloneTest(uvm_test) :
    async def run_phase(self) :
        self.raise_objection()
        mary = StudentRecord("Mary", 33, [97, 82] )
        mary_copy = mary.clone()
        <snipped print statements>
```

Both tests return the same results so we print the output once in figure 16. We see that `copy()` and `clone()` are doing a deep copy because that's what we implemented in `do_copy()`.

Figure 16: Deep copies from the `copy()` method

```
mary: Name: Mary, ID: 33 Grades: [97, 82]
mary_copy: Name: Mary, ID: 33 Grades: [97, 82]
-- grades are different ids --
id(mary.grades) : 140369743867264
id(mary_copy.grades) : 140369746910144
```

Summary

This chapter examined methods that we normally implement in a class that extends `uvm_object`. We saw that Python replaces `convert2string()` with `__str__()` and `compare()` with `__eq__()`.

We examined copying and cloning and saw the difference between `copy.copy()` and `copy.deepcopy()`.

Finally, we learned how to implement `copy()` and `clone()` in the UVM by overriding `do_copy()`, though I recommend using the Python **copy** module functions.

Sequence testbench: 7.0

Testbench 6.0, our most advanced so far, used UVM components to build a testbench out of single-function components that communicated using TLM ports. We created different tests by overriding the `BaseTester` component.

Our next step is to create a testbench whose structure stays the same while generating different stimuli. This approach allows us to create new tests by writing many test programs rather than creating many testbench structures.

The UVM sequence ¹⁵ system separates stimulus from testbench structure and allows us to write test programs that run on an unchanging testbench.

UVM Sequence Overview

UVM sequences are objects that send commands to the DUT. UVM tests instantiate sequences and start them running using the UVM sequence system. The system consists of three parts.

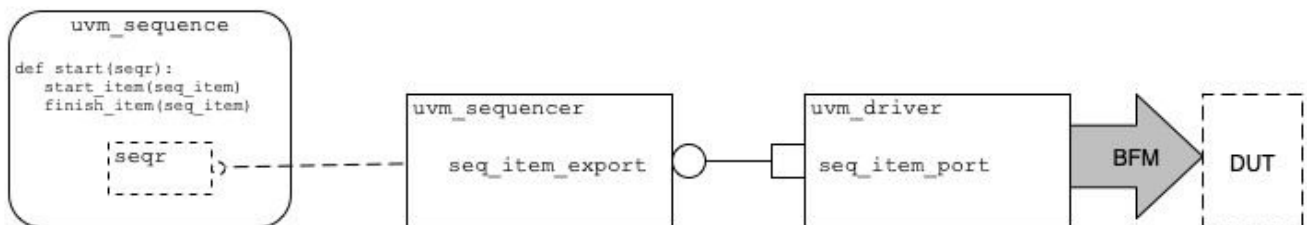


Figure 1: Sequence components and connections

- **UVM Sequence** —A `uvm_sequence` contains a `body()` coroutine that creates commands as `uvm_sequence_items`. A `uvm_sequence_item` is an object that contains all the information needed to send a command to the BFM. The sequence sends sequence items to DUT using coroutines named `start_item()` and `finish_item()`.
- **UVM Sequencer** —The `uvm_sequencer` takes UVM sequence items from one or more UVM sequences and sends them to the UVM driver. We pass the handle to the sequencer to a sequence when we start it.
- **UVM Driver** —The `uvm_driver` gets sequence items from a sequencer and sends them to the DUT using the BFM.

This chapter examines each of these pieces starting closest to the DUT with the `uvm_driver`.

Driver

Testbench 6.0 extended `uvm_driver` to create `Driver` but didn't use any driver features. We'll use those features now that we're implementing sequences.

The `uvm_driver` class contains a data member named `seq_item_port`, a blocking port that delivers the next sequence item. Here is the new TinyALU `Driver`. There are four differences between this version and the 6.0 version:

1. We get the `TinyAluBfm` in the `start_of_simulation_phase()` function instead of `build_phase()` , and we do not instantiate a get port. Instead we inherit `self.seq_item_port` from `uvm_driver` .
2. The `while` loop gets the next sequence item by awaiting `self.seq_item_port.get_next_item()` , storing the sequence item in `cmd` .
3. We use `cmd` in await `self.bfm.send_op(cmd.A, cmd.B, cmd.op)` .
4. We notify the sequencer that we've completed the operation by calling `self.seq_item_port.item_done()` .

Figure 2: The Driver refactored to work with sequences

```
class Driver(uvm_driver) :
    def start_of_simulation_phase(self) :
        self.bfm = TinyAluBfm()

    async def run_phase(self) :
        await self.bfm.reset()
        self.bfm.start_tasks()
        while True:
            cmd = await self.seq_item_port.get_next_item()
            await self.bfm.send_op(cmd.A, cmd.B, cmd.op)
            self.seq_item_port.item_done()
```

AluEnv

The `Driver` uses the inherited `self.seq_item_port` to get data from a source and notify the source when it is ready to get a new item. The source is called a sequencer . We instantiate and connect the `Driver` and `uvm_sequencer` in the `AluEnv` :

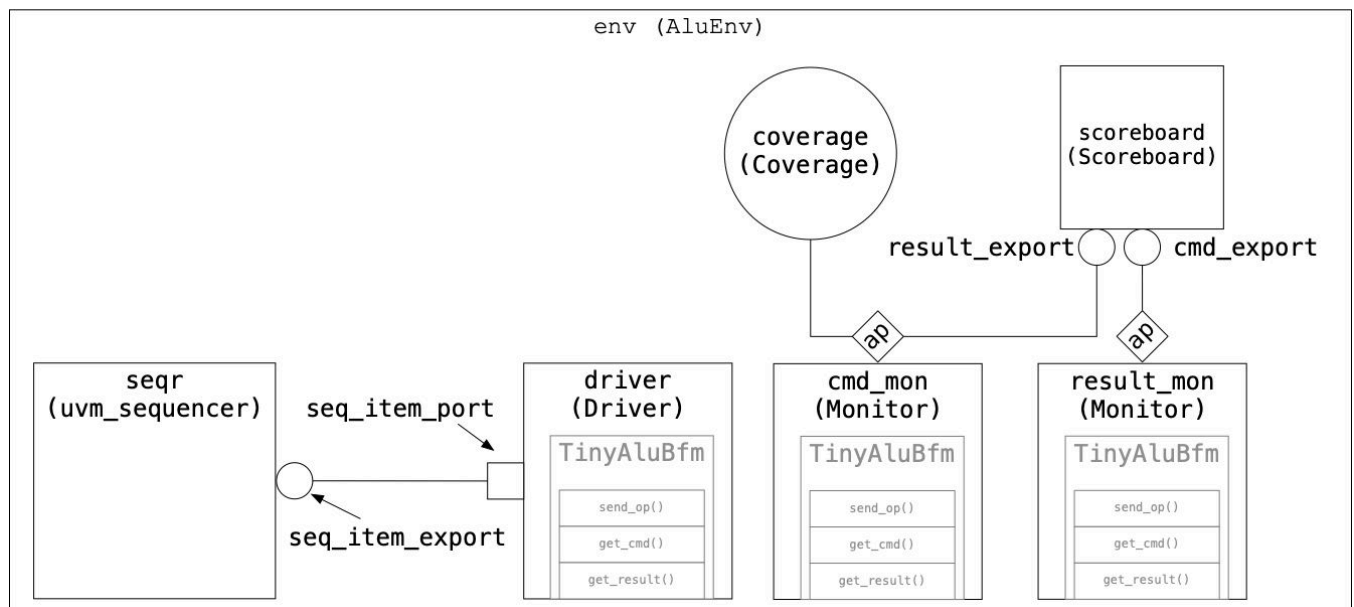


Figure 3: AluEnv with sequencer

Figure 3 shows that we no longer have the `cmd_fifo` . This is because the driver gets sequence items directly from the sequencer. Also, we don't extend `uvm_sequencer` , and we instantiate it as provided by **pyuvvm**.

In figure 4, the `AluEnv.build_phase()` instantiates the objects in figure 3. We instantiate `self.seqr` and immediately store its handle in the `ConfigDB()`. The UVM test will use the handle later to start a sequence.

Figure 4: Instantiating the sequencer in the environment

```
def build_phase(self) :
    self.seqr = uvm_sequencer("seqr", self)
    ConfigDB().set(None, "*", "SEQR", self.seqr)
    self.driver = Driver("driver", self)
    self.cmd_mon = Monitor("cmd_mon", self, "get_cmd")
    self.result_mon = Monitor("result_mon", self, "get_result")
    self.scoreboard = Scoreboard("scoreboard", self)
    self.coverage = Coverage("coverage", self)
```

Now that we have a sequencer and a driver, we connect `self.driver.seq_item_port` to `self.seqr.seq_item_export`. The other connections remain the same.

Figure 5: Connecting the sequencer to the driver

```
def connect_phase(self) :
    self.driver.seq_item_port.connect(
        self.seqr.seq_item_export)
    self.cmd_mon.ap.connect(self.scoreboard.cmd_export)
    self.cmd_mon.ap.connect(self.coverage.analysis_export)
    self.result_mon.ap.connect(self.scoreboard.result_export)
```

Now the driver can get sequence items from the sequencer. The sequencer will get them from the sequence. Next, we define the sequence item.

AluSeqItem

Until now, we've stored TinyALU commands in the `(A, B, op)` tuple. We'll now replace that object by extending `uvm_sequence_item`. A UVM sequence item holds the information the BFM needs to execute a command and can hold the resulting data from the command.

Here, we create an `AluSeqItem` that extends `uvm_sequence_item`. We override `__init__` to force the user to provide the command data. We call `super().__init__()` because **pyuvvm** requires it. We'll break the `AluSeqItem` definition into three pieces.

The `__init__()` method takes the operands `aa`, `bb`, and operation `op` as inputs and stores them in the sequence item. We use `op` to create a new `ops()` object.

Figure 6: Defining an ALU command as a sequence item

```
class AluSeqItem(uvm_sequence_item) :  
  
    def __init__(self, name, aa, bb, op) :  
        super() .__init__(name)  
        self.A = aa  
        self.B = bb  
        self.op = Ops(op)
```

We add the `__eq__()` and `__str__()` magic methods to check the sequence item against another one and print it.

Figure 7: The `__eq__` and `__str__` methods in a sequence item

```
def __eq__(self, other) :  
    same = self.A == other.A and self.B == other.B and self.op == other.op  
    return same  
  
def __str__(self) :  
    return f"{self.get_name() } : A: 0x{self.A:02x} \  
    OP: {self.op.name} ({self.op.value}) B: 0x{self.B:02x}"
```

Creating sequences

Starting with testbench 3.0, we've created a class structure using a base object that generated ALU commands and sent them to the TinyALU and a random and max extension that defined the operands. So the class structure is back, but this time the testbench remains unchanged as we use different sequences to send different commands.

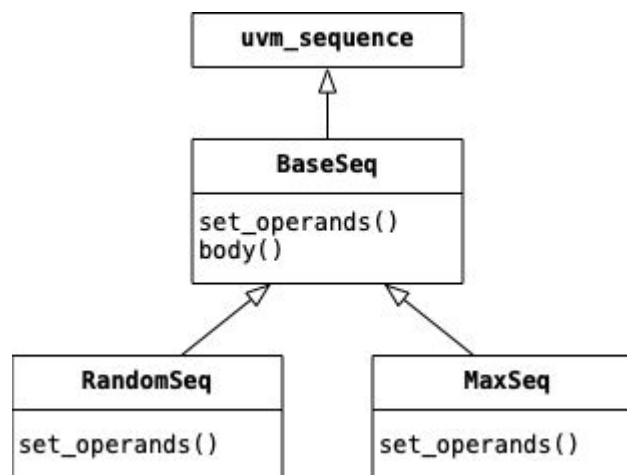


Figure 8: Sequence class diagram

BaseSeq

A sequence contains a `body()` coroutine that creates sequence items and sends them to the DUT. `BaseSeq` loops through the operations, generating sequence items using them and sending the items to the sequencer using `start_item()` and `finish_item()`.

Figure 9: The BaseSeq contains the common body() method

```
class BaseSeq(uvm_sequence) :  
  
    async def body(self) :  
        for op in list(Ops) :  
            cmd_tr = AluSeqItem("cmd_tr", 0, 0, op)  
            await self.start_item(cmd_tr)  
            self.set_operands(cmd_tr)  
            await self.finish_item(cmd_tr)  
  
    def set_operands(self, tr) :  
        pass
```

The example illustrates how to define a sequence by looping through all the operations and randomizing the operands. Here are the sequence-specific elements:

- `class BaseSeq(uvm_sequence)` —We extend `uvm_sequence` .
- `await self.start_item(cmd_tr)` — `BaseSeq` inherited the `self.start_item()` coroutine. We `await self.start()` and pass it the sequence item. It will return when the driver is ready to use the sequence item.
- `self.set_operands(cmd_tr)` —This functions sets the operands in `cmd_tr` . It does nothing in `BaseSeq` because it is supposed to be overridden. This code illustrates *late stimulus setting* , which means we don't set the stimulus until the driver is ready to receive the item. Some testbenches need the latest state of the system to set the stimulus properly, and this feature supports that need.
- `await self.finish_item(cmd_tr)` — `self.finish_item()` sends the sequence item to the driver through the sequencer and returns when the driver `calls self.seq_item_port.item_done()` .

This example is the simplest way to write a sequence. Later in the chapter, we will look at sequences that get data from the DUT.

RandomSeq and MaxSeq

`RandomSeq` and `MaxSeq` extend `BaseSeq` and inherit its `body()` method. The `body()` method passes a handle to a transaction, and the functions use the handle to set the operands. Since these functions and the `body()` method all have the same handle, modifications in these methods set the operands for `body()` .

Figure 10: Extending BaseSeq to create the random and maximum stimulus

```
class RandomSeq(BaseSeq) :  
    def set_operands(self, tr) :  
        tr.A = random.randint(0, 255)  
        tr.B = random.randint(0, 255)  
  
class MaxSeq(BaseSeq) :  
    def set_operands(self, tr) :  
        tr.A = 0xff  
        tr.B = 0xff
```

Starting a sequence in a test

We usually start sequences in a UVM test's run phase. We create the sequence and then call its `start()` method, passing a handle to the sequencer.

BaseTest

`BaseTest` instantiates the environment in `build_phase()`, and then uses the `end_of_elaboration_phase()` method to get a handle to the sequencer.

Figure 11: All tests use the same environment and need a sequence

```
class BaseTest(uvm_test) :
    def build_phase(self) :
        self.env = AluEnv("env", self)

    def end_of_elaboration_phase(self) :
        self.seqr = ConfigDB().get(self, "", "SEQR")
```

The `run_phase()` creates a sequence using the factory. Then it awaits `seq.start(self.seqr)` to start the sequence. The `start()` coroutine returns when the sequence has been completed. We then use `clockcycles()` to wait until all the operations are complete.

Figure 12: All tests start the sequence

```
async def run_phase(self) :
    self.raise_objection()
    seq = BaseSeq.create("seq")
    await seq.start(self.seqr)
    await clockcycles(cocotb.top.clk, 50) # to do last transaction
    self.drop_objection()
```

The `BaseTest` runs with all zeros.

Figure 13: The base test uses all zeros

```
INFO testbench.py(116) [<snip>] : PASSED: 0x00 ADD 0x00 = 0x0000
INFO testbench.py(116) [<snip>] : PASSED: 0x00 AND 0x00 = 0x0000
INFO testbench.py(116) [<snip>] : PASSED: 0x00 XOR 0x00 = 0x0000
INFO testbench.py(116) [<snip>] : PASSED: 0x00 MUL 0x00 = 0x0000
INFO testbench.py(87) [<snip>] : Covered all operations
```

We see that the TinyALU works properly with all zeros as data. The zeros are from the `BaseSeq`.

RandomTest and MaxTest

`RandomTest` overrides `BaseSeq` with `RandomSeq` in the factory and inherits the `run_phase()` coroutine.

Figure 14: Overriding BaseSeq to get random stimulus and all ones

```
@pyuvm.test()
class RandomTest(BaseTest) :
    def start_of_simulation_phase(self) :
        uvm_factory().set_type_override_by_type(BaseSeq, RandomSeq)

@pyuvm.test()
class MaxTest(BaseTest) :
    def start_of_simulation_phase(self) :
        uvm_factory().set_type_override_by_type(BaseSeq, MaxSeq)
```

`RandomTest` runs with random data. The component path is snipped.

Figure 15: Passing random tests

```
INFO testbench.py(116) [<snip>] : PASSED: 0x36 ADD 0x1c = 0x0052
INFO testbench.py(116) [<snip>] : PASSED: 0xbc AND 0x2f = 0x002c
INFO testbench.py(116) [<snip>] : PASSED: 0x48 XOR 0xdd = 0x0095
INFO testbench.py(116) [<snip>] : PASSED: 0x02 MUL 0x4f = 0x009e
INFO testbench.py(87) [<snip>] : Covered all operations
```

`MaxTest` also uses a factory override to run `MaxSeq`.

Figure 16: Passing all ones tests

```
INFO testbench.py(116) [<snip>] : PASSED: 0xff ADD 0xff = 0x01fe
INFO testbench.py(116) [<snip>] : PASSED: 0xff AND 0xff = 0x00ff
INFO testbench.py(116) [<snip>] : PASSED: 0xff XOR 0xff = 0x0000
INFO testbench.py(116) [<snip>] : PASSED: 0xff MUL 0xff = 0xfe01
INFO testbench.py(87) [<snip>] : Covered all operations
```

Summary

This chapter created testbench 7.0, which used UVM sequences to test the TinyALU. We learned about the `uvm_sequencer` and `uvm_driver` and how the `uvm_driver` uses the `seq_item_port` to get sequence items from the `uvm_sequencer`.

We learned to store sequencer handles in the `ConfigDB()` so that other parts of the testbench, such as tests, can get them and use them to start sequences.

Finally, we created sequences that generated the random and max operands that we've seen through the book.

Now that we've created testbench 7.0, we will tweak it to demonstrate different aspects of sequence behavior. These examples will give us testbench 7.1 and 7.2.

Fibonacci testbench: 7.1

Testbench 7.0 is a full-fledged UVM testbench. We've modularized its functions, connected components together to test the DUT, and now are using sequences to provide stimulus.

`Driver` in Testbench 7.0 acts like the `BaseTester` class in Testbench 2.0 in that it sends commands into the DUT and doesn't worry about the result. Instead, the testbench uses the monitors to gather the commands and the results and send them to the scoreboard and coverage components.

Sending sequence items into the DUT and ignoring the result does not work for all testbenches because sometimes the data in one sequence item depends upon the results of a previous one. Testbench 7.1 will implement such a test by using the TinyALU to generate the Fibonacci sequence.

Fibonacci numbers

We first met Fibonacci numbers in the *Generators* chapter (so long ago)! The Fibonacci sequence starts with 0 and 1 then adds two Fibonacci numbers to get the following Fibonacci number. Testbench 7.1 logs the first 9 Fibonacci numbers as a list: `[0, 1, 1, 2, 3, 5, 8, 13, 21]`.

Testbench 7.1 generates the Fibonacci numbers using TinyALU's `ops.ADD` operation. So, we need to modify the driver to get the sum from the TinyALU before sending the next command.

FibonacciSeq

`FibonacciSeq` is a `uvm_sequence` whose `body()` method prints the first nice Fibonacci numbers. It creates a list of the numbers and then prints the list. First, we initialize `prev_num` and `cur_num` to the first two Fibonacci numbers, 0 and 1, and store them in `fib_list`. Then we instantiate an add operation by calling `AluSeqItem(0, 0, ops.ADD)` and storing the handle in `cmd`. The handle does not change throughout the program since we reuse the sequence item for all commands.

Figure 1: Setting up the Fibonacci pattern

```
class FibonacciSeq(uvm_sequence) :
    async def body(self) :
        prev_num = 0
        cur_num = 1
        fib_list = [prev_num, cur_num]
        cmd = AluSeqItem("cmd", None, None, ops.ADD)
```

We have two Fibonacci numbers. Now we'll generate seven more using the TinyAlu. We await `self.start_item(cmd)`. When the driver is ready, we store `prev_num` and `cur_num` as operands. Then we await `self.finish_item(cmd)`, and when the coroutine returns, a miracle happens: `cmd.result` contains the sum of the numbers. We append the result to `fib_list`, move `cur_num` to `prev_num` and `cmd.result` to `cur_num`. Then we go back to the top of the loop.

```
# Figure 2: Generating Fibonacci numbers
```

```
for _ in range(7) :  
    await self.start_item(cmd)  
    cmd.A = prev_num  
    cmd.B = cur_num  
    await self.finish_item(cmd)  
    fib_list.append(cmd.result)  
    prev_num = cur_num  
    cur_num = cmd.result
```

Now that we're through the loop, we log the Fibonacci numbers. The `uvm_sequence` class does not extend `uvm_report_object`, so it has no logger. Instead, we use the logger in the `uvm_root()` singleton.

```
# Figure 3: Logging messages from a sequence
```

```
uvm_root().logger.info("Fibonacci Sequence: " + str(fib_list))
```

```
--
```

```
INFO testbench.py(46) [] : Fibonacci Sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

```
INFO fibonaaci_test pass
```

This sequence works because the `Driver` stores the result in the `AluSeqItem` using the shared handle. Next, we'll look at the `Driver` code to see how it does this.

Driver

The 7.1 `Driver` does three things that the 7.0 `Driver` does not:

1. It gets the result from `TinyAluBfm` using `get_result()`.
2. It writes the result to an analysis port.
3. It copies the result into the sequence item so the sequence can use it.

The code in figure 4 creates the analysis port and gets the handle to `TinyAluBfm`.

```
# Figure 4: Adding an analysis port to the Driver
```

```
class Driver(uvm_driver) :  
    def build_phase(self) :  
        self.ap = uvm_analysis_port("ap", self)  
  
    def start_of_simulation_phase(self) :  
        self.bfm = TinyAluBfm()
```

The `run_phase()` coroutine resets the DUT, starts the BFM tasks, then starts a `while` loop.

First, the `while` loop gets `cmd` by awaiting `self.seq_item_port.get_next_item()`. Next, it sends `cmd` to the TinyALU. Finally, it awaits `self.bfm.get_result()` to get the result.

Figure 5: The run phase gets a result back from the BFM

```
async def run_phase(self) :
    await self.bfm.reset()
    self.bfm.start_tasks()
    while True:
        cmd = await self.seq_item_port.get_next_item()
        await self.bfm.send_op(cmd.A, cmd.B, cmd.op)
        result = await self.bfm.get_result()
```

Now that the `Driver` has `result` it writes it to the analysis port, copies it into `cmd` , and calls `self.seq_item_port.item_done()` to allow the sequence to return from `finish_item()` .

Figure 6: Writing the result into the sequence item

```
self.ap.write(result)
cmd.result = result
self.seq_item_port.item_done()
```

Sequence timing

Python for RTL Verification

1. The sequence awaits `start_item()` and is now blocked, waiting for the sequencer.
2. The driver awaits `self.seq_item_port.get_next_item()` . This allows the sequence to move past `start_item()` .
3. The sequence awaits `finish_item()` . This sends the sequence item to `seq_item_port` and blocks the sequence.
4. The driver completes its work and calls `self.seq_item_port.item_done()` . This allows `finish_item()` to return and the sequence continues.

AluEnv

We modified `Driver` to get a command's result and return it to the calling sequence. The `Driver` also creates an analysis port to provide the results to the `scoreboard` . We connect `self.scoreboard.result_export` to the `Driver` as this diagram shows.

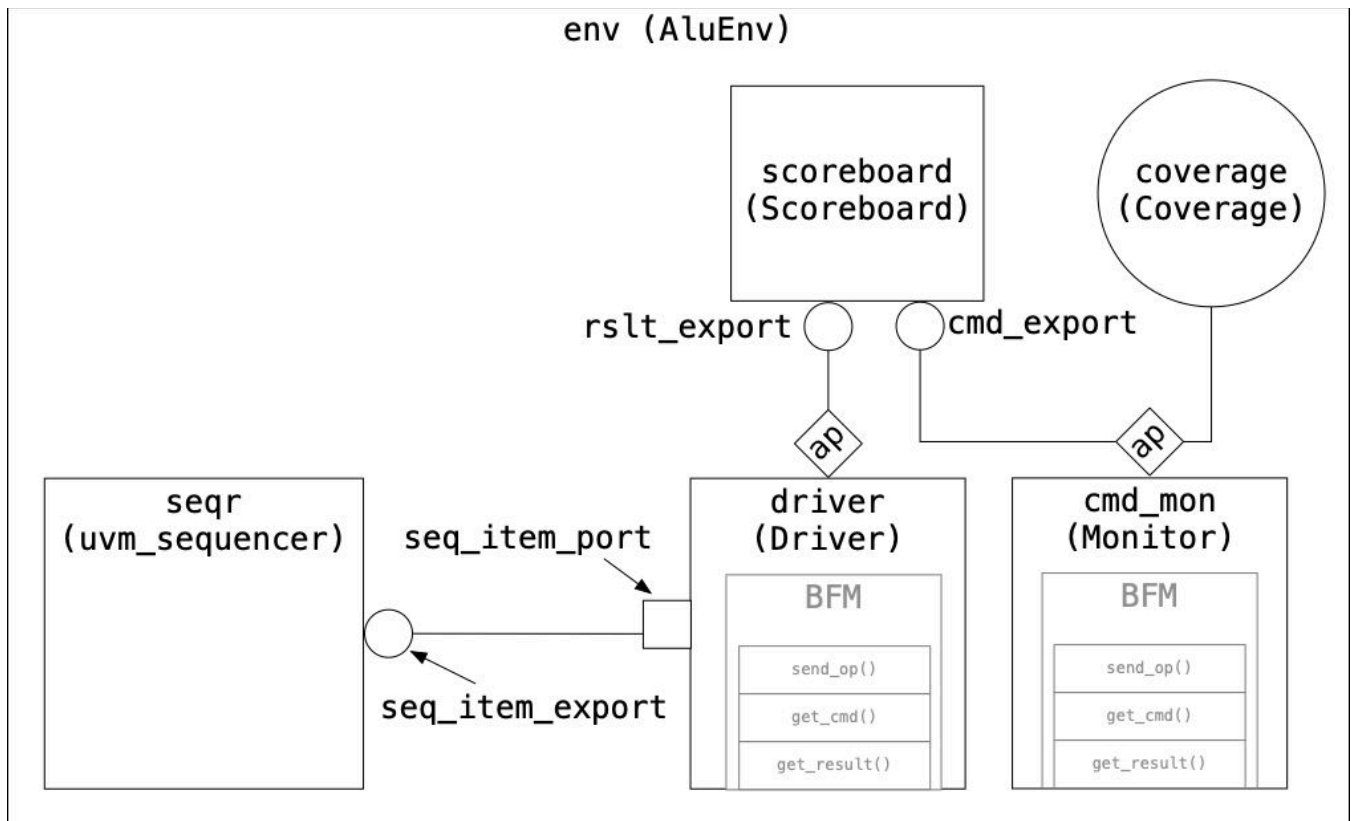


Figure 7: AluEnv with driver getting results

The 7.1 `build_phase()` in figure 8 does not instantiate a result monitor.

```
# Figure 8: AluEnv environment without a result monitor

class AluEnv(uvm_env) :

    # ## Connecting the driver to the sequencer
    def build_phase(self) :
        self.seqr = uvm_sequencer("seqr", self)
        ConfigDB().set(None, "*", "SEQR", self.seqr)
        self.driver = Driver("driver", self)
        self.cmd_mon = Monitor("cmd_mon", self, "get_cmd")
        self.scoreboard = Scoreboard("scoreboard", self)
        self.coverage = Coverage("coverage", self)
```

The 7.1 `connect_phase()` in figure 9 connects the `result_export` to the driver's analysis port.

```
# Figure 9: Connecting the Driver analysis port

def connect_phase(self) :
    self.driver.seq_item_port.connect(
        self.seqr.seq_item_export)
    self.cmd_mon.ap.connect(self.scoreboard.cmd_export)
    self.cmd_mon.ap.connect(self.coverage.analysis_export)
    self.driver.ap.connect(self.scoreboard.result_export)
```

Summary

In Testbench 7.1, we modified `Driver` to get the result of its operations and write them into the `AluSeqItem` handle so the sequence could use the data.

The UVM defines another way to pass data back to the sequence from the driver. We'll examine that approach in Testbench 7.2.

get_response() testbench: 7.2

The UVM allows you to return *result* sequence items from the driver to the sequence. First, the driver instantiates a result sequence item containing the result then sends it to the sequence using `seq_item_port.item_done()`. Finally, The sequence gets the result sequence item by awaiting `get_response()`.

Testbench 7.2 modifies `Driver` and `FibonacciSeq` to use `get_response()`. We also define the result sequence item.

AluResultItem

The `get_response()` system expects the testbench to create a new sequence item containing the return data.

We must return a `uvm_sequence_item` because **pyuvm** uses data in that class to send the correct response to the correct sequence.

Here is `AluResultItem`.

Figure 1: A sequence item that stores a result

```
class AluResultItem(uvm_sequence_item) :
    def __init__(self, name, result) :
        super().__init__(name)
        self.result = result

    def __str__(self) :
        return f"RESULT: {self.result}"
```

Driver

The Testbench 7.2 `Driver`, like the 7.1 `Driver`, instantiates an analysis port, sends commands to the TinyALU, and awaits `self.bfm.get_result()`. Once it has a result, it writes it to the analysis port.

Figure 2: The driver getting the result from the monitor

```
class Driver(uvm_driver) :
    <snipped unchanged code>

    async def run_phase(self) :
        await self.bfm.reset()
        self.bfm.start_tasks()
        while True:
            cmd = await self.seq_item_port.get_next_item()
            await self.bfm.send_op(cmd.A, cmd.B, cmd.op)
            result = await self.bfm.get_result()
            self.ap.write(result)
```

`Driver` sends the result back to the sequence using three steps:

1. It creates a new `AluResultItem` object using `result`.

2. It uses `result_item.set_id_info(cmd)` to indicate that `result_item` is the response to `cmd`.
3. It calls `self.seq_item_port.item_done(result_item)` to return the item.

Figure 3: Sending the result item back

```
result_item = AluResultItem("result_item", result)
result_item.set_id_info(cmd)
self.seq_item_port.item_done(result_item)
```

Next, we see how the sequence gets the data.

FibonacciSeq

The Testbench 7.2 `FibonacciSeq` in figure 4 is identical to the 7.1 version though awaiting `finish_item()`.

Figure 4: Sending commands to the TinyALU

```
class FibonacciSeq(uvm_sequence) :
    async def body(self) :
        prev_num = 0
        cur_num = 1
        fib_list = [prev_num, cur_num]
        cmd = AluSeqItem("cmd", None, None, Ops.ADD)
        for _ in range(7) :
            await self.start_item(cmd)
            cmd.A = prev_num
            cmd.B = cur_num
            await self.finish_item(cmd)
```

As we see in figure 5, once `finish_item()` returns, the sequence awaits `get_response()` to get the `AluResultItem` the driver created. It writes `rsp.result` into the analysis port and stores it in `cur_num`.

Figure 5: Calling `get_response()` to get the response

```
rsp = await self.get_response()
fib_list.append(rsp.result)
prev_num = cur_num
cur_num = rsp.result
uvm_root().logger.info("Fibonacci Sequence: " + str(fib_list))
--
INFO testbench.py(48) [] : Fibonacci Sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

get_response() pitfalls

We've examined two ways the driver can return results to a sequence:

1. Store the result in the sequence item using the shared sequence item handle.
2. Create a response item and send it back to the sequence using `item_done()`.

Writing to the shared sequence item handle is the cleaner of the two approaches and easier to use. In addition it avoids a pitfall with `get_response()`.

In the TinyALU testbench, all operations create a response. However, this isn't true of all designs. For example, a RAM may only send a response when you do a READ operation since a WRITE operation doesn't need a response.

A sequence that tests this RAM must only call `get_response()` on read because it will hang if it calls `get_response()` on a write and the driver doesn't send a response.

One can get around this by having the driver *always* send back something, but that is a needless complication we don't need if we return data using item handles.

Summary

This chapter taught us the `get_response()` approach for a sequence to get a result back from a driver.

Using `get_response()` requires that the driver instantiate a new sequence item and call `set_id_info` to indicate that the new sequence item is the response to the item that came from `seq_item_port.get_item()`.

We discussed the pitfalls of the `get_response()` approach, and I recommended using the shared handle approach illustrated in Testbench 7.1.

Virtual sequence testbench: 8.0

We launched sequences in testbench 7.x by passing the handle to a sequencer. The `start_item()` and `finish_item()` inherited coroutines used the sequencer to send sequence items to the driver.

However, it's possible to launch a sequence without a sequencer. A sequence you start without a sequencer is called a virtual sequence. The *virtual sequence* does not use `start_item()` or `finish_item()`. Instead, it launches other sequences. We can use virtual sequences to create programming interfaces that make it easy to write test programs.

Launching sequences from a virtual sequence

This example runs both the `RandomSeq` and `MaxSeq` from a single sequence named `TestAllSeq`. We instantiate the `TestAllSeq` and run it from `AluTest`. Since `TestAllSeq` is a virtual sequence, `run_phase()` calls `self.test_all.start()` without a sequencer argument.

Figure 1: Calling a virtual sequence

```
@pyuvm.test()
class AluTest(uvm_test) :
    def build_phase(self) :
        self.env = AluEnv("env", self)

    def end_of_elaboration_phase(self) :
        self.test_all = TestAllSeq.create("test_all")

    async def run_phase(self) :
        self.raise_objection()
        await self.test_all.start()
        self.drop_objection()
```

The `body()` coroutine in a virtual sequence looks like a program as it doesn't create items or call `start_item()` or `finish_item()`. In this case, `body()` launches our two TinyALU test sequences by getting the sequencer handle from the `ConfigDB()` and using it to await `start()`.

Figure 2: A virtual sequence starts other sequences

```
class TestAllSeq(uvm_sequence) :

    async def body(self) :
        seqr = ConfigDB().get(None, "", "SEQR")
        rand_seq = RandomSeq("random")
        max_seq = MaxSeq("max")
        await rand_seq.start(seqr)
        await max_seq.start(seqr)
```

When we run this, we see random operands, then `0xFF` operands.

Figure 3: Running RandomSeq then MaxSeq

```
INFO testbench.py(231) [<snip>] : PASSED: 0xa0 ADD 0x59 = 0x00f9
INFO testbench.py(231) [<snip>] : PASSED: 0xec AND 0xf7 = 0x00e4
INFO testbench.py(231) [<snip>] : PASSED: 0xdf XOR 0x7c = 0x00a3
INFO testbench.py(231) [<snip>] : PASSED: 0xd6 MUL 0x8a = 0x735c
INFO testbench.py(231) [<snip>] : PASSED: 0xff ADD 0xff = 0x01fe
INFO testbench.py(231) [<snip>] : PASSED: 0xff AND 0xff = 0x00ff
INFO testbench.py(231) [<snip>] : PASSED: 0xff XOR 0xff = 0x0000
INFO testbench.py(231) [<snip>] : PASSED: 0xff MUL 0xff = 0xfe01
INFO testbench.py(202) [<snip>] : Covered all operations
```

Running sequences in parallel

A virtual sequence can launch sequences in parallel using the `cocotb.start_soon()` function. The function returns handles to the tasks, and we await their completion using `Combine(random_task, max_task)`.

Figure 4: Running RandomSeq and MaxSeq in parallel

```
class TestAllParallelSeq(uvm_sequence) :

    async def body(self) :
        seqr = ConfigDB().get(None, "", "SEQR")
        random_seq = RandomSeq("random")
        max_seq = MaxSeq("max")
        random_task = cocotb.start_soon(random_seq.start(seqr) )
        max_task = cocotb.start_soon(max_seq.start(seqr) )
        await Combine(random_task, max_task)
```

The sequencer runs both sequences in parallel, alternating sequence items between them. The result is that we see random operands interspersed with maximum operands.

Figure 5: Interleaved results from parallel sequences

```
INFO testbench.py(231) [<snip>] : PASSED: 0x28 ADD 0x8c = 0x00b4
INFO testbench.py(231) [<snip>] : PASSED: 0xff ADD 0xff = 0x01fe
INFO testbench.py(231) [<snip>] : PASSED: 0x5d AND 0xa5 = 0x0005
INFO testbench.py(231) [<snip>] : PASSED: 0xff AND 0xff = 0x00ff
INFO testbench.py(231) [<snip>] : PASSED: 0x2a XOR 0xfa = 0x00d0
INFO testbench.py(231) [<snip>] : PASSED: 0xff XOR 0xff = 0x0000
INFO testbench.py(231) [<snip>] : PASSED: 0xc0 MUL 0x21 = 0x18c0
INFO testbench.py(231) [<snip>] : PASSED: 0xff MUL 0xff = 0xfe01
INFO testbench.py(202) [<snip>] : Covered all operations
```

Creating a programming interface

Virtual sequences allow testbench writers to create programming interfaces that make it easier to write tests. Such interfaces are helpful on projects where the testbench creators and test writers are on different teams. A typical programming interface allows test writers to send commands to different interfaces on the testbench and wait for the results. Let's create a programming interface for the TinyALU.

OpSeq

The key to a programming interface is to capture repeated operations in one sequence. For example, given `A`, `B`, and `op`, the `OpSeq` sequence creates a sequence item and sends it to the TinyALU.

Figure 6: A sequence that can run any operation

```
class OpSeq(uvm_sequence) :
    def __init__(self, name, aa, bb, op) :
        super() .__init__(name)
        self.aa = aa
        self.bb = bb
        self.op = Ops(op)

    async def body(self) :
        seq_item = AluSeqItem("seq_item", self.aa, self.bb,
                               self.op)

        await self.start_item(seq_item)
        await self.finish_item(seq_item)
        self.result = seq_item.result
```

TinyALU programming interface

We use `OpSeq` to create a TinyALU command interface. The programmer passes these coroutines a sequencer handle and the operands. Passing the sequencer handle allows these commands to work with multiple TinyALUs in the testbench. There is one command for each operation.

Figure 7: The programming interface coroutines

```
async def do_add(seqr, aa, bb) :
    seq = OpSeq("seq", aa, bb, Ops.ADD)
    await seq.start(seqr)
    return seq.result

async def do_and(seqr, aa, bb) :
    seq = OpSeq("seq", aa, bb, Ops.AND)
    await seq.start(seqr)
    return seq.result

async def do_xor(seqr, aa, bb) :
    seq = OpSeq("seq", aa, bb, Ops.XOR)
    await seq.start(seqr)
    return seq.result

async def do_mul(seqr, aa, bb) :
    seq = OpSeq("seq", aa, bb, Ops.MUL)
    await seq.start(seqr)
    return seq.result
```

FibonacciSeq

Now we can generate Fibonacci numbers using the `do_add()` command.

```
# Figure 8: The Fibonacci program written using
# the programming interface

class FibonacciSeq(uvm_sequence) :
    def __init__(self, name) :
        super().__init__(name)
        self.seqr = ConfigDB().get(None, "", "SEQR")

    async def body(self) :
        prev_num = 0
        cur_num = 1
        fib_list = [prev_num, cur_num]
        for _ in range(7) :
            sum = await do_add(self.seqr, prev_num, cur_num)
            fib_list.append(sum)
            prev_num = cur_num
            cur_num = sum
        uvm_root().logger.info("Fibonacci Sequence: " + str(fib_list))
--
INFO Fibonacci Sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

Summary

In this chapter, we learned about *virtual* sequences . These are sequences you `start()` without passing them a sequencer. Instead, these sequences typically create other sequences and pass them a handle to the sequencer.

We saw how virtual sequences allow us to combine many sequences into one. We saw how to run sequences sequentially and in parallel using `cocotb.start_soon()` .

Finally, we saw how we could use virtual sequences to create programming interfaces that hide the details of a testbench from the test writer. This abstract programming makes writing and maintaining tests for complex systems easier.

The future of Python in verification

We've seen that Python brings considerable advantages to the IC verification problem, and it's natural to wonder whether Python will replace existing languages such as SystemVerilog.

The short answer is No .

The EDA industry has never turned its back on a computer language that supports a significant ecosystem. Even the ancient language e is running today in large testbenches due to the e Verification Component (e VC) library.

This dynamic tells us that Python needs access to a verification ecosystem if it is going to become a mainstream verification language. It can get such access in two ways: leverage the SystemVerilog ecosystem and create a Python verification ecosystem.

Leverage the existing SystemVerilog ecosystem.

Python and SystemVerilog testbenches mostly live in different worlds at the time of this writing. This needs to change if Python is to become a mainstream testbench and test writing language.

SystemVerilog provides an enormous library of Verification IP, and Python will need to leverage that library to interact with many new and popular interfaces.

Python needs a library that allows users to control SystemVerilog components using DPI-C function calls. Such a library would provide a way to transfer objects between Python and SystemVerilog and provide a polling mechanism that allows Python to check transaction statuses using function calls (which don't block.)

Create a Python verification ecosystem.

The **cocotb** ecosystem already features at least one AXI interface module, **cocotbext-axi** . There is room for many more. As the EDA verification community embraces the developer ethos of sharing open-source resources, I hope to see more verification IP written in Python using **cocotb** transactors.

Of course, the Python ecosystem could go beyond transactors. Modern designs that leverage machine learning and artificial intelligence will need stimulus-generation techniques other than existing constrained-random methodologies. With its extensive, existing AI/ML modules, numerical analysis tools, and other advanced resources, Python could provide the foundation for testing future designs.

Bringing EDA into the 21st century

As we saw at the beginning of the book, the verification community is well represented by technologies such as SystemVerilog and the UVM that had their roots in the 20th century.

A new generation of developers has entered the workforce in the twenty years since constrained-random verification technologies were developed. These new developers are attuned to modern development techniques such as agile and modern tech stacks featuring editors such as Visual Studio Code.

The days of Emacs/VI dominance have passed as today's developers embrace software development techniques for EDA and bring the power of open-source development to EDA.

As designs become more complicated and adopt technologies such as machine learning and artificial intelligence, I hope that Python, **cocotb**, and **pyuvvm** enable a new generation of engineers to tackle the verification challenges ahead.

Acknowledgements

It would have been impossible to create this book without many people's help.

Thank you to my wife Karen Salemi who not only provides moral support for my writing efforts but is also an outstanding editor. Karen tirelessly worked on this book to guide me in making it as understandable and as easy to read as possible. If you found the book clear and helpful, thank Karen. If the book is confusing in spots, that's on me.

A book such as this depends upon diligent beta readers to find errors in grammar, technology, and explanations. I was blessed with a generous group of beta readers. Kaleb Barrett, Colin Marquardt, and Jon Povey ensured not only that the **cocotb** chapters were correct, but that the Python explanations were clear and accurate.

Tom Fitzpatrick of Siemens EDA, helped me clean up the grammar, explanations, and writing, as he has for many of my books (including the Tucker mysteries.)

Bob Oden reviewed the first draft, which is always yeoman's work. His comments and reactions assured me that the book would be timely and helpful.

Umut Guvenc caught explanations that didn't hit home, while my Siemens EDA cohorts David Aerne, Amr Baher, Tom Franco, Amr Hany, Robert Jeffrey, Amr Okasha, and Dave Rich provided corrections and convinced me to add figure numbers.

Dave Rich and Tom Fitzpatrick also helped me capture the history of SystemVerilog correctly.

This book would not exist and would be much the poorer without everyone's help.

Thank you!

Appendix

Other Books by Ray Salemi

Leading after a Layoff (McGraw-Hill)

FPGA Simulation (Boston Light Press)

Robot Haiku (Adams Media)

The UVM Primer (Boston Light Press)

The Tucker Mysteries (Writing as Ray Daniel)

The Tucker mysteries are first-person, wise-cracking, Boston-based mysteries featuring Tucker, a hacker who gets pulled into solving murders despite his lack of a gun or even fighting ability. There are four books in the series all written by Ray Daniel and published by Midnight Ink.

Terminated

Corrupted Memory

Child Not Found

Hacked

Copyright

© 2022, Ray Salemi. All rights reserved. No part of this book (other than the source code examples, as the MIT License protects them) may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, scanning to PDF, recording, or by any information storage and retrieval system without permission in writing from the publisher. For information, please contact Boston Light Press at www.bostonlightpress.com.

License

All source code in this book is protected by the following MIT license:

MIT License

Copyright (c) 2022 Ray Salemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Indemnification

Although the author and publisher have made every effort to ensure the accuracy and completeness of the information contained in this book, we assume no responsibility for errors, inaccuracies, omissions, or any inconsistency herein. Any slights of people, places, or organizations are unintentional.

"Python" and the Python Logo are trademarks of the Python Software Foundation.

-
1. [MIL-STD-454N](#) [↵](#)
 2. It should be noted that UVM has no relationship to the UVM other than starting with the letters UV. [↵](#)
 3. The formatting characters make up the *Format Specification Mini-Language* and it is fully documented on [docs.python.com](#). [↵](#)
 4. We have seen several of these common operations in previous chapters. [↵](#)
 5. As of Python 3.7. [↵](#)
 6. The Fibonacci sequence adds two numbers to get the next number in the sequence. $0 + 1 = 1$, $1 + 1 = 2$, $1 + 2 = 3$, etc. [↵](#)
 7. A change approved by Ron Swanson of *Parks and Recreation*. [↵](#)
 8. Metaclasses are beyond the scope of this book, so we'll simply use the argument without explanation. [↵](#)
 9. We now move from using Jupyter notebooks to hold the examples to simulation directories. Instructions are in `python4uvm_examples`. [↵](#)
 10. I've often thought the `initial` block in SystemVerilog should be called a `once` block in contrast with the `always` block. [↵](#)
 11. This is like `join` keyword in SystemVerilog. [↵](#)
 12. This is like `join_any` in SystemVerilog. [↵](#)
 13. The *Modules* chapter shows how we import `tinyalu_utils` from our parent directory. [↵](#)
 14. As large as memory allows. [↵](#)
 15. Do not confuse UVM sequences with the Python sequences. Same word, different meanings. [↵](#)